

ОГЛАВЛЕНИЕ

1. Вводные требования и границы системы	14
1.1. Роли и сценарии использования (моноплата / мультиплата)	14
1.1.1. Общая модель системы	14
1.1.2. Контроллер дверей (Door Controller)	14
1.1.3. Режим Standalone (одиночная плата)	14
1.1.4. Режим MASTER–SLAVE (мультиплатная система)	14
1.1.5. Поведение системы при потере CAN-сети (режим Degraded)	15
1.2. Термины и определения состояний двери	15
1.2.1. Физические элементы двери	15
1.2.2. Типы дверей по принципу блокировки	15
1.2.3. Физические состояния двери	15
1.2.4. Состояние двери (блокировка)	16
1.2.5. Логические состояния двери	16
1.2.6. События двери	16
1.2.7. Приоритеты состояний и событий	16
1.2.8. Индикация состояний двери	16
1.3. Логика блокировки дверей (упрощённая модель)	16
1.3.1. Общий принцип логики	16
1.3.2. Зависимости дверей	17
1.3.3. Блокировка двери	17
1.3.4. Кнопка Alarm	17
1.3.5. Сигнализация	17
1.3.6. Тайм-аут дверей	17
1.3.7. Ограничения логики	17
1.4. Ограничения и допущения	17
1.4.1. Аппаратные ограничения	17
1.4.2. Ограничения сетевой архитектуры	18
1.4.3. Программные ограничения	18
1.4.4. Ограничения логики и конфигурации	18
1.4.5. Допущения по эксплуатации	18
1.4.6. Допущения по безопасности	18
1.4.7. Границы ответственности системы	18
2. Архитектура программного обеспечения	19
2.1. Общие архитектурные принципы	19
2.1.1. Модульность и разделение ответственности	19
2.1.2. Локальная автономность и безопасность	19
2.1.3. Единый источник логических решений	19
2.1.4. Детерминированность и предсказуемость	19
2.1.5. Разделение управления и мониторинга	19
2.1.6. Безопасное применение изменений	19

2.2. Слои и модули прошивки	19
2.2.1. Аппаратный слой (Hardware Layer)	19
2.2.2. Слой драйверов периферии (Drivers Layer)	20
2.2.3. Доменный слой (Domain Layer)	20
2.2.4. Слой логики управления (Logic Core)	20
2.2.5. Слой безопасности (Safety Layer)	20
2.2.6. Слой коммуникаций (Communication Layer)	20
2.2.7. Слой конфигурации и мониторинга (Config & Monitor Layer)	21
2.3. Модель данных проекта	21
2.3.1. Ограничение масштаба проекта	21
2.3.2. Идентификация плат (Node ID)	21
2.3.3. Идентификация дверей	21
2.3.4. Привязка дверей к контроллерам (Mapping)	21
2.3.5. Глобальный идентификатор двери (globalDoorId)	21
2.3.6. Тип двери	22
2.3.7. Зависимости дверей (таблица зависимостей)	22
2.3.8. Состояние двери (Door State)	22
2.3.9. Тайм-ауты	22
2.3.10. Конфигурация проекта	22
2.3.11. Снимок состояния (для мониторинга)	22
2.4. Logic Core (логика управления дверьми)	22
2.4.1. Назначение Logic Core	22
2.4.2. Входные данные Logic Core	23
2.4.3. Выходные данные Logic Core	23
2.4.4. Основное правило логики зависимостей	23
2.4.5. Обработка нескольких зависимостей	23
2.4.6. Учёт тайм-аута после закрытия двери	23
2.4.7. Учёт тайм-аута открытой двери	23
2.4.8. Приоритет кнопки Alarm	23
2.4.9. Работа Logic Core в режиме Degraded	24
2.4.10. Формирование целевого состояния двери	24
2.4.11. Детерминированность логики	24
2.5. Safety Layer (уровень безопасности и инвариантов)	24
2.5.1. Назначение Safety Layer	24
2.5.2. Инварианты безопасности	24
2.5.3. Проверка команд Logic Core	24
2.5.4. Разрешение конфликтов команд	24
2.5.5. Поведение при аварийных ситуациях	25
2.5.6. Контроль целостности данных	25
2.5.7. Интеграция с аппаратным управлением	25
2.5.8. Детерминированность Safety Layer	25
2.6. Модуль индикации и сигнализации	25

2.6.1. Назначение модуля	25
2.6.2. Нормальная индикация состояния двери	25
2.6.3. Режим сигнализации.....	25
2.6.4. Поведение в режиме сигнализации	25
2.6.5. Приоритеты отображения	25
2.6.6. Обработка одновременных причин сигнализации.....	25
2.6.7. Ограничения и допущения	26
2.7. Модуль сетевого взаимодействия	26
2.7.1. CAN-взаимодействие (MASTER–SLAVE)	26
2.7.2. Ethernet-взаимодействие (WEB / HTTP)	26
2.7.3. RS-485 (внешняя индикация, read-only).....	27
2.7.4. Общие принципы сетевого взаимодействия	27
2.8. Модуль управления конфигурацией и хранения данных (QSPI Flash)	27
2.8.1. Назначение модуля.....	27
2.8.2. Состав конфигурации проекта.....	28
2.8.3. Валидация конфигурации.....	28
2.8.4. Атомарное применение конфигурации	28
2.8.5. Хранение конфигурации (QSPI Flash).....	28
2.8.6. Версия формата конфигурации	28
2.8.7. Восстановление после перезапуска	28
2.8.8. Доступ к информации о конфигурации	28
2.8.9. Связь с диагностикой.....	29
2.9. Диагностика и журналирование	29
2.9.1. Назначение диагностики	29
2.9.2. Журнал событий (Event Log)	29
2.9.3. Хранение журнала.....	29
2.9.4. Структура записи журнала	29
2.9.5. Доступ к журналу	29
2.9.6. Диагностические статусы.....	29
2.9.7. Ограничения диагностики.....	30
3. Надёжность, отказоустойчивость и Fail-safe	30
3.1. Общие принципы надёжности	30
3.1.1. Принцип безопасного отказа (Fail-safe)	30
3.1.2. Локальная безопасность двери.....	30
3.1.3. Разделение ответственности	30
3.1.4. Детерминированность поведения.....	30
3.1.5. Независимость от сетевых интерфейсов	30
3.1.6. Принцип контролируемого восстановления	30
3.1.7. Явная фиксация отказов.....	30
3.2. Аппаратный watchdog.....	30
3.2.1. Назначение аппаратного watchdog.....	31
3.2.2. Общие принципы использования	31

3.2.3. Условия обслуживания watchdog	31
3.2.4. Поведение при срабатывании watchdog	31
3.2.5. Безопасное состояние при перезапуске	31
3.2.6. Ограничения и допущения	31
3.3. Программный контроль зависаний	31
3.3.1. Назначение программного контроля	31
3.3.2. Контролируемые компоненты	31
3.3.3. Механизм “heartbeat” модулей.....	31
3.3.4. Реакция на отсутствие heartbeat.....	32
3.3.5. Связь с аппаратным watchdog	32
3.3.6. Журналирование сбоев	32
3.3.7. Ограничения	32
3.4. Поведение при зависании задач.....	32
3.4.1. Общие положения.....	32
3.4.2. Классификация задач по критичности	32
3.4.3. Зависание вспомогательной задачи	32
3.4.4. Зависание критической задачи	32
3.4.5. Одновременное зависание нескольких задач	33
3.4.6. Поведение дверей при зависании задач	33
3.4.7. Журналирование и диагностика.....	33
3.5. Поведение при отказе CAN	33
3.5.1. Определение отказа CAN	33
3.5.2. Поведение MASTER при отказе CAN	33
3.5.3. Поведение SLAVE при отказе CAN	33
3.5.4. Поведение дверей при отказе CAN	33
3.5.5. Восстановление после отказа CAN	33
3.5.6. Журналирование отказов CAN	34
3.6. Поведение при недоступности Ethernet и WEB-сеансе.....	34
3.6.1. Роль Ethernet и WEB в системе	34
3.6.2. WEB-сеанс управления (WEB Session)	34
3.6.3. Подключение WEB-клиента.....	34
3.6.4. Отключение WEB-клиента	34
3.6.5. Обрыв связи во время WEB-сеанса	34
3.6.6. Загрузка конфигурации через WEB	34
3.6.7. Обрыв связи во время загрузки конфигурации.....	34
3.6.8. Поведение дверей при любых проблемах Ethernet.....	35
3.6.9. Журналирование событий WEB и Ethernet.....	35
3.7. Поведение при сбоях памяти (QSPI Flash).....	35
3.7.1. Роль QSPI Flash в системе.....	35
3.7.2. Типы возможных сбоев QSPI Flash.....	35
3.7.3. Обнаружение сбоев памяти	35
3.7.4. Поведение системы при сбое чтения конфигурации	35

3.7.5. Поведение при сбое записи конфигурации.....	35
3.7.6. Поведение при сбоях журнала событий	35
3.7.7. Поведение дверей при сбоях QSPI Flash	36
3.7.8. Восстановление после сбоев памяти	36
3.7.9. Журналирование сбоев памяти	36
3.8. Поведение системы после перезапуска	36
3.8.1. Причины перезапуска	36
3.8.2. Начальное состояние системы.....	36
3.8.3. Инициализация аппаратных ресурсов	36
3.8.4. Восстановление конфигурации	36
3.8.5. Приведение дверей в безопасное состояние.....	36
3.8.6. Запуск логики управления	37
3.8.7. Восстановление сетевых взаимодействий	37
3.8.8. Журналирование перезапуска	37
3.8.9. Поведение при повторных перезапусках.....	37
3.9. Безопасные состояния дверей	37
3.9.1. Понятие безопасного состояния двери.....	37
3.9.2. Безопасные состояния по типу двери.....	37
3.9.3. Инварианты безопасности двери.....	37
3.9.4. Поведение двери при неопределённости	37
3.9.5. Переходы между безопасными состояниями	38
3.9.6. Безопасность при восстановлении	38
3.9.7. Журналирование переходов	38
4. Анализ аппаратной платформы и интерфейсов.....	38
4.1. Карта сигналов двери (замок / сенсор / LED / зуммер / Alarm)	38
4.1.1. Аппаратная модель одной двери.....	38
4.1.2. Сигнал управления замком (Solenoid Lock)	38
4.1.3. Датчик состояния двери.....	38
4.1.4. Кнопка аварийного открытия (Alarm).....	39
4.1.5. Световая индикация двери	39
4.1.6. Зуммер	39
4.1.7. Связь аппаратных сигналов с логикой.....	39
4.1.8. Инварианты аппаратного уровня.....	39
4.2. Электропитание и режимы отказов питания	39
4.2.1. Общие принципы электропитания	39
4.2.2. Тип применяемых замков	39
4.2.3. Безопасное состояние системы при отсутствии питания	39
4.2.4. Поведение системы при аварийном отключении питания	40
4.2.5. Поведение при наличии питания (штатная работа)	40
4.2.6. Поведение при пропадании питания во время работы.....	40
4.2.7. Поведение при восстановлении питания	40
4.2.8. Ограничения и допущения	40

4.3. CANBUS топология расширения.....	40
4.3.1. Назначение CANBUS в системе.....	40
4.3.2. Роли узлов в CAN-сети.....	40
4.3.3. Топология CAN-сети	40
4.3.4. Терминация CANBUS	41
4.3.5. Ограничения топологии	41
4.3.6. Адресация в CAN-сети.....	41
4.3.7. Поведение при ошибках CANBUS.....	41
4.3.8. Электромагнитная совместимость	41
4.3.9. Ограничения и допущения	41
4.4. RS-485 (MODBUS RTU, только индикация) (<i>переработанная редакция</i>)	41
4.4.1. Назначение RS-485	41
4.4.2. Топология и роли устройств	41
4.4.3. Полудуплекс и управление направлением (DE/RE).....	41
4.4.4. Поддерживаемые функции MODBUS.....	42
4.4.5. Модель данных, выдаваемая по MODBUS.....	42
4.4.6. Адресация MODBUS	43
4.4.7. Тайминги и устойчивость	43
4.4.8. Поведение при отказе RS-485/MODBUS	43
4.4.9. Ограничения и допущения	43
4.5. Ethernet (RMII PHY) и сетевые режимы	43
4.5.1. Назначение Ethernet	43
4.5.2. Роль Ethernet в архитектуре системы.....	44
4.5.3. WEB-клиент и HTTP-сервер.....	44
4.5.4. WEB-сеанс управления	44
4.5.5. Поведение системы при отсутствии Ethernet.....	44
4.5.6. Поведение при обрыве Ethernet во время WEB-сеанса	44
4.5.7. Безопасность сетевого доступа.....	44
4.5.8. Ограничения и допущения	44
4.6. QSPI Flash (память конфигурации и логов)	44
4.6.1. Назначение QSPI Flash	44
4.6.2. Данные конфигурации.....	45
4.6.3. Данные журнала событий (логов)	45
4.6.4. Надёжность хранения и целостность	45
4.6.5. Атомарное обновление конфигурации	45
4.6.6. Поведение при ошибках QSPI Flash.....	45
4.6.7. Доступ к информации о конфигурации через WEB	45
4.6.8. Ограничения и допущения	45
5. Обмен между платами по CANBUS	45
5.1. Адресация плат и роль MASTER	45
5.1.1. Node ID (идентификатор контроллера)	46
5.1.2. Роль MASTER.....	46

5.1.3. Роль SLAVE	46
5.1.4. Уникальность и конфликт Node ID	46
5.1.5. Использование Node ID в идентификации дверей	46
5.1.6. Адресация CAN-сообщений	46
5.2. Синхронизация состояний дверей	46
5.2.1. Принцип источника истины	46
5.2.2. Набор синхронизируемых данных.....	46
5.2.3. Периодичность синхронизации	47
5.2.4. Начальная синхронизация	47
5.2.5. Конфликт состояний.....	47
5.2.6. Потеря синхронизации.....	47
5.2.7. Восстановление синхронизации.....	47
5.2.8. Ограничения и допущения	47
5.3. Формат CAN-сообщений и кадры обмена.....	47
5.3.1. Общие принципы CAN-протокола	47
5.3.2. Классы CAN-сообщений	48
5.3.3. Формирование CAN-идентификатора	48
5.3.4. Кадры состояния дверей (Status Frame)	48
5.3.5. Кадры команд (Command Frame).....	48
5.3.6. Служебные кадры (Service Frames).....	48
5.3.7. Контроль целостности и диагностика	49
5.3.8. Приоритеты CAN-сообщений	49
5.3.9. Ограничения и допущения	49
5.4. Распределение ответственности (MASTER / SLAVE)	49
5.4.1. Общий принцип распределения ответственности	49
5.4.2. Ответственность MASTER-контроллера	49
5.4.3. Ответственность SLAVE-контроллера	49
5.4.4. Источники истины	50
5.4.5. Обработка противоречий	50
5.4.6. Поведение при потере связи	50
5.4.7. Восстановление взаимодействия	50
5.4.8. Запрещённые действия.....	50
5.4.9. Ключевой принцип.....	50
5.5. Обработка конфликтов команд.....	50
5.5.1. Что считается конфликтом	50
5.5.2. Общий принцип разрешения конфликтов.....	50
5.5.3. Приоритеты (жёсткий порядок)	51
5.5.4. Конфликт “Нужно заблокировать, но дверь открыта”	51
5.5.5. Конфликт с Alarm	51
5.5.6. Конфликт в пересечении зависимостей	51
5.5.7. Конфликт в Degraded	51
5.5.8. Конфликт “команда устарела / несогласована”	51

5.5.9. Диагностика конфликтов	51
5.6. Тайм-ауты, heartbeat и диагностика линии CAN.....	51
5.6.1. Heartbeat SLAVE-контроллеров	51
5.6.2. Период heartbeat.....	51
5.6.3. Тайм-аут отсутствия heartbeat	52
5.6.4. Поведение SLAVE при потере связи с MASTER.....	52
5.6.5. Диагностика ошибок CAN-линии	52
5.6.6. Восстановление связи	52
5.6.7. Ограничения и допущения	52
6. RS-485 (MODBUS RTU, только индикация)	52
6.1. Модель данных для внешнего клиента	52
6.1.1. Системные данные.....	52
6.1.2. Данные по дверям	52
6.2. Протокол выдачи статусов	53
6.3. Ограничения безопасности (read-only)	53
6.4. Совместимость форматов (ASCII / MODBUS RTU)	53
6.5. Итоговые принципы раздела 6	53
7. Ethernet и встроенный HTTP-сервер	53
7.1. Назначение Ethernet-интерфейса	53
7.1.1. Ethernet как сервисный интерфейс	53
7.1.2. Отделение Ethernet от логики управления	54
7.1.3. Использование Ethernet на этапе эксплуатации	54
7.1.4. Ограничения Ethernet-интерфейса	54
7.1.5. Ключевой принцип.....	54
7.2. Роль Ethernet в архитектуре системы	54
7.2.1. Ethernet как внешняя точка доступа	54
7.2.2. Разграничение архитектурных уровней.....	54
7.2.3. Взаимодействие Ethernet с логикой	54
7.2.4. Ethernet и режимы работы системы	55
7.2.5. Ограничение влияния Ethernet.....	55
7.2.6. Ключевой принцип.....	55
7.3. Встроенный HTTP-сервер на MASTER.....	55
7.3.1. Назначение HTTP-сервера	55
7.3.2. Область работы HTTP-сервера	55
7.3.3. Архитектурные ограничения HTTP-сервера	55
7.3.4. Взаимодействие HTTP-сервера с конфигурацией	55
7.3.5. Ограничение влияния HTTP-нагрузки.....	55
7.3.6. Обработка ошибок HTTP-сервера	55
7.3.7. Ключевой принцип.....	56
7.4. WEB-сеанс управления (<i>уточнённая редакция</i>).....	56
7.4.1. Инициация WEB-сеанса	56
7.4.2. Роли пользователей WEB-интерфейса	56

7.4.3. Главный Администратор.....	56
7.4.4. Поведение системы при активном WEB-сеансе	56
7.4.5. Завершение WEB-сеанса	56
7.4.6. Аварийное завершение WEB-сеанса	56
7.4.7. Логирование действий пользователей.....	57
7.4.8. Ключевой принцип.....	57
7.5. Конфигурирование системы через WEB	57
7.5.1. Назначение WEB-конфигурирования	57
7.5.2. Состав конфигурационных данных	57
7.5.3. Процесс изменения конфигурации	57
7.5.4. Валидация конфигурации.....	57
7.5.5. Атомарная активация конфигурации	57
7.5.6. Поведение системы при применении конфигурации	58
7.5.7. Подтверждение успешной загрузки	58
7.5.8. Логирование изменений конфигурации	58
7.5.9. Ключевой принцип.....	58
7.6. Поведение системы при отказе Ethernet	58
7.6.1. Отсутствие Ethernet как штатный режим	58
7.6.2. Потеря Ethernet при отсутствии активного WEB-сеанса	58
7.6.3. Потеря Ethernet при активном WEB-сеансе	58
7.6.4. Потеря Ethernet во время загрузки конфигурации	58
7.6.5. Восстановление Ethernet-связи	58
7.6.6. Диагностика отказа Ethernet	59
7.6.7. Ключевой принцип.....	59
7.7. Безопасность сетевого доступа.....	59
7.7.1. Аутентификация пользователей	59
7.7.2. Разграничение прав доступа.....	59
7.7.3. Защита конфигурационных операций	59
7.7.4. Защита от несанкционированного доступа.....	59
7.7.5. Хранение учетных данных	59
7.7.6. Аудит и трассируемость	59
7.7.7. Ограничения и допущения	60
7.7.8. Ключевой принцип.....	60
7.8. Ограничения и допущения	60
7.8.1. Ограничения Ethernet и WEB-интерфейса	60
7.8.2. Ограничения по доступности WEB	60
7.8.3. Допущения по эксплуатации	60
7.8.4. Допущения по безопасности.....	60
7.8.5. Границы ответственности системы	60
7.8.6. Ключевой принцип.....	60
8. WEB-интерфейс (Visual Studio)	60
8.1. Назначение WEB-интерфейса	60

8.1.1. Роль WEB-интерфейса в системе.....	60
8.1.2. WEB-интерфейс как средство конфигурирования	61
8.1.3. WEB-интерфейс как средство мониторинга	61
8.1.4. Ограничения WEB-интерфейса	61
8.1.5. Ключевой принцип.....	61
8.2. Роли пользователей и сценарии использования.....	61
8.2.1. Роли пользователей	61
8.2.2. Сценарии использования WEB-интерфейса	62
8.2.3. Принципы разграничения доступа	62
8.2.4. Связь ролей с безопасностью системы.....	62
8.2.5. Ключевой принцип.....	62
8.3. Управление пользователями	62
8.3.1. Назначение управления пользователями	62
8.3.2. Учетные записи пользователей	62
8.3.3. Назначение и изменение ролей	63
8.3.4. Управление учетными данными	63
8.3.5. Аудит действий пользователей	63
8.3.6. Ограничения и допущения	63
8.3.7. Ключевой принцип.....	63
8.4. Общая архитектура WEB-приложения	63
8.4.1. Общий подход к архитектуре.....	63
8.4.2. Архитектурные компоненты WEB-приложения	63
8.4.3. Взаимодействие с HTTP-сервером контроллера	64
8.4.4. Работа с состоянием системы	64
8.4.5. Обработка обрывов связи	64
8.4.6. Требования к производительности и устойчивости.....	64
8.4.7. Ограничения архитектуры	64
8.4.8. Ключевой принцип.....	64
8.5. Экран структуры проекта (двери и зависимости).....	64
8.5.1. Назначение экрана структуры проекта	64
8.5.2. Представление дверей.....	64
8.5.3. Отображение зависимостей	65
8.5.4. Навигация и фильтрация	65
8.5.5. Переход к конфигурированию	65
8.5.6. Ограничения экрана	65
8.5.7. Ключевой принцип.....	65
8.6. Экран конфигурирования дверей	65
8.6.1. Назначение экрана конфигурирования двери	65
8.6.2. Идентификация двери	65
8.6.3. Тип двери и базовое поведение	65
8.6.4. Тайм-ауты двери.....	66
8.6.5. Ограничения экрана конфигурирования двери	66

8.6.6. Ключевой принцип	66
8.7. Экран конфигурирования логики зависимостей	66
8.7.1. Назначение экрана логики зависимостей	66
8.7.2. Базовая модель логики.....	66
8.7.3. Представление зависимостей в интерфейсе	66
8.7.4. Редактирование зависимостей	66
8.7.5. Валидация логики зависимостей	66
8.7.6. Пересечение и совместное участие дверей	67
8.7.7. Ограничения экрана логики зависимостей	67
8.7.8. Ключевой принцип.....	67
8.8. Экран мониторинга состояния дверей	67
8.8.1. Назначение экрана мониторинга	67
8.8.2. Отображаемые данные по дверям.....	67
8.8.3. Индикация состояний	67
8.8.4. Обновление данных	67
8.8.5. Поведение при потере связи	67
8.8.6. Ограничения экрана мониторинга	68
8.8.7. Ключевой принцип.....	68
8.9. Отображение дверей на карте помещений	68
8.9.1. Назначение карты помещений.....	68
8.9.2. Источник и формат карты	68
8.9.3. Привязка дверей к карте	68
8.9.4. Отображение состояния дверей на карте	68
8.9.5. Поведение при отказах и деградации	68
8.9.6. Ограничения карты помещений	68
8.9.7. Ключевой принцип.....	69
8.10. Журнал событий и изменений	69
8.10.1. Назначение журнала	69
8.10.2. Типы фиксируемых событий	69
8.10.3. События конфигурации	69
8.10.4. Аудит действий пользователей.....	69
8.10.5. Отображение журнала в WEB-интерфейсе	69
8.10.6. Хранение и объём журнала.....	70
8.10.7. Ограничения журнала	70
8.10.8. Ключевой принцип	70
8.11. Загрузка, сохранение и версия конфигурации.....	70
8.11.1. Назначение управления конфигурацией	70
8.11.2. Хранение конфигурации	70
8.11.3. Версия конфигурации.....	70
8.11.4. Версия формата конфигурации	70
8.11.5. Процесс загрузки конфигурации	70
8.11.6. Подтверждение и отображение результата	71

8.11.7. Ограничения управления конфигурацией	71
8.11.8. Ключевой принцип	71
8.12. Обработка ошибок и уведомления пользователя	71
8.12.1. Назначение обработки ошибок	71
8.12.2. Классификация ошибок	71
8.12.3. Источники ошибок	71
8.12.4. Отображение ошибок пользователю	71
8.12.5. Поведение при ошибках конфигурирования	71
8.12.6. Поведение при сетевых ошибках	72
8.12.7. Журналирование ошибок	72
8.12.8. Ключевой принцип	72
8.13. Ограничения и допущения	72
8.13.1. Ограничения WEB-интерфейса	72
8.13.2. Ограничения при отсутствии связи	72
8.13.3. Допущения по эксплуатации	72
8.13.4. Допущения по безопасности	72
8.13.5. Границы ответственности WEB-интерфейса	72
8.13.6. Ключевой принцип	73
9. Безопасность и промышленная надёжность	73
9.1. Модель угроз (LAN / CAN / RS-485 / физический доступ)	73
9.1.1. Угрозы со стороны LAN (Ethernet)	73
9.1.2. Угрозы со стороны CANBUS	73
9.1.3. Угрозы со стороны RS-485	73
9.1.4. Угрозы физического доступа	73
9.2. Аутентификация и права доступа	73
9.2.1. Принципы аутентификации	73
9.2.2. Разграничение прав	74
9.2.3. Привязка действий к пользователю	74
9.3. Защита конфигурации (целостность / подмена)	74
9.3.1. Целостность конфигурации	74
9.3.2. Защита от подмены	74
9.3.3. Атомарность и откат	74
9.4. Fail-safe стратегии	74
9.4.1. Отказ питания	74
9.4.2. Отказ контроллера	74
9.4.3. Отказ CAN-сети	74
9.4.4. Отказ Ethernet	74
9.4.5. Отказ датчиков и исполнителей	74
9.5. EMC / ESD / помехоустойчивость на уровне ПО	75
9.5.1. Программная устойчивость к помехам	75
9.5.2. Защита от ложных событий	75
9.5.3. Поведение при неопределённых состояниях	75

9.6. Ключевые принципы раздела 9	75
10. Тестирование и валидация	75
10.1. Модульные тесты логики.....	75
10.1.1. Область модульного тестирования	75
10.1.2. Проверяемые сценарии	75
10.1.3. Цель модульных тестов	75
10.2. Интеграционные тесты (CAN / RS-485 / Ethernet)	76
10.2.1. Интеграция MASTER-SLAVE по CAN	76
10.2.2. Интеграция RS-485 (MODBUS RTU).....	76
10.2.3. Интеграция Ethernet и WEB	76
10.3. Стендовые испытания дверей	76
10.3.1. Аппаратная часть испытаний.....	76
10.3.2. Проверяемые сценарии	76
10.3.3. Цель стендовых испытаний	76
10.4. Тесты отказов (датчики, кнопки, питание)	76
10.4.1. Отказы датчиков.....	76
10.4.2. Отказы кнопки Alarm	76
10.4.3. Отказ питания.....	77
10.5. Нагрузочные тесты и устойчивость RTOS.....	77
10.5.1. Нагрузочные сценарии.....	77
10.5.2. Устойчивость планировщика и задач	77
10.5.3. Длительная работа	77
10.6. Итоговая валидация	77
10.7. Ключевой принцип раздела 10	77
11. Производственная подготовка и сопровождение.....	77
11.1. Первичная прошивка и серийная настройка	77
11.1.1. Первичная прошивка контроллеров.....	77
11.1.2. Назначение Node ID и роли контроллера	78
11.1.3. Проверка аппаратных каналов	78
11.1.4. Сброс к заводскому состоянию.....	78
11.2. Сервисный режим и диагностика	78
11.2.1. Назначение сервисного режима	78
11.2.2. Возможности сервисного режима	78
11.2.3. Ограничения сервисного режима	78
11.3. Документация для монтажника / интегратора.....	78
11.3.1. 1 Документация для монтажника	78
11.3.2. Документация для интегратора	78
11.3.3. Эксплуатационная документация.....	79
11.4. Версионирование и релизный цикл (FW + WEB)	79
11.4.1. Версионирование прошивки (FW)	79
11.4.2. Версионирование WEB-приложения	79
11.4.3. Процесс обновления	79

11.4.4. Поддержка и сопровождение.....	79
11.5. Ключевой принцип раздела 11	79

1. ВВОДНЫЕ ТРЕБОВАНИЯ И ГРАНИЦЫ СИСТЕМЫ

1.1. РОЛИ И СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ (МОНОПЛАТА / МУЛЬТИПЛАТА)

(модель MASTER–SLAVE и режим Degraded)

1.1.1. ОБЩАЯ МОДЕЛЬ СИСТЕМЫ

Система управления дверями состоит из одного или нескольких контроллеров дверей, объединённых логически в проект. Основные положения:

- Базовым объектом системы является **дверь**
- Каждая дверь имеет **уникальный идентификатор** в пределах проекта
- Для каждой двери задаётся **список зависимых дверей**
- Логика системы заключается в управлении блокировкой дверей на основе этих зависимостей

1.1.2. КОНТРОЛЛЕР ДВЕРЕЙ (DOOR CONTROLLER)

Контроллер дверей — программно-аппаратное устройство на базе STM32H723, предназначенное для управления дверями с электромеханическими замками.

Основные свойства:

- Поддержка от **1 до 8 дверей** на одной плате
- Каждая дверь имеет:
 - соленоидный замок;
 - датчик состояния двери;
 - световую индикацию (красный / зелёный);
 - зуммер;
 - кнопку аварийного открытия (Alarm)
- Контроллер всегда напрямую управляет **своими локальными дверями**
- Контроллер обеспечивает **локальную безопасность дверей** независимо от состояния сети

1.1.3. РЕЖИМ STANDALONE (ОДИНОЧНАЯ ПЛАТА)

Режим Standalone используется, если вся система дверей размещается на одной плате.

Условия применения:

- Общее количество дверей в проекте: **от 2 до 8**
- Используется **один контроллер**
- Все двери проекта подключены к одной плате

Характеристики:

- Управление дверями полностью локальное
- Все правила логики применяются локально
- Ethernet используется для:
 - конфигурирования;
 - мониторинга
- RS-485 используется только для индикации
- CAN-интерфейс не используется

1.1.4. РЕЖИМ MASTER–SLAVE (МУЛЬТИПЛАТНАЯ СИСТЕМА)

Режим MASTER–SLAVE используется, если общее количество дверей превышает 8 и требуется использование нескольких плат.

1.1.4.1. MASTER-КОНТРОЛЛЕР

MASTER — центральный логический контроллер системы.

Функции MASTER:

- Хранение полной конфигурации проекта
 - Хранит конфигурацию зависимостей дверей
 - Вычисляет, какие двери должны быть заблокированы
 - Распределение логических команд SLAVE-контроллерам
 - Сбор состояний дверей от SLAVE по CAN
 - Контроль доступности SLAVE-контроллеров
- MASTER не управляет физическими выходами SLAVE напрямую.

1.1.4.2. SLAVE-КОНТРОЛЛЕР

SLAVE — подчинённый контроллер.

Функции SLAVE:

- Управление **1...8 локальными дверями**

- Передача состояний дверей MASTER-контроллеру
 - Исполнение логических команд MASTER
 - Обеспечение локальной аварийной и безопасной работы дверей
- SLAVE хранит **минимальную информацию**, необходимую для безопасной автономной работы дверей.

1.1.5. ПОВЕДЕНИЕ СИСТЕМЫ ПРИ ПОТЕРЕ CAN-СЕТИ (РЕЖИМ DEGRADED)

1.1.5.1. ПЕРЕХОД СИСТЕМЫ В РЕЖИМ DEGRADED

При потере CAN-связи в режиме MASTER–SLAVE:

- Логика зависимостей **временно не применяется**;
- Система переходит в режим **Degraded**

1.1.5.2. ПОВЕДЕНИЕ ДВЕРЕЙ В РЕЖИМЕ DEGRADED

В режиме Degraded:

- Логика зависимостей **временно не применяется**;
- Каждая дверь работает как **независимая**
- Состояние двери фиксируется в безопасном положении:
 - заблокирована — для дверей типа NC;
 - разблокирована — для дверей типа NO
- Кнопка Alarm всегда работает локально
- CARD READER двери продолжают работать как NC

1.1.5.3. ПОВЕДЕНИЕ ПРИ ВОССТАНОВЛЕНИИ СВЯЗИ

После восстановления CAN-связи:

- Выполняется ресинхронизация состояний дверей
- Логика зависимостей вновь активируется
- Возврат логики выполняется без резких переключений состояний дверей

1.2. ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ СОСТОЯНИЙ ДВЕРИ

1.2.1. ФИЗИЧЕСКИЕ ЭЛЕМЕНТЫ ДВЕРИ

В данном разделе фиксируются аппаратные элементы, относящиеся к одной двери и используемые системой управления.

Состав физических элементов двери:

- **Замок (соленоид)**
Исполнительный механизм блокировки двери.
- **Датчик состояния двери**
Датчик, определяющий физическое положение двери (открыта / закрыта).
- **Кнопка аварийного открытия (Alarm)**
Механизм принудительной локальной разблокировки двери.
- **Световая индикация**
Индикация состояния двери:
 - красный — дверь заблокирована;
 - зелёный — дверь не заблокирована.
- **Звуковая индикация (зуммер)**
Используется для предупреждающих и аварийных сигналов.

1.2.2. ТИПЫ ДВЕРЕЙ ПО ПРИНЦИПУ БЛОКИРОВКИ

Тип двери определяет **базовое безопасное состояние двери** при отсутствии управляющего воздействия.

Типы дверей:

- **NC (Normally Closed)**
Дверь по умолчанию находится в заблокированном состоянии.
- **NO (Normally Open)**
Дверь по умолчанию находится в разблокированном состоянии.
- **CARD READER**
Специализированный режим двери, использующий внешний механизм авторизации доступа.

Тип двери задаётся в конфигурации и используется системой при:

- инициализации;
- деградации логики;
- аварийных ситуациях.

1.2.3. ФИЗИЧЕСКИЕ СОСТОЯНИЯ ДВЕРИ

Физическое состояние отражает **фактическое механическое положение двери**.

Физические состояния:

- **Открыта**
Дверь физически открыта. Датчик состояния двери разомкнут.

- **Закрыта**
Дверь физически закрыта. Датчик состояния двери замкнут.
Физическое состояние определяется **исключительно** датчиком двери.

1.2.4. СОСТОЯНИЕ ДВЕРИ (БЛОКИРОВКА)

Состояние двери определяет, **зафиксирована ли дверь замком**.

Состояния двери:

- **Дверь заблокирована**
Дверь физически закрыта и зафиксирована замком.
- **Дверь разблокирована**
Замок деактивирован, дверь может быть открыта.

Инвариант безопасности:

- Дверь **не может быть заблокирована**, если она физически открыта.
- Активация замка **разрешена только при физически закрытой двери**.

1.2.5. ЛОГИЧЕСКИЕ СОСТОЯНИЯ ДВЕРИ

Логическое состояние отражает **текущее состояние двери с точки зрения системы управления**.

Логические состояния:

- **Normal**
Дверь работает согласно заданной логике зависимостей.
- **Degraded**
Дверь работает в упрощённом режиме из-за потери части логики зависимостей.
- **Alarm**
Дверь принудительно разблокирована в результате активации кнопки Alarm.

Каждая дверь в любой момент времени может находиться **только в одном логическом состоянии**.

1.2.6. СОБЫТИЯ ДВЕРИ

Событие — это изменение условий, влияющее на состояние двери.

Основные события:

- Изменение физического состояния двери (открыта / закрыта)
- Активация кнопки Alarm
- Деактивация кнопки Alarm
- Истечение тайм-аутов
- Потеря связи с другими контроллерами
- Восстановление связи
- Изменение конфигурации

События используются для переходов между логическими состояниями двери.

1.2.7. ПРИОРИТЕТЫ СОСТОЯНИЙ И СОБЫТИЙ

В системе устанавливается строгий порядок приоритетов.

Общие принципы приоритета:

- **Alarm** имеет наивысший приоритет
- **Degraded** имеет приоритет над **Normal**
- Конфликтующие события разрешаются согласно приоритету логических состояний

Приоритеты используются для:

- предотвращения конфликтов логики;
- обеспечения предсказуемого поведения;
- гарантии безопасности.

1.2.8. ИНДИКАЦИЯ СОСТОЯНИЙ ДВЕРИ

Индикация предназначена для визуального и звукового отображения состояния двери.

Общие принципы:

- Индикация отражает состояние блокировки двери
- Логическое состояние влияет на режим индикации
- Аварийные состояния имеют приоритет в индикации
- Индикация должна быть однозначной и понятной

Конкретные соответствия индикации определяются в отдельных разделах.

1.3. ЛОГИКА БЛОКИРОВКИ ДВЕРЕЙ (УПРОЩЁННАЯ МОДЕЛЬ)

1.3.1. ОБЩИЙ ПРИНЦИП ЛОГИКИ

Логика системы основана на **физическом состоянии дверей** и заданных зависимостях между ними.

Основное правило:

Если дверь физически открыта, все двери, связанные с ней по зависимостям, должны быть заблокированы.

1.3.2. ЗАВИСИМОСТИ ДВЕРЕЙ

Для каждой двери в конфигурации задаётся:

- список дверей, которые должны быть заблокированы при её открытии.

Особенности:

- одна дверь может участвовать в нескольких зависимостях;
- зависимости могут связывать двери на разных платах.

1.3.3. БЛОКИРОВКА ДВЕРИ

Блокировка двери означает:

- активацию замка-соленоида;
- выключение зеленой и включение красной индикации.

Блокировка:

- **разрешена только при физически закрытой двери;**
- запрещена при физически открытой двери.

1.3.4. КНОПКА ALARM

Кнопка Alarm:

- имеет абсолютный приоритет над логикой зависимостей;
- при нажатии:
 - дверь разблокируется;
 - включается режим сигнализации;
- при повторном нажатии:
 - дверь возвращается в состояние, требуемое логикой зависимостей в текущий момент.

1.3.5. СИГНАЛИЗАЦИЯ

Сигнализация — специальный режим индикации двери.

Сигнализация включается:

- при разблокировке двери по Alarm;
- при превышении допустимого времени открытого состояния двери.

В режиме сигнализации:

- красный и зелёный индикаторы переключаются поочерёдно;
- зуммер подаёт периодический звуковой сигнал.
- Периодичность смены по 1 секунде на каждого.

1.3.6. ТАЙМ-АУТ ДВЕРЕЙ

1.3.6.1. ТАЙМ-АУТ ОТКРЫТОЙ ДВЕРИ

Тайм-аут допустимого открытого состояния задается в конфигураторе одинаковый для всех дверей.

При превышении тайм-аута:

- включается режим сигнализации;
- логика блокировки остальных дверей продолжает действовать.

1.3.6.2. ТАЙМ-АУТ ПОСЛЕ ЗАКРЫТИЯ ДВЕРИ

Для каждой двери может быть задан тайм-аут после закрытия двери, то есть время задержки реакции логики после того, как дверь закроется. Если Тайм-аут после закрытия двери равен «0», то логика отработывает сразу же после того, как дверь закроется. Иначе выдерживается заданное время тайм-аута после закрытия двери для того, чтобы логика отреагировала.

1.3.7. ОГРАНИЧЕНИЯ ЛОГИКИ

Логика системы **не должна**:

- нарушать инварианты физической безопасности;
- блокировать работу Alarm;
- приводить дверь в недопустимое состояние;
- зависеть от наличия сети для локальной безопасности.

1.4. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

Цель пункта 1.4 — **жёстко зафиксировать границы системы**, чтобы:

- избежать неверных ожиданий;
- защитить архитектуру от “ползучего” расширения;
- заранее определить, какие ситуации считаются корректными, а какие — вне области ответственности системы.

1.4.1. АППАРАТНЫЕ ОГРАНИЧЕНИЯ

- Один контроллер поддерживает **не более 8 дверей**
- Минимальное количество дверей в проекте Standalone — **2**
- Максимальное количество дверей в проекте определяется:
 - возможностями CAN-сети;
 - производительностью MASTER-контроллера;
- Каждая дверь должна иметь:

- исправный датчик состояния;
 - исправный исполнительный механизм замка;
- Работа системы предполагается при штатных параметрах питания

1.4.2. ОГРАНИЧЕНИЯ СЕТЕВОЙ АРХИТЕКТУРЫ

- CAN используется **только для обмена состояниями и логическими командами**
- Потеря CAN-связи:
 - не должна приводить к опасным состояниям дверей;
 - переводит систему в режим Degraded
- Ethernet:
- Ethernet используется для конфигурирования и мониторинга системы;
- Ethernet не используется для управления дверьми и не влияет на логику блокировки в runtime;
- работа дверей и логики блокировки не зависит от наличия Ethernet-соединения;
- мониторинг через Ethernet может выполняться с высокой частотой обновления и носит наблюдательный характер.
- RS-485:
 - используется только для индикации;
 - не может влиять на логику и управление дверями

1.4.3. ПРОГРАММНЫЕ ОГРАНИЧЕНИЯ

- Логика системы основана **только на физическом состоянии дверей и заданных зависимостях**
- Система **не выполняет оптимизацию логики** и не перестраивает зависимости автоматически
- Изменение конфигурации возможно только:
 - через WEB-конфигуратор;
 - при корректной валидации параметров
- Логика не должна зависеть от:
 - скорости сети;
 - задержек Ethernet;
 - внешних клиентов

1.4.4. ОГРАНИЧЕНИЯ ЛОГИКИ И КОНФИГУРАЦИИ

- Одна дверь:
 - может участвовать в нескольких зависимостях;
 - не может находиться более чем в одном логическом состоянии
- Конфигурация не должна:
 - создавать циклы, приводящие к неопределённому поведению;
 - нарушать инварианты безопасности двери
- Система не гарантирует корректную работу при:
 - физическом обходе датчиков;
 - механических неисправностях дверей

1.4.5. ДОПУЩЕНИЯ ПО ЭКСПЛУАТАЦИИ

- Предполагается корректный монтаж дверей и датчиков
- Предполагается, что:
 - дверь физически закрывается до активации замка;
 - пользователь не удерживает дверь в полукрытом состоянии
- Предполагается, что кнопка Alarm используется:
 - только в аварийных ситуациях;
 - осознанно персоналом

1.4.6. ДОПУЩЕНИЯ ПО БЕЗОПАСНОСТИ

- Кнопка Alarm всегда имеет приоритет над логикой
- В случае неопределённого состояния система выбирает **безопасное состояние**
- Любой отказ сети или внешних интерфейсов:
 - не блокирует аварийное открытие;
 - не приводит к самопроизвольной блокировке открытой двери
- Система не предназначена для:
 - управления эвакуацией людей;
 - замены сертифицированных систем пожарной безопасности

1.4.7. ГРАНИЦЫ ОТВЕТСТВЕННОСТИ СИСТЕМЫ

Система отвечает за:

- управление блокировкой дверей;
- индикацию состояний;
- выполнение заданных зависимостей;
- локальную безопасность дверей.

Система **не отвечает** за:

- механическое состояние дверей;
- физическую прочность замков;
- поведение пользователей;

- соблюдение нормативов эксплуатации объекта.

2. АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Цель раздела 2 — зафиксировать **модульную структуру прошивки** и границы ответственности модулей так, чтобы:

- логика была детерминированной и безопасной,
- MASTER–SLAVE масштабировался,
- WEB/RS-485 оставались “наблюдением и конфигом”, не влияя на runtime-логику.

2.1. ОБЩИЕ АРХИТЕКТУРНЫЕ ПРИНЦИПЫ

Данный раздел определяет базовые принципы построения программного обеспечения контроллера дверей, обязательные для всех режимов работы системы.

2.1.1. МОДУЛЬНОСТЬ И РАЗДЕЛЕНИЕ ОТВЕТСТВЕННОСТИ

Программное обеспечение строится по модульному принципу с чётким разделением зон ответственности:

- аппаратный уровень;
- уровень драйверов;
- уровень доменной логики (двери и зависимости);
- уровень коммуникаций;
- уровень конфигурации и мониторинга.

Каждый модуль:

- выполняет строго определённую функцию;
- не содержит логики, не относящейся к его ответственности;
- взаимодействует с другими модулями через формализованные интерфейсы.

2.1.2. ЛОКАЛЬНАЯ АВТОНОМНОСТЬ И БЕЗОПАСНОСТЬ

- Каждая дверь должна обеспечивать **локальную безопасную работу**, независимо от:
 - наличия CAN-связи;
 - наличия Ethernet;
 - состояния внешних систем.
- Любой отказ сети **не должен приводить к опасному состоянию двери**.
- Локальные аппаратные события (датчик двери, кнопка Alarm) имеют приоритет над сетевыми событиями.

2.1.3. ЕДИНЫЙ ИСТОЧНИК ЛОГИЧЕСКИХ РЕШЕНИЙ

- В режиме **Standalone** все логические решения принимаются локально.
- В режиме **MASTER–SLAVE**:
 - MASTER является **единственным источником логических решений**;
 - SLAVE исполняет команды и обеспечивает локальную безопасность.
- SLAVE не принимает самостоятельных логических решений, кроме аварийных.

2.1.4. ДЕТЕРМИНИРОВАННОСТЬ И ПРЕДСКАЗУЕМОСТЬ

- Логика должна быть:
 - детерминированной;
 - повторяемой;
 - не зависящей от порядка прихода сетевых сообщений.
- В одинаковых входных условиях система должна всегда приходить к одинаковому результату.
- Конфликтующие события разрешаются в соответствии с зафиксированными приоритетами.

2.1.5. РАЗДЕЛЕНИЕ УПРАВЛЕНИЯ И МОНИТОРИНГА

- Контур управления дверьми полностью автономен.
- Внешние интерфейсы (Ethernet, RS-485):
 - используются для конфигурирования и мониторинга;
 - не участвуют в принятии логических решений в runtime.
- Потеря внешнего мониторинга не влияет на работу системы.

2.1.6. БЕЗОПАСНОЕ ПРИМЕНЕНИЕ ИЗМЕНЕНИЙ

- Любое изменение конфигурации:
 - проходит валидацию;
 - применяется атомарно;
 - не должно приводить к кратковременным опасным состояниям.
- На время применения конфигурации система может переходить в безопасный режим.

2.2. СЛОИ И МОДУЛИ ПРОШИВКИ

Данный раздел описывает логическую структуру программного обеспечения и назначение основных модулей.

2.2.1. АППАРАТНЫЙ СЛОЙ (HARDWARE LAYER)

Аппаратный слой отвечает за:

- базовую инициализацию микроконтроллера;
- настройку тактирования;

- инициализацию памяти;
- базовую обработку прерываний.

Особенности:

- не содержит логики управления дверями;
- используется всеми вышележащими слоями.

2.2.2. СЛОЙ ДРАЙВЕРОВ ПЕРИФЕРИИ (DRIVERS LAYER)

Слой драйверов обеспечивает доступ к аппаратным ресурсам.

Основные драйверы:

- **Door I/O Driver**
 - управление замками;
 - чтение датчиков дверей;
 - управление световой и звуковой индикацией;
 - обработка кнопки Alarm.
- **CAN Driver**
 - обмен данными между контроллерами.
- **Ethernet Driver**
 - сетевой стек;
 - HTTP-сервер.
- **RS-485 Driver**
 - передача статусов во внешние системы.
- **QSPI Flash Driver**
 - хранение конфигурации и журналов.

Драйверы:

- не принимают логических решений;
- не знают о зависимостях дверей;
- предоставляют абстрактные функции управления и чтения состояний.

2.2.3. ДОМЕННЫЙ СЛОЙ (DOMAIN LAYER)

Доменный слой реализует **смысловую модель системы**.

Включает:

- модель двери;
- состояние двери;
- логику зависимостей;
- обработку событий.

Этот слой:

- не зависит от конкретных интерфейсов связи;
- работает с абстрактными данными;
- является ядром логики системы.

2.2.4. СЛОЙ ЛОГИКИ УПРАВЛЕНИЯ (LOGIC CORE)

Logic Core:

- анализирует физические состояния дверей;
- применяет правила зависимостей;
- учитывает тайм-ауты и приоритеты;
- формирует целевые команды управления дверями.

Logic Core:

- не управляет аппаратурой напрямую;
- не взаимодействует с сетью напрямую;
- выдаёт только **решения**, а не физические действия.

2.2.5. СЛОЙ БЕЗОПАСНОСТИ (SAFETY LAYER)

Safety Layer обеспечивает выполнение инвариантов безопасности.

Функции:

- запрет блокировки физически открытой двери;
- приоритет кнопки Alarm;
- контроль допустимости команд логики;
- фильтрация недопустимых состояний.

Все команды на управление дверями проходят через этот слой.

2.2.6. СЛОЙ КОММУНИКАЦИЙ (COMMUNICATION LAYER)

Слой коммуникаций отвечает за:

- обмен состояниями дверей;
- передачу логических команд;
- уведомления о событиях.

Интерфейсы:

- CAN — для взаимодействия MASTER–SLAVE;
- Ethernet — для WEB-конфигурации и мониторинга;
- RS-485 — для внешней индикации.

Коммуникационный слой **не содержит логики блокировки**.

2.2.7. СЛОЙ КОНФИГУРАЦИИ И МОНИТОРИНГА (CONFIG & MONITOR LAYER)

Этот слой отвечает за:

- загрузку конфигурации из WEB;
- валидацию параметров;
- сохранение конфигурации;
- формирование данных мониторинга.

Мониторинг:

- только чтение;
- не влияет на работу логики;
- может выполняться с высокой частотой.

2.3. МОДЕЛЬ ДАННЫХ ПРОЕКТА

Цель раздела 2.3 — зафиксировать единое представление данных, используемое во всей системе: в логике, CAN-обмене, WEB-конфигураторе, мониторинге и хранении конфигурации.

2.3.1. ОГРАНИЧЕНИЕ МАСШТАБА ПРОЕКТА

В рамках проекта принимаются следующие ограничения масштаба:

- Максимальное количество плат в системе: **10**
- Максимальное количество дверей на плату: **8**
- Максимальное количество дверей в проекте: **до 80**

2.3.2. ИДЕНТИФИКАЦИЯ ПЛАТ (NODE ID)

Каждый контроллер в CAN-сети имеет уникальный идентификатор платы (**Node ID**).

- Диапазон Node ID: **1...10**
- Node ID используется:
 - для маршрутизации CAN-сообщений;
 - для однозначного определения принадлежности двери плате.

2.3.3. ИДЕНТИФИКАЦИЯ ДВЕРЕЙ

Каждая дверь имеет три идентификатора/поля отображения:

2.3.3.1. ТЕХНИЧЕСКИЙ ИДЕНТИФИКАТОР (TECHID)

Технический идентификатор используется в прошивке и протоколах.

Формат:

- **NxxDyy**, где:
 - xx — Node ID платы (01...10)
 - yy — локальный номер двери на плате (01...08)

Пример: пятая дверь третьей платы → **N03D05**

Технический идентификатор обеспечивает однозначное определение:

- номера платы;
- номера двери на плате.

2.3.3.2. ИДЕНТИФИКАТОР ДВЕРИ НА ЧЕРТЕЖЕ (DRAWINGID)

drawingId — строковый идентификатор двери, соответствующий обозначению в строительной документации.

- Формат drawingId не ограничивается шаблоном и хранится как строка
- drawingId используется:
 - для отображения в WEB;
 - для мониторинга и “карты дверей”;
 - для журналов и отчетов, ориентированных на клиента.

drawingId связан с techId в конфигурации и обеспечивает однозначное сопоставление “как у клиента” ↔ “как в системе”.

2.3.3.3. КОММЕНТАРИЙ (COMMENT)

comment — необязательное текстовое поле для пояснений (расположение двери, назначение, примечания).

- Заполняется по желанию
- Не участвует в логике
- Используется для удобства эксплуатации и мониторинга.

2.3.4. ПРИВЯЗКА ДВЕРЕЙ К КОНТРОЛЛЕРАМ (MAPPING)

Для каждой двери задаётся физическая привязка:

- Node ID контроллера (1...10), где 1 — это плата MASTER.
- Локальный номер двери на плате (1...8)

Правило:

- каждая дверь принадлежит **ровно одному** контроллеру.

2.3.5. ГЛОБАЛЬНЫЙ ИДЕНТИФИКАТОР ДВЕРИ (GLOBALDOORID)

Для внутреннего использования в прошивке вводится глобальный числовой идентификатор.

Формула (1-based):

- **globalDoorId = (nodeId - 1) * 8 + localDoor**

Диапазон:

- при nodeId 1...10 и localDoor 1...8 → globalDoorId 1...80

Назначение:

- индексирование структур данных;
- упрощение таблиц зависимостей;
- единая адресация дверей внутри логики.

2.3.6. ТИП ДВЕРИ

Для каждой двери задаётся тип:

- **NC (Normally Closed)**
- **NO (Normally Open)**
- **CARD READER**

Тип двери влияет на:

- безопасное состояние в режиме Degraded;
- правила разблокировки (CARD READER);
- поведение при инициализации.

2.3.7. ЗАВИСИМОСТИ ДВЕРЕЙ (ТАБЛИЦА ЗАВИСИМОСТЕЙ)

Для каждой двери в конфигурации задаётся список дверей, которые должны быть заблокированы при её физическом открытии. Семантика зависимости:

- если дверь-источник физически **открыта** → все двери-цели должны быть **заблокированы** (если они физически закрыты).

Особенности:

- зависимости могут связывать двери на разных платах;
- одна дверь может иметь несколько зависимостей;
- одна дверь может являться целью зависимостей от нескольких дверей.

2.3.8. СОСТОЯНИЕ ДВЕРИ (DOOR STATE)

Состояние двери включает:

- физическое состояние (открыта/закрыта);
- состояние блокировки (заблокирована/разблокирована);
- логическое состояние (Normal/Degraded/Alarm);
- состояние сигнализации (вкл/выкл);
- состояние таймеров (открыта слишком долго, тайм-аут после закрытия).

2.3.9. ТАЙМ-АУТЫ

В системе используются тайм-ауты:

- **тайм-аут открытой двери** — общий для всех дверей, задаётся в конфигурации;
- **тайм-аут после закрытия** — индивидуальный для каждой двери, задаётся в конфигурации.

2.3.10. КОНФИГУРАЦИЯ ПРОЕКТА

Конфигурация проекта должна содержать:

- список дверей;
- mapping дверей (nodeId + localDoor);
- techId, drawingId, comment;
- тип двери (NC/NO/CARD READER);
- таблицу зависимостей дверей;
- тайм-ауты.

Конфигурация:

- загружается через WEB;
- валидируется;
- применяется атомарно;
- сохраняется во Flash (QSPI).

2.3.11. СНИМОК СОСТОЯНИЯ (ДЛЯ МОНИТОРИНГА)

Мониторинг системы опирается на снимок состояния:

- состояние всех дверей;
- режим системы (Normal/Degraded);
- состояние связи (CAN link state).

Снимок используется:

- WEB-монитором (в т.ч. “карта дверей” по drawingId);
- RS-485 индикацией.

2.4. LOGIC CORE (ЛОГИКА УПРАВЛЕНИЯ ДВЕРЬМИ)

Цель Logic Core — на основании текущих состояний дверей и конфигурации зависимостей **сформировать корректные команды блокировки, индикации и сигнализации**, соблюдая все инварианты безопасности.

Logic Core **не управляет аппаратурой напрямую и не зависит от сетевых интерфейсов**.

2.4.1. НАЗНАЧЕНИЕ LOGIC CORE

Logic Core выполняет следующие функции:

- анализ физических состояний дверей;
 - применение заданных зависимостей между дверями;
 - учёт тайм-аутов;
 - обработку приоритета кнопки Alarm;
 - формирование целевого состояния каждой двери;
 - взаимодействие с Safety Layer перед выдачей команд.
- Logic Core является **единственным источником логических решений** в системе.

2.4.2. ВХОДНЫЕ ДАННЫЕ LOGIC CORE

Logic Core работает на основе следующих входных данных:

- текущее физическое состояние всех дверей (открыта / закрыта);
- текущее состояние блокировки дверей;
- логические состояния дверей (Normal / Degraded / Alarm);
- состояние кнопки Alarm;
- состояние тайм-аутов;
- активная конфигурация проекта;
- состояние сети (Normal / Degraded).

Все входные данные рассматриваются как **снимок состояния**.

2.4.3. ВЫХОДНЫЕ ДАННЫЕ LOGIC CORE

Результатом работы Logic Core являются:

- целевое состояние блокировки каждой двери:
 - заблокировать;
 - разблокировать;
- целевой режим индикации двери:
 - нормальная индикация;
 - сигнализация;
- диагностические флаги для мониторинга.

Logic Core **не применяет** команды напрямую, а передаёт их в Safety Layer.

2.4.4. ОСНОВНОЕ ПРАВИЛО ЛОГИКИ ЗАВИСИМОСТЕЙ

Основное правило логики:

Если дверь физически открыта, все двери, указанные в её списке зависимостей, должны быть заблокированы.

Применение правила:

- выполняется для каждой двери проекта;
- учитывает только физическое состояние двери;
- не зависит от порядка обработки дверей.

2.4.5. ОБРАБОТКА НЕСКОЛЬКИХ ЗАВИСИМОСТЕЙ

Logic Core должен учитывать, что:

- одна дверь может участвовать в нескольких зависимостях;
- одна дверь может быть заблокирована по зависимостям от нескольких дверей одновременно.

Правило:

- если **хотя бы одна** активная зависимость требует блокировки двери — дверь считается требующей блокировки.

2.4.6. УЧЁТ ТАЙМ-АУТА ПОСЛЕ ЗАКРЫТИЯ ДВЕРИ

Для каждой двери может быть задан тайм-аут после закрытия двери.

Logic Core:

- фиксирует момент перехода двери из состояния «открыта» в «закрыта»;
- откладывает применение логики зависимостей для этой двери на заданное время;
- при значении тайм-аута «0» применяет логику немедленно.

2.4.7. УЧЁТ ТАЙМ-АУТА ОТКРЫТОЙ ДВЕРИ

Logic Core контролирует длительность нахождения двери в физически открытом состоянии.

При превышении допустимого времени:

- активируется режим сигнализации двери;
- логика зависимостей продолжает применяться без изменений.

2.4.8. ПРИОРИТЕТ КНОПКИ ALARM

Logic Core учитывает абсолютный приоритет кнопки Alarm:

- при активации Alarm:
 - дверь немедленно переводится в разблокированное состояние;
 - логика зависимостей для данной двери игнорируется;
 - активируется режим сигнализации;
- при повторной активации Alarm:
 - дверь возвращается в состояние, требуемое текущей логикой зависимостей.

Alarm не отключает логику зависимостей для других дверей.

2.4.9. РАБОТА LOGIC CORE В РЕЖИМЕ DEGRADED

В режиме Degraded:

- логика зависимостей **не применяется**;
- каждая дверь рассматривается как независимая;
- Logic Core формирует целевое состояние двери на основе:
 - типа двери (NC / NO / CARD READER);
 - состояния Alarm;
 - тайм-аутов.

2.4.10. ФОРМИРОВАНИЕ ЦЕЛЕВОГО СОСТОЯНИЯ ДВЕРИ

Для каждой двери Logic Core формирует целевое состояние:

- требуется ли блокировка;
- требуется ли разблокировка;
- требуется ли сигнализация.

Целевое состояние:

- передаётся в Safety Layer;
- используется для управления аппаратурой;
- отражается в мониторинге.

2.4.11. ДЕТЕРМИНИРОВАННОСТЬ ЛОГИКИ

Logic Core должен обеспечивать:

- одинаковый результат при одинаковых входных данных;
- независимость от порядка обработки дверей;
- отсутствие “дребезга” логических решений.

2.5. SAFETY LAYER (УРОВЕНЬ БЕЗОПАСНОСТИ И ИНВАРИАНТОВ)

Цель Safety Layer — **гарантировать физическую и логическую безопасность дверей**, независимо от:

- ошибок логики;
- сетевых сбоев;
- некорректной конфигурации;
- асинхронных событий.

Safety Layer является **обязательным фильтром** между Logic Core и аппаратным управлением дверьми.

2.5.1. НАЗНАЧЕНИЕ SAFETY LAYER

Safety Layer выполняет следующие функции:

- контроль выполнения инвариантов безопасности;
- проверка допустимости команд управления дверями;
- разрешение конфликтов между логикой и аварийными событиями;
- предотвращение перехода двери в недопустимое состояние.

Ни одна команда управления дверью **не может миновать Safety Layer**.

2.5.2. ИНВАРИАНТЫ БЕЗОПАСНОСТИ

Safety Layer обеспечивает выполнение следующих инвариантов:

- **Запрещена блокировка физически открытой двери**
- **Активация замка разрешена только при физически закрытой двери**
- **Кнопка Alarm имеет абсолютный приоритет**
- **Потеря сети не должна приводить к опасному состоянию двери**
- **Недопустимые состояния двери не могут быть сгенерированы логикой**

Нарушение инварианта приводит к блокированию соответствующей команды.

2.5.3. ПРОВЕРКА КОМАНД LOGIC CORE

Каждая команда, сформированная Logic Core, проходит проверку:

- соответствие физическому состоянию двери;
- соответствие текущему логическому состоянию;
- соответствие приоритетам (Alarm / Degraded / Normal).

Недопустимые команды:

- не выполняются;
- заменяются безопасным состоянием.

2.5.4. РАЗРЕШЕНИЕ КОНФЛИКТОВ КОМАНД

Возможные конфликты:

- логика требует блокировки, но дверь физически открыта;
- логика требует блокировки, но активен Alarm;
- несколько логических зависимостей требуют противоположных действий.

Правила разрешения:

1. **Alarm всегда побеждает**
2. **Физическое состояние двери имеет приоритет над логикой**
3. **Безопасное состояние предпочтительнее логического**

2.5.5. ПОВЕДЕНИЕ ПРИ АВАРИЙНЫХ СИТУАЦИЯХ

Safety Layer определяет поведение двери при:

- потере CAN-связи;
- некорректной конфигурации;
- отсутствии актуальных данных состояния.

В этих случаях:

- дверь переводится в безопасное состояние;
- логика зависимостей игнорируется;
- Alarm остаётся активным.

2.5.6. КОНТРОЛЬ ЦЕЛОСТНОСТИ ДАННЫХ

Safety Layer:

- проверяет корректность входных данных;
- отслеживает несогласованные состояния;
- может инициировать переход в режим Degraded при необходимости.

2.5.7. ИНТЕГРАЦИЯ С АППАРАТНЫМ УПРАВЛЕНИЕМ

После прохождения Safety Layer:

- формируются окончательные команды управления замком;
- формируются команды индикации и сигнализации;
- команды передаются в драйверы аппаратуры.

Safety Layer не взаимодействует напрямую с сетью.

2.5.8. ДЕТЕРМИНИРОВАННОСТЬ SAFETY LAYER

Safety Layer должен обеспечивать:

- одинаковый результат при одинаковых входных данных;
- отсутствие временной зависимости;
- отсутствие побочных эффектов.

2.6. МОДУЛЬ ИНДИКАЦИИ И СИГНАЛИЗАЦИИ

Цель модуля — обеспечить единообразное и однозначное отображение состояния двери для пользователя, включая режим сигнализации при аварийных и предупредительных событиях.

2.6.1. НАЗНАЧЕНИЕ МОДУЛЯ

Модуль индикации и сигнализации выполняет:

- управление красным и зелёным индикаторами;
- управление зуммером;
- реализацию режима сигнализации;
- отображение состояний двери согласно принятым правилам приоритетов.

Модуль принимает решения **только на основании целевого состояния**, сформированного Logic Core и подтверждённого Safety Layer.

2.6.2. НОРМАЛЬНАЯ ИНДИКАЦИЯ СОСТОЯНИЯ ДВЕРИ

В нормальном режиме индикация отражает состояние блокировки двери:

- **красный** — дверь заблокирована;
- **зелёный** — дверь разблокирована.

Нормальная индикация применяется во всех логических состояниях, если не активен режим сигнализации.

2.6.3. РЕЖИМ СИГНАЛИЗАЦИИ

Сигнализация — специальный режим индикации двери.

Режим сигнализации включается при:

- разблокировке двери через **Alarm**;
- превышении тайм-аута открытой двери.

2.6.4. ПОВЕДЕНИЕ В РЕЖИМЕ СИГНАЛИЗАЦИИ

В режиме сигнализации:

- красный и зелёный индикаторы переключаются поочерёдно;
- период переключения каждого состояния составляет **1 секунду**;
- зуммер подаёт периодический звуковой сигнал.

Режим сигнализации действует до момента завершения условия, вызвавшего его включение, согласно правилам логики и приоритетов.

2.6.5. ПРИОРИТЕТЫ ОТОБРАЖЕНИЯ

Приоритеты режимов отображения:

1. Сигнализация (Alarm / тайм-аут открытой двери)
2. Нормальная индикация состояния блокировки

Сигнализация имеет приоритет над обычной индикацией независимо от того, заблокирована дверь или разблокирована.

2.6.6. ОБРАБОТКА ОДНОВРЕМЕННЫХ ПРИЧИН СИГНАЛИЗАЦИИ

Если сигнализация вызвана одновременно несколькими причинами (например Alarm и тайм-аут открытой двери), модуль должен обеспечивать:

- непрерывность режима сигнализации;
- корректное завершение сигнализации только после завершения всех активных причин.

2.6.7. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- Модуль индикации не должен:
 - изменять состояние блокировки двери;
 - влиять на логику зависимостей;
 - зависеть от наличия сети.
- Индикация должна быть однозначной и понятной пользователю.

2.7. МОДУЛЬ СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ

Цель модуля сетевого взаимодействия — обеспечить обмен данными между контроллерами и внешними системами **без влияния на контур управления дверьми**.

Модуль сетевого взаимодействия **не принимает логических решений и не управляет дверьми напрямую**.

2.7.1. CAN-ВЗАИМОДЕЙСТВИЕ (MASTER–SLAVE)

CAN используется для связи между контроллерами в режиме MASTER–SLAVE.

2.7.1.1. НАЗНАЧЕНИЕ CAN-ВЗАИМОДЕЙСТВИЯ

CAN-взаимодействие обеспечивает:

- передачу состояний дверей от SLAVE к MASTER;
- передачу логических команд от MASTER к SLAVE;
- контроль доступности SLAVE-контроллеров.

2.7.1.2. РОЛИ В CAN-СЕТИ

- **MASTER (Node ID = 1)**
 - принимает состояния дверей от всех SLAVE;
 - вычисляет логику зависимостей;
 - формирует команды управления дверьми;
 - отслеживает состояние сети.
- **SLAVE (Node ID = 2...10)**
 - передаёт состояния своих дверей;
 - принимает и исполняет команды MASTER;
 - обеспечивает локальную безопасность дверей.

2.7.1.3. ПЕРЕДАЧА СОСТОЯНИЙ ДВЕРЕЙ

SLAVE-контроллеры периодически передают MASTER:

- физическое состояние дверей;
- состояние блокировки;
- логическое состояние;
- состояние сигнализации;
- состояние тайм-аутов.

MASTER использует эти данные как вход для Logic Core.

2.7.1.4. ПЕРЕДАЧА КОМАНД УПРАВЛЕНИЯ

MASTER передаёт SLAVE:

- команды блокировки/разблокировки дверей;
- команды управления режимом сигнализации.

SLAVE:

- проверяет команды через Safety Layer;
- применяет допустимые команды к аппаратуре.

2.7.1.5. КОНТРОЛЬ СВЯЗИ И ПЕРЕХОД В DEGRADED

CAN-модуль:

- отслеживает тайм-ауты сообщений;
- определяет потерю связи с отдельным SLAVE или всей сетью.

При потере связи:

- система переходит в режим **Degraded**;
- логика зависимостей временно не применяется;
- двери работают автономно в безопасном режиме.

2.7.2. ETHERNET-ВЗАИМОДЕЙСТВИЕ (WEB / HTTP)

Ethernet используется для конфигурирования и мониторинга системы.

2.7.2.1. НАЗНАЧЕНИЕ ETHERNET

Ethernet обеспечивает:

- загрузку конфигурации проекта;
 - получение текущего состояния системы;
 - работу WEB-монитора (включая “карту дверей”).
- Ethernet **не используется** для управления дверьми в runtime.

2.7.2.2. WEB-КОНФИГУРАТОР

Через WEB-интерфейс выполняется:

- добавление и настройка дверей;
- задание зависимостей дверей;
- настройка тайм-аутов;
- привязка techId ↔ drawingId ↔ comment.

Все изменения:

- валидируются;
- применяются атомарно;
- сохраняются во Flash.

2.7.2.3. WEB-МОНИТОРИНГ

WEB-мониторинг предоставляет:

- текущее состояние дверей;
- отображение дверей по drawingId;
- отображение сигнализации и Alarm-событий;
- индикацию режима системы (Normal / Degraded).

Мониторинг:

- носит наблюдательный характер;
- может выполняться с высокой частотой обновления;
- не влияет на логику управления.

2.7.3. RS-485 (ВНЕШНЯЯ ИНДИКАЦИЯ, READ-ONLY)

RS-485 используется для передачи информации о состоянии дверей во внешние системы.

2.7.3.1. НАЗНАЧЕНИЕ RS-485

RS-485 обеспечивает:

- передачу текущих состояний дверей;
- передачу системных флагов (Alarm, Degraded).

RS-485 **не используется** для:

- изменения конфигурации;
- управления дверьми;
- вмешательства в логику.

2.7.3.2. ПРИНЦИПЫ РАБОТЫ

- Передача данных осуществляется:
 - по запросу внешнего устройства;
 - либо периодически (в зависимости от протокола).
- Формат данных ориентирован на:
 - внешнюю индикацию;
 - SCADA / диспетчеризацию.

2.7.4. ОБЩИЕ ПРИНЦИПЫ СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ

- Ни один сетевой интерфейс:
 - не может обойти Safety Layer;
 - не может напрямую управлять замками.
- Потеря любого сетевого интерфейса:
 - не влияет на локальную безопасность дверей;
 - не блокирует кнопку Alarm.
- Все сетевые события учитываются как входные данные, но не как управляющие.

2.8. МОДУЛЬ УПРАВЛЕНИЯ КОНФИГУРАЦИЕЙ И ХРАНЕНИЯ ДАННЫХ (QSPI FLASH)

Цель модуля — обеспечить **надёжное, проверяемое и безопасное хранение конфигурации проекта**, а также её корректное и предсказуемое применение.

Модуль конфигурации **не управляет дверьми напрямую и не участвует в логике зависимостей**.

2.8.1. НАЗНАЧЕНИЕ МОДУЛЯ

Модуль управления конфигурацией выполняет:

- приём конфигурации через WEB-интерфейс;
- проверку корректности и целостности данных;
- безопасное применение конфигурации;
- хранение конфигурации в QSPI Flash;

- восстановление конфигурации после перезапуска;
- откат к последней валидной конфигурации при ошибках.

2.8.2. СОСТАВ КОНФИГУРАЦИИ ПРОЕКТА

Конфигурация проекта включает:

- описание всех дверей проекта;
- techId, drawingId, comment для каждой двери;
- привязку двери к контроллеру (Node ID, localDoor);
- тип двери (NC / NO / CARD READER);
- таблицу зависимостей дверей;
- параметры тайм-аутов;
- **служебные сетевые параметры:**
 - CAN (Node ID, скорость, тайм-ауты связи);
 - Ethernet (IP/DHCP, порт WEB);
- служебные метаданные конфигурации.

Сетевые параметры:

- не участвуют в логике блокировки дверей;
- используются для обеспечения связи и диагностики.

2.8.3. ВАЛИДАЦИЯ КОНФИГУРАЦИИ

Перед применением конфигурация проходит обязательную валидацию:

- корректность диапазонов Node ID и localDoor;
- уникальность techId и globalDoorId;
- корректность ссылок в таблице зависимостей;
- допустимость значений тайм-аутов;
- соответствие масштабу системы (до 10 плат, до 80 дверей);
- отсутствие конфигураций, нарушающих инварианты безопасности.

Некорректная конфигурация **не может быть активирована**.

2.8.4. АТОМАРНОЕ ПРИМЕНЕНИЕ КОНФИГУРАЦИИ

Применение конфигурации выполняется атомарно:

- конфигурация загружается во временную область;
- выполняется полная проверка целостности и валидности;
- только после успешной проверки конфигурация становится активной;
- при любой ошибке система продолжает работу с предыдущей конфигурацией.

Не допускается частичное применение конфигурации.

2.8.5. ХРАНЕНИЕ КОНФИГУРАЦИИ (QSPI FLASH)

Конфигурация хранится в QSPI Flash:

- в сериализованном виде;
- с контрольной суммой (CRC);
- с указанием версии формата конфигурации.

Рекомендуется хранить:

- активную конфигурацию;
- резервную (последнюю валидную) конфигурацию.

2.8.6. ВЕРСИЯ ФОРМАТА КОНФИГУРАЦИИ

Версия формата конфигурации определяет структуру и набор полей конфигурационных данных.

Назначение:

- обеспечение совместимости между версиями прошивки;
- корректная обработка старых конфигураций;
- предотвращение некорректного применения несовместимых данных.

При несовпадении версии формата:

- конфигурация не активируется;
- система переходит в безопасный режим или использует резервную конфигурацию.

2.8.7. ВОССТАНОВЛЕНИЕ ПОСЛЕ ПЕРЕЗАПУСКА

При старте системы:

- конфигурация считывается из QSPI Flash;
- проверяется целостность данных;
- при успехе конфигурация активируется;
- при ошибке выполняется откат к резервной конфигурации;
- при отсутствии валидной конфигурации система стартует в безопасном режиме.

2.8.8. ДОСТУП К ИНФОРМАЦИИ О КОНФИГУРАЦИИ

Через WEB-интерфейс должна быть доступна информация:

- версия активной конфигурации;
- версия формата конфигурации;

- дата и время загрузки конфигурации;
 - краткое описание проекта.
- WEB-интерфейс предоставляет **доступ только на чтение** к активной конфигурации.

2.8.9. СВЯЗЬ С ДИАГНОСТИКОЙ

Ошибки загрузки, валидации и применения конфигурации:

- фиксируются в журнале событий;
- доступны для просмотра через WEB-интерфейс.

2.9. ДИАГНОСТИКА И ЖУРНАЛИРОВАНИЕ

Цель раздела — определить, **какие события фиксируются системой**, где и как они хранятся, и каким образом доступны для анализа.

2.9.1. НАЗНАЧЕНИЕ ДИАГНОСТИКИ

Диагностика предназначена для:

- анализа работы системы;
- выявления причин сбоев;
- подтверждения корректной работы логики;
- упрощения обслуживания и эксплуатации.

2.9.2. ЖУРНАЛ СОБЫТИЙ (EVENT LOG)

Система ведёт журнал событий.

Журнал фиксирует:

- физическое открытие двери;
- физическое закрытие двери;
- активацию и деактивацию кнопки Alarm;
- превышение тайм-аута открытой двери;
- переход системы в режим Degraded;
- восстановление нормального режима;
- потерю и восстановление CAN-связи;
- загрузку, применение и откат конфигурации;
- ошибки конфигурации;
- перезапуски системы.

2.9.3. ХРАНЕНИЕ ЖУРНАЛА

Журнал событий хранится:

- в QSPI Flash;
- в кольцевом буфере (ограниченный объём);
- с сохранением порядка событий.

При переполнении:

- старые записи перезаписываются новыми.

2.9.4. СТРУКТУРА ЗАПИСИ ЖУРНАЛА

Каждая запись журнала должна содержать:

- временную метку;
- тип события;
- идентификатор двери (techId, при наличии);
- дополнительную информацию (код причины).

2.9.5. ДОСТУП К ЖУРНАЛУ

Журнал событий доступен:

- через WEB-интерфейс;
- в виде списка событий;
- с возможностью фильтрации по типу события и двери.

WEB-доступ:

- только на чтение;
- не влияет на работу системы.

2.9.6. ДИАГНОСТИЧЕСКИЕ СТАТУСЫ

Система формирует диагностические флаги:

- состояние CAN-сети;
- состояние Ethernet;
- режим системы (Normal / Degraded);
- состояние Alarm.

Диагностические статусы включаются в мониторинг.

2.9.7. ОГРАНИЧЕНИЯ ДИАГНОСТИКИ

- Диагностика не должна влиять на:
 - логику управления;
 - тайминги работы дверей;
 - безопасность системы.
- Потеря журнала не влияет на работу дверей.

3. НАДЁЖНОСТЬ, ОТКАЗОУСТОЙЧИВОСТЬ И FAIL-SAFE

3.1. ОБЩИЕ ПРИНЦИПЫ НАДЁЖНОСТИ

Система управления дверьми относится к классу **промышленных управляющих систем с требованиями к безопасности**, поэтому при её проектировании и реализации применяются следующие базовые принципы.

3.1.1. ПРИНЦИП БЕЗОПАСНОГО ОТКАЗА (FAIL-SAFE)

При любом отказе системы или её части:

- дверь **не должна переходить в опасное состояние**;
- при неопределённости система выбирает **безопасное состояние двери** в соответствии с её типом (NC / NO / CARD READER);
- логика зависимостей не должна приводить к неконтролируемым действиям.

3.1.2. ЛОКАЛЬНАЯ БЕЗОПАСНОСТЬ ДВЕРИ

Каждая дверь должна сохранять безопасное поведение независимо от:

- состояния CAN-сети;
- состояния Ethernet;
- корректности работы других плат;
- состояния WEB-интерфейса.

Кнопка **Alarm** и инварианты безопасности действуют **локально** и имеют приоритет над любой распределённой логикой.

3.1.3. РАЗДЕЛЕНИЕ ОТВЕТСТВЕННОСТИ

Для обеспечения надёжности система разделяется на уровни ответственности:

- Logic Core — принятие логических решений;
- Safety Layer — контроль допустимости решений;
- аппаратные драйверы — физическое управление.

Отказ одного уровня не должен автоматически приводить к опасному действию на другом уровне.

3.1.4. ДЕТЕРМИНИРОВАННОСТЬ ПОВЕДЕНИЯ

При одинаковых входных условиях система должна:

- всегда принимать одинаковые решения;
- не зависеть от порядка обработки событий;
- не иметь скрытых состояний, влияющих на безопасность.

3.1.5. НЕЗАВИСИМОСТЬ ОТ СЕТЕВЫХ ИНТЕРФЕЙСОВ

Работа дверей и их безопасность:

- **не зависят** от наличия Ethernet;
- **ограниченно зависят** от CAN (только для логики зависимостей);
- не зависят от RS-485.

Потеря связи не считается аварией двери, а рассматривается как штатный сценарий деградации.

3.1.6. ПРИНЦИП КОНТРОЛИРУЕМОГО ВОССТАНОВЛЕНИЯ

После устранения отказа:

- система должна восстанавливаться в предсказуемое состояние;
- не допускаются резкие или хаотичные переключения дверей;
- восстановление логики происходит мягко и последовательно.

3.1.7. ЯВНАЯ ФИКСАЦИЯ ОТКАЗОВ

Все значимые отказы:

- фиксируются в журнале событий;
- доступны для диагностики;
- не маскируются автоматическими перезапусками без регистрации факта сбоя.

3.2. АППАРАТНЫЙ WATCHDOG

Аппаратный watchdog используется как **последний уровень защиты** от зависаний и некорректного выполнения программного кода.

3.2.1. НАЗНАЧЕНИЕ АППАРАТНОГО WATCHDOG

Аппаратный watchdog предназначен для:

- обнаружения зависания процессора;
- автоматического перезапуска системы при потере управления;
- возврата системы в контролируемое и безопасное состояние.

Watchdog рассматривается как **обязательный элемент промышленной надёжности**, а не как отладочная опция.

3.2.2. ОБЩИЕ ПРИНЦИПЫ ИСПОЛЬЗОВАНИЯ

Аппаратный watchdog:

- включён постоянно в рабочем режиме;
- не может быть отключён пользователем;
- обслуживается только при корректном выполнении программы.

Сброс watchdog возможен **только при прохождении штатного рабочего цикла системы**.

3.2.3. УСЛОВИЯ ОБСЛУЖИВАНИЯ WATCHDOG

Обслуживание watchdog допускается только если:

- основной цикл Logic Core выполняется без ошибок;
- Safety Layer не зафиксировал критических нарушений;
- система не находится в состоянии необработанного исключения;
- основные задачи RTOS находятся в рабочем состоянии.

Если любое из условий не выполнено — watchdog **не обслуживается**.

3.2.4. ПОВЕДЕНИЕ ПРИ СРАБАТЫВАНИИ WATCHDOG

При срабатывании аппаратного watchdog:

- микроконтроллер выполняет аппаратный reset;
- питание исполнительных выходов переинициализируется;
- система запускается в соответствии с процедурой старта;
- факт перезапуска фиксируется в журнале событий.

3.2.5. БЕЗОПАСНОЕ СОСТОЯНИЕ ПРИ ПЕРЕЗАПУСКЕ

После перезапуска:

- двери приводятся в безопасное состояние согласно их типу (NC / NO / CARD READER);
- логика зависимостей не применяется до завершения инициализации;
- Alarm остаётся доступным локально;
- выполняется восстановление конфигурации из QSPI Flash.

3.2.6. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- Аппаратный watchdog не заменяет программную диагностику;
- watchdog не используется для маскировки ошибок логики;
- повторные срабатывания watchdog рассматриваются как аварийная ситуация и должны быть видны в диагностике.

3.3. ПРОГРАММНЫЙ КОНТРОЛЬ ЗАВИСАНИЙ

Программный контроль зависаний дополняет аппаратный watchdog и предназначен для **раннего обнаружения некорректной работы отдельных модулей**, до того как потребуется аппаратный перезапуск.

3.3.1. НАЗНАЧЕНИЕ ПРОГРАММНОГО КОНТРОЛЯ

Программный контроль зависаний используется для:

- обнаружения зависания отдельных задач RTOS;
- выявления логических закликиваний;
- контроля регулярности выполнения критических модулей;
- фиксации деградаций, не приводящих сразу к аппаратному reset.

3.3.2. КОНТРОЛИРУЕМЫЕ КОМПОНЕНТЫ

Программный контроль охватывает:

- Logic Core;
- Safety Layer;
- задачу обмена по CAN;
- задачу обработки событий дверей;
- задачу индикации и сигнализации.

Каждый контролируемый компонент рассматривается как **логический участник системы**.

3.3.3. МЕХАНИЗМ “HEARTBEAT” МОДУЛЕЙ

Для программного контроля используется механизм heartbeat:

- каждый контролируемый модуль периодически подтверждает свою работоспособность;
- подтверждение фиксируется в общей структуре состояния;

- отсутствие heartbeat в заданный интервал считается признаком зависания.

Интервалы heartbeat:

- фиксированные;
- не зависят от нагрузки системы;
- выбираются с запасом по времени.

3.3.4. РЕАКЦИЯ НА ОТСУТСТВИЕ HEARTBEAT

При отсутствии heartbeat от модуля:

- фиксируется диагностическое событие;
- система переводится в режим Degraded (если применимо);
- инициируется процедура восстановления;
- при критическом характере сбоя — подготавливается аппаратный reset.

3.3.5. СВЯЗЬ С АППАРАТНЫМ WATCHDOG

Программный контроль зависаний:

- является источником решения о том, **обслуживать или не обслуживать аппаратный watchdog**;
- при критическом сбое блокирует обслуживание watchdog;
- тем самым гарантирует аппаратный перезапуск системы.

3.3.6. ЖУРНАЛИРОВАНИЕ СБОЕВ

Все обнаруженные программные зависания:

- фиксируются в журнале событий;
- сопровождаются указанием модуля-источника;
- доступны для анализа через WEB-интерфейс.

3.3.7. ОГРАНИЧЕНИЯ

Программный контроль зависаний:

- не вмешивается напрямую в логику дверей;
- не изменяет состояние дверей без участия Safety Layer;
- не должен создавать ложных срабатываний при штатной нагрузке.

3.4. ПОВЕДЕНИЕ ПРИ ЗАВИСАНИИ ЗАДАЧ

Данный пункт описывает поведение системы при зависании одной или нескольких задач RTOS, а также порядок перехода в безопасные режимы.

3.4.1. ОБЩИЕ ПОЛОЖЕНИЯ

Под зависанием задачи понимается состояние, при котором задача:

- не выполняет свой основной функционал;
- не обновляет heartbeat в заданный интервал;
- не реагирует на события или сообщения.

Зависание задачи рассматривается как **нештатная ситуация**, потенциально влияющая на безопасность.

3.4.2. КЛАССИФИКАЦИЯ ЗАДАЧ ПО КРИТИЧНОСТИ

Все задачи системы логически делятся на:

- **критические задачи:**
 - Logic Core;
 - Safety Layer;
 - обработка состояний дверей;
- **вспомогательные задачи:**
 - индикация;
 - сетевые интерфейсы (Ethernet, RS-485);
 - WEB-сервер.

Критичность задачи определяет реакцию системы на её зависание.

3.4.3. ЗАВИСАНИЕ ВСПОМОГАТЕЛЬНОЙ ЗАДАЧИ

При зависании вспомогательной задачи:

- фиксируется диагностическое событие;
- работа дверей и логики продолжается;
- задача может быть перезапущена программно (если предусмотрено);
- аппаратный watchdog не блокируется.

Пример:

- зависание WEB-интерфейса не влияет на управление дверьми.

3.4.4. ЗАВИСАНИЕ КРИТИЧЕСКОЙ ЗАДАЧИ

При зависании критической задачи:

- фиксируется диагностическое событие;
- система переводится в режим Degraded;

- логика зависимостей временно приостанавливается;
 - обслуживание аппаратного watchdog блокируется.
- Если зависание не устранено — происходит аппаратный reset.

3.4.5. ОДНОВРЕМЕННОЕ ЗАВИСАНИЕ НЕСКОЛЬКИХ ЗАДАЧ

При зависании нескольких задач:

- приоритет имеет худший сценарий;
- если среди зависших есть критическая задача — применяется сценарий для критической задачи;
- система готовится к перезапуску.

3.4.6. ПОВЕДЕНИЕ ДВЕРЕЙ ПРИ ЗАВИСАНИИ ЗАДАЧ

При зависании задач:

- двери сохраняют текущее физическое состояние;
- новые команды блокировки формируются только при прохождении Safety Layer;
- Alarm продолжает работать локально;
- недопустимые состояния двери исключаются.

3.4.7. ЖУРНАЛИРОВАНИЕ И ДИАГНОСТИКА

События зависания задач:

- записываются в журнал событий;
- содержат информацию о задаче и типе зависания;
- доступны для анализа после восстановления системы.

3.5. ПОВЕДЕНИЕ ПРИ ОТКАЗЕ CAN

Данный пункт определяет поведение системы при частичной или полной потере связи по CAN в режиме MASTER–SLAVE.

3.5.1. ОПРЕДЕЛЕНИЕ ОТКАЗА CAN

Отказ CAN фиксируется при одном или нескольких условиях:

- превышение тайм-аута получения сообщений от SLAVE (для MASTER);
- превышение тайм-аута получения команд от MASTER (для SLAVE);
- аппаратные ошибки CAN-контроллера;
- переход CAN-контроллера в состояние bus-off.

Отказ CAN считается **штатным аварийным сценарием**, а не критической ошибкой системы.

3.5.2. ПОВЕДЕНИЕ MASTER ПРИ ОТКАЗЕ CAN

При обнаружении отказа CAN MASTER:

- фиксирует диагностическое событие;
- помечает недоступные SLAVE-контроллеры;
- приостанавливает логику зависимостей, затрагивающую недоступные двери;
- переводит систему в режим **Degraded**.

MASTER не формирует логические команды для дверей, состояние которых невозможно определить достоверно.

3.5.3. ПОВЕДЕНИЕ SLAVE ПРИ ОТКАЗЕ CAN

При потере связи с MASTER SLAVE:

- фиксирует диагностическое событие;
- переходит в режим **Degraded**;
- прекращает выполнение логики зависимостей;
- управляет дверями автономно в безопасном режиме.

SLAVE не ожидает восстановления связи для обеспечения локальной безопасности дверей.

3.5.4. ПОВЕДЕНИЕ ДВЕРЕЙ ПРИ ОТКАЗЕ CAN

При отказе CAN:

- логика зависимостей не применяется;
- каждая дверь работает как независимая;
- дверь переводится в безопасное состояние:
 - NC / CARD READER → заблокирована;
 - NO → разблокирована;
- Alarm всегда работает локально;
- тайм-ауты и сигнализация продолжают работать.

3.5.5. ВОССТАНОВЛЕНИЕ ПОСЛЕ ОТКАЗА CAN

После восстановления CAN-связи:

- фиксируется диагностическое событие;
- выполняется ресинхронизация состояний дверей;
- логика зависимостей активируется вновь;
- возврат логики происходит мягко, без резких переключений.

3.5.6. ЖУРНАЛИРОВАНИЕ ОТКАЗОВ CAN

Фиксируются события:

- потеря связи с MASTER / SLAVE;
- восстановление связи;
- переходы Normal ↔ Degraded, связанные с CAN.

3.6. ПОВЕДЕНИЕ ПРИ НЕДОСТУПНОСТИ ETHERNET И WEB-СЕАНСЕ

3.6.1. РОЛЬ ETHERNET И WEB В СИСТЕМЕ

Ethernet используется исключительно для:

- конфигурирования системы;
- сервисного мониторинга;
- диагностики и обслуживания.

Ethernet **не является обязательным интерфейсом** для штатной работы системы управления дверьми. После ввода в эксплуатацию система может работать автономно без WEB-подключения.

3.6.2. WEB-СЕАНС УПРАВЛЕНИЯ (WEB SESSION)

Для работы с WEB-конфигуратором используется **явный WEB-сеанс управления**.

WEB-сеанс:

- инициируется пользователем вручную;
- требует аутентификации (логин / пароль);
- устанавливается только с **MASTER-контроллером**;
- имеет чёткие состояния: *подключён* / *отключён*.

Система **не использует тайм-ауты активности WEB-сеанса** для его завершения.

3.6.3. ПОДКЛЮЧЕНИЕ WEB-КЛИЕНТА

Подключение WEB-клиента выполняется по явному действию пользователя:

- пользователь нажимает кнопку **«Подключиться к контроллеру»**;
- выполняется аутентификация к MASTER;
- при успешном подключении:
 - WEB-сеанс считается активным;
 - в интерфейсе отображается состояние **«Подключено»**;
 - MASTER фиксирует активный WEB-сеанс.

Активный WEB-сеанс означает, что система находится в режиме обслуживания.

3.6.4. ОТКЛЮЧЕНИЕ WEB-КЛИЕНТА

Завершение работы выполняется явным действием пользователя:

- пользователь нажимает кнопку **«Отключиться»**;
- WEB-сеанс корректно закрывается;
- MASTER переводит систему в автономный режим работы.

После отключения:

- Ethernet считается неактивным;
- система продолжает работу без WEB.

3.6.5. ОБРЫВ СВЯЗИ ВО ВРЕМЯ WEB-СЕАНСА

Если во время активного WEB-сеанса происходит потеря Ethernet-связи:

- фиксируется событие **отказа Ethernet в режиме обслуживания**;
- активный WEB-сеанс принудительно завершается;
- система остаётся на **последней валидной конфигурации**;
- логика дверей продолжает работать без изменений;
- система ожидает нового подключения пользователя через кнопку «Подключиться».

Обрыв связи во время WEB-сеанса **не влияет** на безопасность дверей.

3.6.6. ЗАГРУЗКА КОНФИГУРАЦИИ ЧЕРЕЗ WEB

Загрузка конфигурации через WEB является **транзакционной операцией**.

Во время загрузки:

- активен WEB-сеанс;
- включён усиленный контроль целостности данных;
- конфигурация принимается во временную область.

Конфигурация считается применённой **только после явного подтверждения от MASTER**.

3.6.7. ОБРЫВ СВЯЗИ ВО ВРЕМЯ ЗАГРУЗКИ КОНФИГУРАЦИИ

Если Ethernet-связь теряется во время загрузки конфигурации:

- загрузка считается неуспешной;
- конфигурация **не активируется**;
- система продолжает работу с предыдущей конфигурацией;
- фиксируется диагностическое событие;

- пользователь должен повторно подключиться и повторить операцию.
- Частичное или повреждённое применение конфигурации **исключено**.

3.6.8. ПОВЕДЕНИЕ ДВЕРЕЙ ПРИ ЛЮБЫХ ПРОБЛЕМАХ ETHERNET

При любой недоступности Ethernet или WEB:

- состояние дверей не изменяется;
- логика блокировки не пересчитывается;
- Alarm продолжает работать локально;
- система не переходит в режим Degraded.

Ethernet **никогда не является триггером деградации логики дверей**.

3.6.9. ЖУРНАЛИРОВАНИЕ СОБЫТИЙ WEB И ETHERNET

В журнал событий могут фиксироваться:

- начало WEB-сеанса;
- завершение WEB-сеанса;
- обрыв связи во время активного WEB-сеанса;
- успешная загрузка конфигурации;
- ошибка загрузки конфигурации.

Отсутствие Ethernet без активного WEB-сеанса **не является событием**.

3.7. ПОВЕДЕНИЕ ПРИ СБОЯХ ПАМЯТИ (QSPI FLASH)

Данный пункт описывает поведение системы при ошибках чтения, записи или повреждении данных в QSPI Flash, используемой для хранения конфигурации и журнала событий.

3.7.1. РОЛЬ QSPI FLASH В СИСТЕМЕ

QSPI Flash используется для:

- хранения активной конфигурации проекта;
- хранения резервной (последней валидной) конфигурации;
- хранения журнала событий.

QSPI Flash **не участвует в контуре реального времени** управления дверьми.

3.7.2. ТИПЫ ВОЗМОЖНЫХ СБОЕВ QSPI FLASH

В системе различаются следующие типы сбоев памяти:

- ошибка чтения данных;
- ошибка записи данных;
- нарушение контрольной суммы (CRC);
- отсутствие валидной конфигурации;
- физическая недоступность памяти.

3.7.3. ОБНАРУЖЕНИЕ СБОЕВ ПАМЯТИ

Сбои QSPI Flash обнаруживаются:

- при старте системы (чтение конфигурации);
- при применении новой конфигурации;
- при записи журнала событий.

Каждый сбой сопровождается диагностическим событием.

3.7.4. ПОВЕДЕНИЕ СИСТЕМЫ ПРИ СБОЕ ЧТЕНИЯ КОНФИГУРАЦИИ

Если при старте или работе обнаружена ошибка чтения активной конфигурации:

- активная конфигурация признаётся недействительной;
- выполняется попытка загрузки резервной конфигурации;
- при успешной загрузке резервной конфигурации:
 - система продолжает работу;
 - фиксируется диагностическое событие;
- при отсутствии валидной резервной конфигурации:
 - система переходит в безопасный режим;
 - логика зависимостей не применяется.

3.7.5. ПОВЕДЕНИЕ ПРИ СБОЕ ЗАПИСИ КОНФИГУРАЦИИ

Если при записи новой конфигурации в QSPI Flash возникает ошибка:

- конфигурация **не активируется**;
- активной остаётся предыдущая валидная конфигурация;
- система продолжает работу без изменений;
- фиксируется диагностическое событие;
- пользователю возвращается сообщение об ошибке через WEB.

3.7.6. ПОВЕДЕНИЕ ПРИ СБОЯХ ЖУРНАЛА СОБЫТИЙ

Если возникает ошибка записи в журнал событий:

- запись события может быть пропущена;
 - управление дверьми продолжается без изменений;
 - система не переходит в аварийный режим;
 - при возможности фиксируется диагностический флаг.
- Отказ журнала **не влияет** на безопасность дверей.

3.7.7. ПОВЕДЕНИЕ ДВЕРЕЙ ПРИ СБОЯХ QSPI FLASH

При любых сбоях QSPI Flash:

- двери продолжают работать по последней валидной конфигурации;
- логика зависимостей выполняется только при наличии валидной конфигурации;
- Alarm продолжает работать локально;
- недопустимые состояния дверей исключены Safety Layer.

3.7.8. ВОССТАНОВЛЕНИЕ ПОСЛЕ СБОЕВ ПАМЯТИ

После восстановления работоспособности QSPI Flash:

- система может принять новую конфигурацию через WEB;
- журнал событий продолжает работу с новой области памяти;
- автоматического изменения состояний дверей не происходит.

3.7.9. ЖУРНАЛИРОВАНИЕ СБОЕВ ПАМЯТИ

В журнал событий фиксируются:

- ошибки чтения конфигурации;
- ошибки записи конфигурации;
- использование резервной конфигурации;
- отсутствие валидной конфигурации.

3.8. ПОВЕДЕНИЕ СИСТЕМЫ ПОСЛЕ ПЕРЕЗАПУСКА

Данный пункт описывает порядок действий системы после перезапуска, независимо от причины перезапуска (power-on, reset, watchdog, программный сбой).

3.8.1. ПРИЧИНЫ ПЕРЕЗАПУСКА

Перезапуск системы может быть вызван:

- включением питания;
- аппаратным reset;
- срабатыванием аппаратного watchdog;
- программной инициацией перезапуска;
- критической ошибкой выполнения.

Причина перезапуска должна быть определена и зафиксирована в диагностике.

3.8.2. НАЧАЛЬНОЕ СОСТОЯНИЕ СИСТЕМЫ

После перезапуска система всегда стартует в **безопасном начальном состоянии**.

Начальные условия:

- логика зависимостей **не активна**;
- команды блокировки от Logic Core не применяются;
- система готовится к инициализации модулей.

3.8.3. ИНИЦИАЛИЗАЦИЯ АППАРАТНЫХ РЕСУРСОВ

В процессе старта выполняется:

- инициализация GPIO и исполнительных выходов;
- инициализация CAN, Ethernet, RS-485;
- инициализация таймеров и RTOS;
- инициализация QSPI Flash.

До завершения инициализации **управление дверьми не производится**.

3.8.4. ВОССТАНОВЛЕНИЕ КОНФИГУРАЦИИ

После инициализации памяти:

- выполняется чтение активной конфигурации из QSPI Flash;
- проверяется целостность данных (CRC);
- при успешной проверке конфигурация принимается как рабочая;
- при ошибке выполняется загрузка резервной конфигурации;
- при отсутствии валидной конфигурации система остаётся в безопасном режиме.

3.8.5. ПРИВЕДЕНИЕ ДВЕРЕЙ В БЕЗОПАСНОЕ СОСТОЯНИЕ

После загрузки конфигурации каждая дверь приводится в безопасное состояние:

- двери типа **NC / CARD READER** — заблокированы;
- двери типа **NO** — разблокированы.

Блокировка выполняется **только при физически закрытой двери**.

3.8.6. ЗАПУСК ЛОГИКИ УПРАВЛЕНИЯ

После успешной инициализации:

- запускается Logic Core;
- активируется Safety Layer;
- начинается приём состояний дверей;
- логика зависимостей включается поэтапно.

Резкие или мгновенные переключения дверей не допускаются.

3.8.7. ВОССТАНОВЛЕНИЕ СЕТЕВЫХ ВЗАИМОДЕЙСТВИЙ

После перезапуска:

- CAN-сеть проходит процедуру повторного обнаружения SLAVE;
- состояние сети фиксируется;
- Ethernet доступен только после инициализации WEB-сеанса;
- RS-485 начинает передачу статусов.

3.8.8. ЖУРНАЛИРОВАНИЕ ПЕРЕЗАПУСКА

В журнал событий записываются:

- факт перезапуска системы;
- причина перезапуска (если определена);
- информация о восстановленной конфигурации.

3.8.9. ПОВЕДЕНИЕ ПРИ ПОВТОРНЫХ ПЕРЕЗАПУСКАХ

Если система фиксирует частые перезапуски:

- каждый перезапуск регистрируется;
- система остаётся в безопасном режиме до вмешательства оператора;
- конфигурирование возможно только через WEB.

3.9. БЕЗОПАСНЫЕ СОСТОЯНИЯ ДВЕРЕЙ

Данный пункт фиксирует **набор допустимых и безопасных состояний дверей** и правила перехода между ними при любых аварийных, деградированных и переходных режимах работы системы.

3.9.1. ПОНЯТИЕ БЕЗОПАСНОГО СОСТОЯНИЯ ДВЕРИ

Безопасное состояние двери — это состояние, при котором:

- дверь не создаёт угрозы для людей и оборудования;
- исключена возможность механического повреждения замка;
- соблюдаются инварианты физической безопасности;
- дверь может быть аварийно разблокирована локально.

Безопасное состояние определяется **типом двери** и её **физическим состоянием**.

3.9.2. БЕЗОПАСНЫЕ СОСТОЯНИЯ ПО ТИПУ ДВЕРИ

Для каждого типа двери определяются следующие безопасные состояния:

- **NC (Normally Closed)**
Безопасное состояние:
 - дверь физически закрыта;
 - замок активирован (дверь заблокирована).
- **NO (Normally Open)**
Безопасное состояние:
 - замок деактивирован;
 - дверь может быть открыта.
- **CARD READER**
Безопасное состояние:
 - дверь физически закрыта;
 - замок активирован;
 - разблокировка возможна только по CARD READER или Alarm.

3.9.3. ИНВАРИАНТЫ БЕЗОПАСНОСТИ ДВЕРИ

В системе всегда должны соблюдаться следующие инварианты:

- дверь не может быть заблокирована, если она физически открыта;
- активация замка допускается только при физически закрытой двери;
- кнопка Alarm всегда имеет приоритет;
- отсутствие сети не должно нарушать безопасность двери;
- логика зависимостей не может нарушать физические ограничения.

Нарушение инварианта недопустимо при любых условиях.

3.9.4. ПОВЕДЕНИЕ ДВЕРИ ПРИ НЕОПРЕДЕЛЁННОСТИ

При неопределённом состоянии системы (сбой, потеря данных, частичная инициализация):

- дверь переводится в безопасное состояние по своему типу;
- логика зависимостей не применяется;
- активность сети игнорируется;
- Alarm остаётся активным.

3.9.5. ПЕРЕХОДЫ МЕЖДУ БЕЗОПАСНЫМИ СОСТОЯНИЯМИ

Переходы между состояниями двери:

- выполняются только через Safety Layer;
- учитывают физическое состояние двери;
- исключают резкие и конфликтующие команды;

3.9.6. БЕЗОПАСНОСТЬ ПРИ ВОССТАНОВЛЕНИИ

При восстановлении после отказа:

- дверь не должна мгновенно менять состояние;
- логика включается после стабилизации входных данных;
- приоритет всегда отдаётся текущему физическому состоянию двери.

3.9.7. ЖУРНАЛИРОВАНИЕ ПЕРЕХОДОВ

Фиксируются события:

- переход системы в безопасный режим;
- восстановление из безопасного режима;
- активация Alarm.

Состояния блокировки как таковые не журналируются, а выводятся из контекста событий.

4. АНАЛИЗ АППАРАТНОЙ ПЛАТФОРМЫ И ИНТЕРФЕЙСОВ

4.1. КАРТА СИГНАЛОВ ДВЕРИ (ЗАМОК / СЕНСОР / LED / ЗУММЕР / ALARM)

Цель данного пункта — **зафиксировать аппаратную модель двери**, набор сигналов и их назначение, чтобы далее:

- однозначно связать аппаратные входы/выходы с логикой;
- корректно описать драйверы;
- исключить неоднозначность при разработке и обслуживании.

4.1.1. АППАРАТНАЯ МОДЕЛЬ ОДНОЙ ДВЕРИ

Каждая дверь в системе рассматривается как **набор строго определённых аппаратных сигналов**, объединённых в логический объект «Дверь».

Минимальный состав аппаратных сигналов двери:

- исполнительный выход управления замком (соленоид);
- вход датчика состояния двери;
- выход красного светодиода;
- выход зелёного светодиода;
- выход управления зуммером;
- вход кнопки аварийного открытия (Alarm).

4.1.2. СИГНАЛ УПРАВЛЕНИЯ ЗАМКОМ (SOLENOID LOCK)

Назначение:

- управление электромеханическим соленоидным замком двери.

Свойства сигнала:

- цифровой выход;
- активное состояние соответствует **блокировке двери**;
- управление осуществляется только через Safety Layer.

Ограничения:

- активация замка запрещена при физически открытой двери;
- при перезапуске системы выход инициализируется в безопасное состояние.

4.1.3. ДАТЧИК СОСТОЯНИЯ ДВЕРИ

Назначение:

- определение физического состояния двери (открыта / закрыта).

Свойства сигнала:

- цифровой вход;
- логическое состояние напрямую отражает положение двери;
- используется как первичный источник истины о физическом состоянии.

Особенности:

- используется для контроля инвариантов безопасности;

- используется для запуска тайм-аутов и логики зависимостей.

4.1.4. КНОПКА АВАРИЙНОГО ОТКРЫТИЯ (ALARM)

Назначение:

- принудительная локальная разблокировка двери.

Свойства сигнала:

- цифровой вход;
- обрабатывается независимо от сетевых интерфейсов;
- имеет наивысший приоритет.

Поведение:

- первое нажатие — перевод двери в режим Alarm;
- повторное нажатие — возврат к логике зависимостей.

4.1.5. СВЕТОВАЯ ИНДИКАЦИЯ ДВЕРИ

Каждая дверь имеет два независимых световых выхода:

- **красный индикатор**
 - отражает состояние блокировки двери;
- **зелёный индикатор**
 - отражает состояние разблокировки двери.

Особенности:

- в режиме сигнализации индикаторы работают по специальному алгоритму;
- управление индикацией не влияет на состояние замка.

4.1.6. ЗУММЕР

Назначение:

- звуковая индикация аварийных и предупредительных состояний.

Свойства сигнала:

- цифровой выход;
- используется только в режиме сигнализации.

Зуммер:

- не используется для штатной индикации;
- не влияет на логику дверей.

4.1.7. СВЯЗ АППАРАТНЫХ СИГНАЛОВ С ЛОГИКОЙ

Все аппаратные сигналы двери:

- обрабатываются через драйверы аппаратного уровня;
- передаются в Logic Core в виде нормализованных состояний;
- управляются только через Safety Layer.

Прямая связь логики с GPIO **запрещена**.

4.1.8. ИНВАРИАНТЫ АППАРАТНОГО УРОВНЯ

На уровне аппаратных сигналов должны соблюдаться:

- невозможность одновременного включения противоречащих состояний;
- корректная инициализация всех выходов при старте;
- предсказуемое состояние при reset.

4.2. ЭЛЕКТРОПИТАНИЕ И РЕЖИМЫ ОТКАЗОВ ПИТАНИЯ

(переработанная редакция, без BOD)

Цель данного пункта — зафиксировать **реальное поведение системы при наличии и отсутствии питания**, исходя из применяемых замков и требований эвакуационной безопасности.

4.2.1. ОБЩИЕ ПРИНЦИПЫ ЭЛЕКТРОПИТАНИЯ

Система управления дверьми проектируется с учётом следующих принципов:

- отказ питания является штатным и допустимым сценарием;
- безопасность людей **обеспечивается аппаратно**, а не программно;
- программная логика не должна быть необходима для эвакуации.

Электропитание рассматривается как **необязательное условие** для обеспечения безопасного состояния дверей.

4.2.2. ТИП ПРИМЕНЯЕМЫХ ЗАМКОВ

Во всех дверях используются **соленоидные замки следующего типа**:

- при **отсутствии напряжения** замок находится в **разблокированном состоянии**;
- блокировка двери возможна **только при подаче напряжения** на соленоид.

Это свойство замка является **базовым элементом безопасности всей системы**.

4.2.3. БЕЗОПАСНОЕ СОСТОЯНИЕ СИСТЕМЫ ПРИ ОТСУТСТВИИ ПИТАНИЯ

При полном отсутствии питания:

- все контроллеры обесточены;
- программное управление отсутствует;

- все замки находятся в **разблокированном состоянии**;
 - двери можно свободно открыть вручную.
- Это состояние считается **основным безопасным состоянием системы**.

4.2.4. ПОВЕДЕНИЕ СИСТЕМЫ ПРИ АВАРИЙНОМ ОТКЛЮЧЕНИИ ПИТАНИЯ

Аварийное отключение питания (в том числе по сигналу пожарной системы) приводит к:

- отключению питания шкафа управления дверьми;
- полной остановке работы контроллеров;
- разблокировке всех дверей за счёт аппаратных свойств замков.

Программная логика в этом сценарии **не участвует и не требуется**.

4.2.5. ПОВЕДЕНИЕ ПРИ НАЛИЧИИ ПИТАНИЯ (ШТАТНАЯ РАБОТА)

При наличии питания:

- система может блокировать двери согласно логике;
- замки блокируются только по явной команде управления;
- Safety Layer контролирует допустимость блокировки;
- Alarm остаётся доступным локально.

4.2.6. ПОВЕДЕНИЕ ПРИ ПРОПАДАНИИ ПИТАНИЯ ВО ВРЕМЯ РАБОТЫ

Если питание пропадает во время штатной работы системы:

- управление дверьми немедленно прекращается;
- замки переходят в разблокированное состояние;
- все логические состояния теряются;
- система фактически переходит в аппаратный Safety-режим.

Никаких программных действий при этом не выполняется.

4.2.7. ПОВЕДЕНИЕ ПРИ ВОССТАНОВЛЕНИИ ПИТАНИЯ

При восстановлении питания:

- система стартует как после перезапуска;
- выполняется инициализация аппаратных ресурсов;
- загружается конфигурация из QSPI Flash;
- двери **не блокируются автоматически** до завершения инициализации;
- логика включается только после стабилизации работы системы.

4.2.8. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- система не гарантирует сохранность оперативных данных при пропадании питания;
- отсутствие питания всегда трактуется как разрешение на свободный выход;
- безопасность не зависит от состояния прошивки, CAN или Ethernet при отсутствии питания.

4.3. CANBUS ТОПОЛОГИЯ РАСШИРЕНИЯ

Цель данного пункта — зафиксировать принципы построения CAN-сети между контроллерами, ограничения топологии и базовые правила подключения, чтобы обеспечить **надёжный обмен данными в режиме MASTER-SLAVE**.

4.3.1. НАЗНАЧЕНИЕ CANBUS В СИСТЕМЕ

CANBUS используется для:

- обмена состояниями дверей между платами;
- передачи логических команд от MASTER к SLAVE;
- контроля доступности плат в системе.

CANBUS является **единственным интерфейсом**, влияющим на распределённую логику дверей.

4.3.2. РОЛИ УЗЛОВ В CAN-СЕТИ

В системе используются следующие роли:

- **MASTER-контроллер (Node ID = 1)**
 - центральный узел логики;
 - принимает состояния дверей от всех SLAVE;
 - вычисляет зависимости;
 - рассылает команды управления.
- **SLAVE-контроллеры (Node ID = 2...10)**
 - управляют локальными дверями;
 - передают их состояния MASTER;
 - исполняют команды MASTER через Safety Layer.

Роль контроллера определяется его Node ID.

4.3.3. ТОПОЛОГИЯ CAN-СЕТИ

Используется классическая топология CANBUS:

- линейная (шина);

- без разветвлений типа «звезда»;
 - с последовательным подключением всех плат.
- Особенности:
- все контроллеры подключены к одной CAN-шине;
 - длина шины и скорость передачи должны соответствовать стандарту CAN.

4.3.4. ТЕРМИНАЦИЯ CANBUS

Для корректной работы CAN-сети:

- на обоих концах шины должны быть установлены терминаторы;
- номинал терминаторов — согласно стандарту CAN;
- промежуточные узлы терминаторами не оснащаются.

Неправильная терминация рассматривается как аппаратная ошибка.

4.3.5. ОГРАНИЧЕНИЯ ТОПОЛОГИИ

В рамках проекта принимаются следующие ограничения:

- максимальное количество плат — **10**;
- максимальная длина CAN-шины определяется скоростью передачи и условиями монтажа;
- использование повторителей и шлюзов не предусматривается.

4.3.6. АДРЕСАЦИЯ В CAN-СЕТИ

Каждому контроллеру присваивается уникальный **Node ID**:

- диапазон: 1...10;
- Node ID = 1 — всегда MASTER;
- Node ID используется:
 - в идентификации сообщений;
 - в маршрутизации команд;
 - в диагностике.

Два контроллера с одинаковым Node ID в системе недопустимы.

4.3.7. ПОВЕДЕНИЕ ПРИ ОШИБКАХ CANBUS

При ошибках CANBUS:

- фиксируются диагностические события;
- применяется логика отказа CAN (см. раздел 3.5);
- система может переходить в режим Degraded;
- безопасность дверей сохраняется локально.

4.3.8. ЭЛЕКТРОМАГНИТНАЯ СОВМЕСТИМОСТЬ

При проектировании и монтаже CANBUS должны учитываться:

- экранирование кабеля;
- корректное заземление;
- разделение силовых и сигнальных линий.

EMC-ошибки CANBUS рассматриваются как вероятный источник деградации связи.

4.3.9. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- CANBUS не используется для конфигурирования системы;
- CANBUS не используется для обновления прошивки;
- отказ CANBUS не должен приводить к опасному состоянию дверей.

4.4. RS-485 (MODBUS RTU, ТОЛЬКО ИНДИКАЦИЯ) (ПЕРЕРАБОТАННАЯ РЕДАКЦИЯ)

4.4.1. НАЗНАЧЕНИЕ RS-485

RS-485 используется как физический транспорт для протокола **MODBUS RTU** с целью передачи состояния системы на внешние устройства (SCADA/панель/ПЛК).

Интерфейс предназначен **только для индикации/мониторинга** и **не используется для управления дверьми или изменения конфигурации**.

4.4.2. ТОПОЛОГИЯ И РОЛИ УСТРОЙСТВ

- На стороне системы дверей **всегда работает только MASTER-плата** (Node ID = 1).
- SLAVE-платы по CAN **не участвуют** в RS-485.
- Внешнее устройство является **MODBUS Master** (клиент), опрашивает наш контроллер.

4.4.3. ПОЛУДУПЛЕКС И УПРАВЛЕНИЕ НАПРАВЛЕНИЕМ (DE/RE)

- Линии **DE** и **/RE** аппаратно объединены.
- Передача выполняется в **полудуплексном режиме**:
 - при передаче: включён драйвер, приёмник отключён;
 - при приёме: драйвер выключен, приёмник включён (в зависимости от реализации управления DE).

Требование к ПО:

- корректное управление направлением передачи RS-485 (через UART RS-485 mode или GPIO/RTS) для соблюдения таймингов MODBUS RTU.

4.4.4. ПОДДЕРЖИВАЕМЫЕ ФУНКЦИИ MODBUS

Интерфейс **read-only**.

Разрешены только функции чтения:

- 0x04 Read Input Registers (рекомендуется для статусов)
- (опционально) 0x03 Read Holding Registers — если понадобится

Запись **запрещена**:

- любые функции записи (0x06, 0x10, и т.п.) должны возвращать MODBUS exception или игнорироваться согласно выбранной политике.

4.4.5. МОДЕЛЬ ДАННЫХ, ВЫДАВАЕМАЯ ПО MODBUS

По MODBUS выдаются только данные мониторинга, например:

- состояние системы (Normal/Degraded);
- состояние связи CAN;
- состояния дверей (открыта/закрыта);
- состояние Alarm (активна/не активна);
- состояние сигнализации (вкл/выкл);
- диагностические флаги (по необходимости).

Важно: события “открылась/закрылась” логируются локально (см. 2.9), а по MODBUS выдаётся **текущий снимок состояния**.

Карта регистров (Input Registers, 3xxxx)

Нумерация ниже в стиле “человеческих” регистров 30001...

В реализации Modbus обычно адрес будет 0-based, но это уже вопрос библиотеки.

Системные регистры (30001...30020)

30001 — SystemState

- 0 = Normal
- 1 = Degraded (CAN деградация)
- 2 = Safe (нет валидной конфигурации / стартовый безопасный режим)

30002 — CANState

- 0 = CAN not used (Standalone)
- 1 = CAN OK
- 2 = CAN degraded (потеря части узлов)
- 3 = CAN down (нет связи)

30003 — OnlineSlavesMask (битовая маска SLAVE 2..10)

- bit0 = Node2 online
- bit1 = Node3 online
- ...
- bit8 = Node10 online

30004 — TotalDoorsConfigured (2...80)

30005 — ConfigVersion (версия конфигурации, число)

30006 — ConfigFormatVersion (версия формата конфигурации, число)

30007 — FirmwareVersionMajor

30008 — FirmwareVersionMinor

30009–30010 — UptimeSeconds (uint32)

30011 — LastResetCause

- 0 = PowerOn
- 1 = Manual reset
- 2 = Watchdog
- 3 = Software reset
- 4 = Other/Unknown

30012 — OpenTimeoutGlobalSeconds

- общий тайм-аут “дверь слишком долго открыта” (сек)

30013–30020 — reserve

Таблица дверей (начиная с 30100)

Каждая дверь занимает **3 регистра** (так проще всего для SCADA, и хватает на все нужные флаги/таймеры).

Двери идут по $*globalDoorId = (nodeId-1)8 + localDoor$, где:

- nodeId = 1..10
- localDoor = 1..8
- globalDoorId = 1..80

Базовый адрес для двери:

$base = 30100 + (globalDoorId-1)*3$

То есть:

- Door #1 → 30100..30102
- Door #2 → 30103..30105
- ...
- Door #80 → 30337..30339

Регистры двери

base+0 (StatusBits) — 16 бит флагов

Предлагаю такой битовый формат:

- **bit0** — **Present** (1 = дверь присутствует/сконфигурирована)
- **bit1** — **PhysClosed** (1 = дверь физически закрыта, 0 = открыта)
- **bit2** — **LockCommanded** (1 = логика требует блокировку, 0 = требует разблокировку)
- **bit3** — **LockOutputActive** (1 = выход на соленоид активен)
(это фактическое управление выходом, а не “заблокирована/нет”; учитываем инвариант: при открытой двери выход должен быть 0)
- **bit4** — **AlarmActive** (1 = режим Alarm активен)
- **bit5** — **SignalingActive** (1 = сигнализация мигает + buzzer)
- **bit6** — **DegradedDoor** (1 = дверь сейчас в Degraded режиме)
- **bit7** — **CardReaderMode** (1 = дверь типа CARD READER)
- **bit8** — **OpenTimeoutExceeded** (1 = превышен тайм-аут открытой двери)
- **bit9** — **CloseDelayActive** (1 = действует тайм-аут после закрытия)

• **bit10..15** — резерв

base+1 (OpenSeconds)

- Сколько секунд дверь находится в состоянии **открыта** (если закрыта — 0)
(очень удобно для диспетчеризации)

base+2 (CloseDelayRemainingSeconds)

- Остаток задержки “после закрытия двери” (сек)
(если не активна — 0)

4.4.6. АДРЕСАЦИЯ MODBUS

- MODBUS Slave Address контроллера задаётся в конфигурации (или фиксируется по умолчанию).
- В системе предполагается один RS-485 узел (наш MASTER), поэтому адресация нужна в первую очередь для совместимости со стандартом и SCADA.

4.4.7. ТАЙМИНГИ И УСТОЙЧИВОСТЬ

- Реализация обязана соблюдать требования MODBUS RTU по межкадровым паузам и формированию кадра.
- RS-485 обмен не должен влиять на контур управления дверьми:
 - низкий приоритет задачи;
 - ограничение частоты/времени обработки запросов.

4.4.8. ПОВЕДЕНИЕ ПРИ ОТКАЗЕ RS-485/MODBUS

При отказе RS-485 или ошибках протокола:

- управление дверьми продолжается без изменений;
- система не переходит в Degraded из-за RS-485;
- фиксируется диагностическое событие (опционально).

4.4.9. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- RS-485/MODBUS не используется для управления дверьми.
- RS-485/MODBUS не используется для загрузки конфигурации.
- Единственный источник данных по RS-485 — MASTER-плата.

4.5. ETHERNET (RMII PHY) И СЕТЕВЫЕ РЕЖИМЫ

Цель данного пункта — зафиксировать назначение Ethernet-интерфейса, его ограничения и правила использования в системе управления дверьми.

4.5.1. НАЗНАЧЕНИЕ ETHERNET

Ethernet используется **исключительно** для:

- подключения WEB-конфигуратора;
- загрузки и обновления конфигурации;
- сервисного мониторинга состояния системы;
- диагностики и обслуживания.

Ethernet **не участвует** в контуре реального времени управления дверьми.

4.5.2. РОЛЬ ETHERNET В АРХИТЕКТУРЕ СИСТЕМЫ

Ethernet рассматривается как:

- сервисный интерфейс;
- интерфейс обслуживания;
- интерфейс конфигурирования.

Отсутствие Ethernet **не влияет** на:

- логику блокировки дверей;
- безопасность дверей;
- работу CAN и локальных контроллеров.

4.5.3. WEB-КЛИЕНТ И HTTP-СЕРВЕР

На MASTER-контроллере работает встроенный HTTP-сервер, обеспечивающий:

- приём соединений от WEB-клиента;
- аутентификацию пользователя;
- управление WEB-сеансом;
- передачу конфигурации;
- отображение текущего состояния системы.

SLAVE-контроллеры **не поднимают HTTP-сервер**.

4.5.4. WEB-СЕАНС УПРАВЛЕНИЯ

WEB-сеанс:

- инициируется пользователем вручную;
- требует аутентификации;
- имеет состояния «подключено / отключено»;
- влияет только на режим обслуживания системы.

Во время активного WEB-сеанса:

- разрешено конфигурирование;
- разрешён мониторинг;
- запрещено автоматическое завершение сеанса по тайм-ауту.

4.5.5. ПОВЕДЕНИЕ СИСТЕМЫ ПРИ ОТСУТСТВИИ ETHERNET

При отсутствии Ethernet:

- система работает автономно;
- логика дверей выполняется без ограничений;
- CAN и RS-485 продолжают работу;
- никаких аварийных режимов не активируется.

Отсутствие Ethernet считается **нормальным состоянием**.

4.5.6. ПОВЕДЕНИЕ ПРИ ОБРЫВЕ ETHERNET ВО ВРЕМЯ WEB-СЕАНСА

Если Ethernet пропадает во время активного WEB-сеанса:

- WEB-сеанс считается аварийно завершённым;
- конфигурация не изменяется;
- система продолжает работу по последней валидной конфигурации;
- пользователь должен повторно подключиться.

4.5.7. БЕЗОПАСНОСТЬ СЕТЕВОГО ДОСТУПА

Для Ethernet должны соблюдаться следующие требования:

- доступ только после аутентификации;
- разграничение ролей (при необходимости);
- отсутствие управляющих интерфейсов без WEB-сеанса.

4.5.8. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- Ethernet не используется для передачи управляющих команд дверям;
- Ethernet не используется для RS-485 или CAN туннелирования;
- Ethernet не должен блокировать или задерживать работу Logic Core.

4.6. QSPI FLASH (ПАМЯТЬ КОНФИГУРАЦИИ И ЛОГОВ)

Цель пункта — зафиксировать роль QSPI Flash в системе, состав хранимых данных и ключевые требования к надёжности хранения (конфигурация и журнал событий).

4.6.1. НАЗНАЧЕНИЕ QSPI FLASH

QSPI Flash используется для хранения **энергонезависимых данных**:

- активная конфигурация проекта;
- резервная (последняя валидная) конфигурация;
- журнал событий (Event Log);
- служебные метаданные (версии, CRC, время загрузки).

QSPI Flash **не используется** для работы логики в реальном времени.

4.6.2. ДАННЫЕ КОНФИГУРАЦИИ

В QSPI Flash должны храниться:

- список дверей и их параметры (techId, Node/localDoor, тип NC/NO/CARD READER, тайм-ауты);
- таблица зависимостей дверей;
- сетевые параметры (CAN nodeId, Ethernet параметры, RS-485 Modbus адрес);
- версии:
 - ConfigVersion;
 - ConfigFormatVersion;
- контроль целостности (CRC).

4.6.3. ДАННЫЕ ЖУРНАЛА СОБЫТИЙ (ЛОГОВ)

В QSPI Flash хранится журнал событий, который фиксирует:

- физическое открытие/закрытие дверей;
- Alarm (включение/выключение);
- тайм-аут «дверь слишком долго открыта»;
- переходы Normal/Degraded;
- события связи CAN (потеря/восстановление);
- успешная/неуспешная загрузка конфигурации;
- перезапуски системы (reset cause).

Журнал хранит **события**, а не команды блокировки.

4.6.4. НАДЁЖНОСТЬ ХРАНЕНИЯ И ЦЕЛОСТНОСТЬ

Для обеспечения надёжности хранения:

- все критичные блоки данных имеют CRC;
- конфигурация хранится в двух копиях:
 - активная;
 - резервная (последняя валидная);
- при старте выполняется проверка CRC;
- при ошибке активной конфигурации выполняется откат на резервную.

4.6.5. АТОМАРНОЕ ОБНОВЛЕНИЕ КОНФИГУРАЦИИ

Запись новой конфигурации выполняется транзакционно:

- новая конфигурация записывается во временную область;
- проверяется целостность и корректность;
- только после успешной проверки:
 - конфигурация помечается активной;
 - обновляются метаданные и версии.

Частично записанная конфигурация не должна становиться активной.

4.6.6. ПОВЕДЕНИЕ ПРИ ОШИБКАХ QSPI FLASH

При ошибках памяти:

- управление дверьми продолжается по последней валидной конфигурации;
- если валидной конфигурации нет — система работает без логики зависимостей, в безопасном режиме;
- ошибки фиксируются в диагностике (если это возможно).

Отказ записи журнала событий не влияет на работу дверей.

4.6.7. ДОСТУП К ИНФОРМАЦИИ О КОНФИГУРАЦИИ ЧЕРЕЗ WEB

Через WEB-интерфейс должны быть доступны (read-only):

- версия активной конфигурации (ConfigVersion);
- версия формата (ConfigFormatVersion);
- дата/время последней загрузки;
- краткая сводка по проекту (кол-во дверей, режим Standalone/Master-Slave).

WEB не является источником истины — истина хранится в QSPI Flash.

4.6.8. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- QSPI Flash не гарантирует сохранность последних записей журнала при внезапном отключении питания;
- ресурс записи должен учитываться (журнал — кольцевой буфер);
- любая запись в QSPI Flash должна быть отделена от задач реального времени по приоритету.

5. ОБМЕН МЕЖДУ ПЛАТАМИ ПО CANBUS

5.1. АДРЕСАЦИЯ ПЛАТ И РОЛЬ MASTER

Цель пункта — зафиксировать правила адресации узлов CAN, роли MASTER/SLAVE и как эти идентификаторы используются в обмене и диагностике.

5.1.1. NODE ID (ИДЕНТИФИКАТОР КОНТРОЛЛЕРА)

Каждая плата в проекте имеет уникальный **Node ID**:

- диапазон: **1...10**
- **Node ID = 1** — всегда **MASTER**
- **Node ID = 2...10** — **SLAVE**

Node ID задаётся при конфигурировании проекта и используется в обмене по CAN и в расчёте глобальных идентификаторов дверей.

5.1.2. Роль MASTER

MASTER (Node ID = 1) выполняет функции центрального узла:

- хранит полную конфигурацию проекта;
- принимает состояния дверей от всех SLAVE;
- вычисляет логику зависимостей дверей;
- формирует команды управления для SLAVE;
- контролирует доступность SLAVE и состояние CAN-сети.

MASTER является единственным узлом, который:

- взаимодействует с WEB;
- отдаёт данные по RS-485 (MODBUS RTU).

5.1.3. Роль SLAVE

SLAVE (Node ID = 2...10):

- управляет своими локальными дверьми (1...8);
- считывает физические состояния;
- применяет команды MASTER через Safety Layer;
- передаёт состояния дверей MASTER;
- при потере связи по CAN переходит в Degraded (см. 3.5).

SLAVE не хранит глобальную конфигурацию как источник истины.

5.1.4. УНИКАЛЬНОСТЬ И КОНФЛИКТ NODE ID

В системе недопустимо наличие двух плат с одинаковым Node ID.

При обнаружении конфликта Node ID (например, диагностикой на MASTER):

- конфликт фиксируется как критическая ошибка сети;
- сеть считается деградированной;
- логика зависимостей для затронутых SLAVE не применяется.

5.1.5. ИСПОЛЬЗОВАНИЕ NODE ID В ИДЕНТИФИКАЦИИ ДВЕРЕЙ

Node ID является частью идентификатора двери:

- localDoor: 1...8
- globalDoorId = (nodeId - 1) * 8 + localDoor (1...80)

Это обеспечивает однозначную связь:

- дверь ↔ плата ↔ номер разъёма/канала на плате.

5.1.6. АДРЕСАЦИЯ CAN-СООБЩЕНИЙ

Node ID используется в CAN-протоколе для:

- маркировки источника статуса (SLAVE → MASTER);
- адресации команд (MASTER → конкретный SLAVE);
- диагностики доступности узлов.

5.2. Синхронизация состояний дверей

Цель пункта — описать, **как обеспечивается согласованность состояний дверей** между MASTER и SLAVE-контроллерами по CANBUS, и какие данные считаются источником истины.

5.2.1. Принцип источника истины

В системе действует разделение ответственности:

- **SLAVE** является источником истины для:
 - физических состояний своих дверей (открыта / закрыта);
 - локальных событий (Alarm, тайм-ауты);
 - фактического состояния выходов (соленоид, индикация).
- **MASTER** является источником истины для:
 - логики зависимостей дверей;
 - расчёта команд блокировки;
 - глобального состояния системы.

Физические данные **всегда поднимаются вверх** (SLAVE → MASTER), логические команды **всегда спускаются вниз** (MASTER → SLAVE).

5.2.2. НАБОР синхронизируемых данных

От SLAVE к MASTER передаются:

- состояние каждой двери:
 - физически открыта / закрыта;
 - Alarm активен / не активен;
 - сигнализация активна / не активна;
- диагностические флаги SLAVE;
- признак доступности SLAVE.

От MASTER к SLAVE передаются:

- команды блокировки / разблокировки дверей;
- команды перехода в Degraded;
- служебные команды синхронизации.

5.2.3. ПЕРИОДИЧНОСТЬ СИНХРОНИЗАЦИИ

Синхронизация выполняется:

- периодически (циклический обмен);
- дополнительно по событию (изменение состояния двери).

Период обмена выбирается таким образом, чтобы:

- обеспечить актуальность логики;
- не перегружать CAN-шину;
- не влиять на детерминированность управления дверьми.

5.2.4. НАЧАЛЬНАЯ СИНХРОНИЗАЦИЯ

При старте системы или восстановлении CAN-связи:

1. SLAVE передаёт MASTER текущие состояния всех своих дверей;
2. MASTER формирует полную картину системы;
3. MASTER рассчитывает актуальные команды;
4. команды применяются к SLAVE **плавно**, без резких переключений.

До завершения начальной синхронизации логика зависимостей может быть временно отключена.

5.2.5. КОНФЛИКТ СОСТОЯНИЙ

Возможные конфликты:

- логика MASTER требует блокировки;
- физическое состояние двери не позволяет выполнить блокировку (дверь открыта).

В таких случаях:

- приоритет имеет физическое состояние двери;
- блокировка откладывается;
- конфликт фиксируется диагностически.

5.2.6. ПОТЕРЯ СИНХРОНИЗАЦИИ

Если MASTER не получает данные от SLAVE:

- SLAVE считается недоступным;
- система переходит в режим Degraded для затронутых дверей;
- логика зависимостей для них не применяется;
- локальная безопасность SLAVE сохраняется.

5.2.7. ВОССТАНОВЛЕНИЕ СИНХРОНИЗАЦИИ

После восстановления связи:

- выполняется повторная полная синхронизация;
- MASTER обновляет глобальное состояние;
- логика зависимостей включается мягко;
- недопустимые переходы исключены Safety Layer.

5.2.8. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- синхронизация не должна блокировать управление дверьми;
- CANBUS используется только для обмена состояниями и командами;
- любые ошибки синхронизации не должны приводить к опасным состояниям дверей.

5.3. ФОРМАТ CAN-СООБЩЕНИЙ И КАДРЫ ОБМЕНА

Цель пункта — зафиксировать **структуру CAN-кадров**, типы сообщений и правила обмена между MASTER и SLAVE, чтобы обеспечить однозначную интерпретацию данных и расширяемость протокола.

5.3.1. ОБЩИЕ ПРИНЦИПЫ CAN-ПРОТОКОЛА

- Используется стандартный CAN 2.0 (11-битный идентификатор).
- Обмен детерминированный, без широковещательного управления логикой.
- Все сообщения имеют чёткое назначение и фиксированный формат.
- Протокол ориентирован на:
 - компактность;
 - предсказуемость;

- минимальную нагрузку на шину.

5.3.2. КЛАССЫ CAN-СООБЩЕНИЙ

В системе используются следующие классы сообщений:

1. **Status Frames (SLAVE → MASTER)**
Передача текущих состояний дверей.
2. **Command Frames (MASTER → SLAVE)**
Передача команд управления дверьми.
3. **Service Frames (двусторонние)**
Диагностика, синхронизация, heartbeat.

5.3.3. ФОРМИРОВАНИЕ CAN-ИДЕНТИФИКАТОРА

CAN-ID формируется логически и содержит:

- тип сообщения;
- Node ID источника;
- (опционально) номер двери или подтип сообщения.

Рекомендуемая логическая структура CAN-ID:

[10..8] MessageType

[7..4] NodeID

[3..0] SubIndex

Где:

- MessageType — класс сообщения;
- NodeID — идентификатор платы-источника;
- SubIndex — номер двери или служебный индекс.

5.3.4. КАДРЫ СОСТОЯНИЯ ДВЕРЕЙ (STATUS FRAME)

Назначение:

Передача состояния дверей от SLAVE к MASTER.

Периодичность:

- циклически;
- по событию (изменение состояния двери).

Payload (пример):

Byte Содержание

- | | |
|---|---|
| 0 | Маска дверей (бит = дверь присутствует) |
| 1 | Маска PhysClosed |
| 2 | Маска AlarmActive |
| 3 | Маска SignalingActive |
| 4 | Маска LockOutputActive |
| 5 | Резерв |
| 6 | Резерв |
| 7 | CRC / счетчик кадров |

Один кадр описывает до **8 дверей SLAVE**.

5.3.5. КАДРЫ КОМАНД (COMMAND FRAME)

Назначение:

Передача команд управления дверьми от MASTER к SLAVE.

Payload (пример):

Byte Содержание

- | | |
|---|---|
| 0 | Маска дверей, к которым применяется команда |
| 1 | Маска LockCommand (1 = заблокировать) |
| 2 | Маска UnlockCommand (1 = разблокировать) |
| 3 | Маска Degraded |
| 4 | Резерв |
| 5 | Резерв |
| 6 | Sequence ID |
| 7 | CRC |

Команды:

- не являются абсолютными;
- проверяются Safety Layer на SLAVE перед применением.

5.3.6. СЛУЖЕБНЫЕ КАДРЫ (SERVICE FRAMES)

Используются для:

- heartbeat SLAVE → MASTER;
- подтверждения доступности узлов;
- начальной синхронизации;
- диагностики ошибок.

Пример payload:

Byte Содержание

- 0 ServiceCode
- 1 ServiceData
- 2–6 Резерв
- 7 CRC

5.3.7. КОНТРОЛЬ ЦЕЛОСТНОСТИ И ДИАГНОСТИКА

- CRC или счётчик кадров используется для:
 - обнаружения потери кадров;
 - диагностики рассинхронизации.
- Ошибочные кадры:
 - игнорируются;
 - могут инициировать повторную синхронизацию.

5.3.8. ПРИОРИТЕТЫ CAN-СООБЩЕНИЙ

Рекомендуемый приоритет:

1. Status Frames
2. Command Frames
3. Service Frames

При перегрузке шины:

- служебные кадры могут пропускаться;
- статус и команды имеют приоритет.

5.3.9. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- CAN-обмен не должен блокировать локальную обработку дверей;
- потеря отдельных кадров не должна приводить к опасным состояниям;
- протокол должен допускать расширение без изменения базовой логики.

5.4. РАСПРЕДЕЛЕНИЕ ОТВЕТСТВЕННОСТИ (MASTER / SLAVE)

Цель пункта — чётко зафиксировать **границы ответственности** между MASTER- и SLAVE-контроллерами, чтобы исключить:

- дублирование функций;
- противоречивые решения;
- небезопасное поведение при сбоях связи.

5.4.1. ОБЩИЙ ПРИНЦИП РАСПРЕДЕЛЕНИЯ ОТВЕТСТВЕННОСТИ

В системе действует строгий принцип разделения ролей:

- **MASTER отвечает за логику**
- **SLAVE отвечает за физику и безопасность**

Ни один контроллер не выходит за пределы своей зоны ответственности.

5.4.2. ОТВЕТСТВЕННОСТЬ MASTER-КОНТРОЛЛЕРА

MASTER-контроллер имеет 8 локальных дверей (в режиме работы блоков MASTER-SLAVE у MASTER все 8 дверей подключены, иначе при меньшем количестве дверей на MASTER это режим STADALONE), которые обрабатываются по тем же правилам безопасности и логики, что и двери SLAVE-контроллеров.

MASTER-контроллер отвечает за:

- хранение полной конфигурации проекта;
- хранение таблицы зависимостей дверей;
- вычисление логики блокировки дверей;
- принятие решений о том, какие двери должны быть заблокированы или разблокированы;
- формирование логических команд для SLAVE;
- контроль доступности SLAVE-контроллеров;
- определение режима системы (Normal / Degraded);
- взаимодействие с WEB-интерфейсом;
- предоставление данных по RS-485 (MODBUS RTU).

MASTER не управляет напрямую:

- физическими выходами замков;
- локальной индикацией SLAVE;
- обработкой кнопки Alarm.

5.4.3. ОТВЕТСТВЕННОСТЬ SLAVE-КОНТРОЛЛЕРА

SLAVE-контроллер отвечает за:

- считывание физических датчиков дверей;
- управление соленоидными замками;

- управление световой и звуковой индикацией;
- обработку кнопки Alarm;
- реализацию тайм-аутов двери;
- обеспечение инвариантов физической безопасности;
- выполнение команд MASTER только после проверки допустимости.

SLAVE не принимает решений:

- о глобальной логике блокировки;
- о зависимостях между дверями;
- о конфигурации проекта.

5.4.4. Источники истины

В системе чётко определены источники истины:

- **Физическое состояние двери** — SLAVE
- **Состояние замка и выходов** — SLAVE
- **Логические зависимости дверей** — MASTER
- **Глобальное состояние системы** — MASTER

MASTER не переопределяет физическую реальность, полученную от SLAVE.

5.4.5. ОБРАБОТКА ПРОТИВОРЕЧИЙ

Если команда MASTER противоречит физическому состоянию двери:

- SLAVE обязан **не выполнить** команду;
- приоритет имеет физическое состояние;
- состояние передаётся обратно MASTER;
- конфликт разрешается на уровне логики MASTER.

Противоречие **не является ошибкой**, а является штатной ситуацией.

5.4.6. ПОВЕДЕНИЕ ПРИ ПОТЕРЕ СВЯЗИ

При потере связи между MASTER и SLAVE:

- SLAVE переходит в локальный режим Degraded;
- SLAVE прекращает исполнение новых команд MASTER;
- SLAVE обеспечивает безопасную локальную работу дверей;
- MASTER исключает двери недоступного SLAVE из логики зависимостей.

5.4.7. ВОССТАНОВЛЕНИЕ ВЗАИМОДЕЙСТВИЯ

После восстановления связи:

- SLAVE передаёт MASTER актуальное физическое состояние дверей;
- MASTER пересчитывает логику зависимостей;
- команды применяются постепенно, без резких переходов;
- безопасность всегда имеет приоритет над логикой.

5.4.8. ЗАПРЕЩЁННЫЕ ДЕЙСТВИЯ

Запрещается:

- SLAVE самостоятельно изменять логику зависимостей;
- MASTER принудительно игнорировать физическое состояние двери;
- выполнение команд, нарушающих инварианты безопасности;
- перекладывание ответственности безопасности на сеть.

5.4.9. КЛЮЧЕВОЙ ПРИНЦИП

MASTER думает, SLAVE действует, безопасность — всегда локальная.

5.5. ОБРАБОТКА КОНФЛИКТОВ КОМАНД

Цель пункта — описать, как система разрешает ситуации, когда одновременно действуют разные факторы: физика двери, Alarm, тайм-ауты, команды логики и состояния сети.

5.5.1. ЧТО СЧИТАЕТСЯ КОНФЛИКТОМ

Конфликт — это ситуация, когда:

- логика требует **заблокировать** дверь, но дверь **физически открыта**;
- логика требует **заблокировать**, но **Alarm активен**;
- логика требует **разблокировать**, но другая дверь через зависимости требует блокировки (пересечение правил);
- команды MASTER и локальные условия SLAVE приводят к разным требуемым состояниям;
- система находится в Degraded, но логика пытается применить зависимости.

5.5.2. ОБЩИЙ ПРИНЦИП РАЗРЕШЕНИЯ КОНФЛИКТОВ

Главный принцип:

Физическая безопасность и локальные приоритеты всегда выше логики зависимостей.

Система обязана сохранять предсказуемость: конфликт не должен приводить к “дёрганью” выходов и хаотичным переключениям.

5.5.3. ПРИОРИТЕТЫ (ЖЁСТКИЙ ПОРЯДОК)

Приоритет принятия решения для одной двери:

1. **Инварианты физической безопасности**
(запрет блокировки при открытой двери)
2. **Alarm (локальный приоритет)**
3. **Режим Degraded**
4. **Тайм-ауты и сигнализация**
(как режим индикации/предупреждения, без нарушения инвариантов)
5. **Логика зависимостей (команды MASTER / Logic Core)**

5.5.4. КОНФЛИКТ “НУЖНО ЗАБЛОКИРОВАТЬ, НО ДВЕРЬ ОТКРЫТА”

Если дверь открыта, а логика требует блокировки:

- блокировка **не выполняется** (выход соленоида не активируется);
- требование блокировки сохраняется как “ожидающее”;
- как только дверь закроется — блокировка может быть применена (с учётом тайм-аута после закрытия, если он задан).

5.5.5. КОНФЛИКТ С ALARM

Если активирован Alarm:

- дверь принудительно разблокирована;
- любые команды блокировки игнорируются до выхода из Alarm;
- после повторного нажатия Alarm дверь возвращается в состояние, требуемое логикой в текущий момент.

5.5.6. КОНФЛИКТ В ПЕРЕСЕЧЕНИИ ЗАВИСИМОСТЕЙ

Если одна дверь участвует в нескольких зависимостях и приходит несколько требований:

- для блокировки используется правило:
“если хотя бы одно активное правило требует блокировки — дверь должна быть заблокирована”
- для разблокировки используется правило:
“дверь может быть разблокирована только если ни одно активное правило не требует блокировки”

Таким образом блокировка имеет приоритет над разблокировкой (кроме Alarm).

5.5.7. КОНФЛИКТ В DEGRADED

В режиме Degraded:

- логика зависимостей не применяется;
- команды, основанные на зависимостях, не исполняются;
- дверь работает автономно в безопасной модели (с учётом типа двери);
- Alarm всегда доступен.

5.5.8. КОНФЛИКТ “КОМАНДА УСТАРЕЛА / НЕСОГЛАСОВАНА”

Если SLAVE получает команду, которая:

- относится к устаревшему состоянию сети;
 - противоречит текущим физическим данным;
- то:
- команда не применяется;
 - SLAVE продолжает работу по текущим условиям;
 - актуальное состояние будет отправлено MASTER при следующей синхронизации.

5.5.9. ДИАГНОСТИКА КОНФЛИКТОВ

Конфликты фиксируются как диагностические события (по необходимости), например:

- “Lock requested while door open”
- “Lock suppressed due to Alarm”
- “Multiple lock dependencies active”

Это нужно для обслуживания и анализа проекта.

5.6. ТАЙМ-АУТЫ, HEARTBEAT И ДИАГНОСТИКА ЛИНИИ CAN

Цель пункта — зафиксировать механизмы обнаружения потери связи, деградации CAN-линии и восстановления взаимодействия между MASTER и SLAVE.

5.6.1. HEARTBEAT SLAVE-КОНТРОЛЛЕРОВ

Каждый SLAVE-контроллер обязан периодически отправлять **heartbeat-сообщение** MASTER-контроллеру.

Heartbeat используется для:

- подтверждения доступности SLAVE;
- контроля состояния CAN-линии;
- обнаружения зависаний и потери связи.

5.6.2. ПЕРИОД HEARTBEAT

Период heartbeat:

- фиксированный;
- одинаковый для всех SLAVE;

- задаётся в прошивке (или конфигурации).

Выбор периода должен обеспечивать:

- быстрое обнаружение потери связи;
- отсутствие перегрузки CAN-шины.

5.6.3. ТАЙМ-АУТ ОТСУТСТВИЯ HEARTBEAT

Если от SLAVE не поступает heartbeat в течение заданного тайм-аута:

- SLAVE считается недоступным;
- MASTER исключает его двери из логики зависимостей;
- затронутые двери считаются перешедшими в Degraded;
- событие фиксируется в диагностике.

5.6.4. ПОВЕДЕНИЕ SLAVE ПРИ ПОТЕРЕ СВЯЗИ С MASTER

Если SLAVE не получает команд или служебных сообщений от MASTER:

- SLAVE переходит в локальный режим Degraded;
- прекращает исполнение логики зависимостей;
- продолжает локальное безопасное управление дверьми;
- периодически пытается восстановить связь.

5.6.5. ДИАГНОСТИКА ОШИБОК CAN-ЛИНИИ

Диагностируются следующие ситуации:

- потеря heartbeat;
- повторяющиеся ошибки передачи;
- CAN error-passive / bus-off состояния;
- конфликты Node ID.

Диагностика используется для:

- логирования;
- отображения в WEB;
- передачи статуса по RS-485 (опционально).

5.6.6. ВОССТАНОВЛЕНИЕ СВЯЗИ

При восстановлении CAN-связи:

- SLAVE автоматически возвращается в режим Normal;
- передаёт MASTER актуальное состояние дверей;
- MASTER пересчитывает логику зависимостей;
- команды применяются без резких переходов.

5.6.7. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- кратковременные потери отдельных кадров не считаются потерей связи;
- диагностика CAN не должна блокировать управление дверьми;
- потеря CAN не приводит к опасному состоянию дверей.

6. RS-485 (MODBUS RTU, ТОЛЬКО ИНДИКАЦИЯ)

Раздел описывает интерфейс RS-485, используемый для передачи информации о состоянии системы внешним устройствам.

RS-485 **не участвует в управлении дверями, не влияет на логику блокировки и не может изменять конфигурацию системы.**

6.1. МОДЕЛЬ ДАННЫХ ДЛЯ ВНЕШНЕГО КЛИЕНТА

Внешнему клиенту по RS-485 предоставляется **текущее состояние системы** в виде набора параметров.

Модель данных включает:

6.1.1. СИСТЕМНЫЕ ДАННЫЕ

- текущее состояние системы (Normal / Degraded);
- состояние CAN-сети (OK / Degraded / Down);
- количество активных SLAVE-контроллеров;
- версия прошивки MASTER;
- версия активной конфигурации;
- признак активного WEB-сеанса.

6.1.2. ДАННЫЕ ПО ДВЕРЯМ

Для каждой двери в проекте доступны:

- globalDoorId — глобальный идентификатор двери (1...80);
- nodeId — идентификатор платы (1...10);
- localDoor — номер двери на плате (1...8);
- физическое состояние двери (открыта / закрыта);
- состояние замка (заблокирована / разблокирована);
- логическое состояние двери (Normal / Degraded / Alarm);
- состояние сигнализации;

- превышение тайм-аута открытой двери.
- По RS-485 передаётся **снимок текущего состояния**, а не история событий.

6.2. ПРОТОКОЛ ВЫДАЧИ СТАТУСОВ

Для обмена по RS-485 используется протокол:

MODBUS RTU поверх RS-485

Принципы работы:

- MASTER-контроллер является **MODBUS Slave**;
- внешнее устройство является **MODBUS Master**;
- обмен выполняется **по запросу внешнего клиента**;
- поддерживаются только функции чтения регистров.

Данные выдаются:

- в фиксированной карте регистров;
- в документированном формате;
- без влияния на работу логики системы.

6.3. ОГРАНИЧЕНИЯ БЕЗОПАСНОСТИ (READ-ONLY)

Интерфейс RS-485 имеет строгие ограничения:

Запрещено:

- управление дверями;
- изменение конфигурации;
- управление Alarm;
- воздействие на логику блокировки;
- изменение сетевых параметров.

Любые попытки записи по MODBUS:

- игнорируются;
- либо возвращают ошибку протокола.

RS-485 **не может**:

- перевести систему в опасное состояние;
- нарушить инварианты безопасности;
- повлиять на работу MASTER или SLAVE.

6.4. СОВМЕСТИМОСТЬ ФОРМАТОВ (ASCII / MODBUS RTU)

В системе принят следующий подход:

- основным и рекомендуемым форматом является **MODBUS RTU**;
- использование ASCII-форматов не предусматривается по умолчанию;
- возможная поддержка альтернативных форматов рассматривается только как опция для отдельных проектов.

Совместимость обеспечивается:

- стабильной картой регистров;
- версионированием формата данных;
- обратной совместимостью в рамках одной версии формата.

6.5. ИТОГОВЫЕ ПРИНЦИПЫ РАЗДЕЛА 6

- RS-485 — **только индикация**;
- MODBUS RTU — **единственный основной протокол**;
- управление системой через RS-485 невозможно;
- отказ RS-485 не влияет на безопасность и работу дверей.

7. ETHERNET И ВСТРОЕННЫЙ HTTP-СЕРВЕР

7.1. НАЗНАЧЕНИЕ ETHERNET-ИНТЕРФЕЙСА

Ethernet-интерфейс в системе управления дверьми предназначен **исключительно для сервисных и эксплуатационных задач** и не является частью контура реального времени управления дверьми.

Основные назначения Ethernet:

- подключение WEB-интерфейса для конфигурирования системы;
- загрузка и обновление конфигурации проекта;
- мониторинг текущего состояния системы;
- сервисная диагностика и обслуживание оборудования.

Ethernet используется **только на MASTER-контроллере**.

SLAVE-контроллеры не имеют прямого сетевого доступа и не поднимают сетевые сервисы.

7.1.1. ETHERNET КАК СЕРВИСНЫЙ ИНТЕРФЕЙС

Ethernet рассматривается как:

- интерфейс обслуживания;
- интерфейс ввода и просмотра данных;
- вспомогательный канал связи.

Ethernet **не используется** для:

- передачи команд управления дверьми;
- прямого управления замками;
- передачи логических зависимостей;
- дублирования CAN-обмена.

7.1.2. ОТДЕЛЕНИЕ ETHERNET ОТ ЛОГИКИ УПРАВЛЕНИЯ

В архитектуре системы:

- логика блокировки дверей работает независимо от Ethernet;
- потеря Ethernet не приводит к переходу системы в аварийный режим;
- наличие Ethernet не ускоряет и не замедляет работу логики.

Даже при полном отсутствии Ethernet:

- система управления дверьми продолжает штатную работу;
- безопасность дверей сохраняется.

7.1.3. ИСПОЛЬЗОВАНИЕ ETHERNET НА ЭТАПЕ ЭКСПЛУАТАЦИИ

Типовой сценарий использования Ethernet:

1. Подключение WEB-клиента к MASTER-контроллеру.
2. Конфигурирование системы или просмотр состояния.
3. Завершение WEB-сеанса.
4. Дальнейшая автономная работа системы без Ethernet-подключения.

Постоянное подключение WEB-клиента **не является обязательным** для работы системы.

7.1.4. ОГРАНИЧЕНИЯ ETHERNET-ИНТЕРФЕЙСА

Ethernet-интерфейс:

- не участвует в контуре реального времени;
- не влияет на безопасность дверей;
- не является обязательным для работы системы;
- не используется как резервный канал управления.

7.1.5. КЛЮЧЕВОЙ ПРИНЦИП

Ethernet — это инструмент конфигурирования и наблюдения, а не управления и не безопасности.

7.2. РОЛЬ ETHERNET В АРХИТЕКТУРЕ СИСТЕМЫ

Данный подпункт определяет **место Ethernet в общей архитектуре** и его взаимодействие с остальными подсистемами контроллера.

7.2.1. ETHERNET КАК ВНЕШНЯЯ ТОЧКА ДОСТУПА

Ethernet является **единственной внешней точкой доступа** к системе со стороны пользователя и обслуживающего персонала.

Через Ethernet обеспечивается:

- доступ к WEB-интерфейсу;
- передача конфигурационных данных;
- получение статуса системы для человека-оператора.

Ethernet **не используется** для межплатного обмена и не заменяет CANBUS.

7.2.2. РАЗГРАНИЧЕНИЕ АРХИТЕКТУРНЫХ УРОВНЕЙ

В архитектуре системы чётко разделены уровни:

- **Уровень управления дверьми**
 - локальная логика дверей;
 - обработка Alarm;
 - тайм-ауты;
 - Safety-инварианты.
- **Уровень распределённой логики**
 - CANBUS;
 - MASTER–SLAVE взаимодействие;
 - синхронизация состояний.
- **Уровень сервиса и эксплуатации**
 - Ethernet;
 - HTTP-сервер;
 - WEB-интерфейс.

Ethernet относится **исключительно к сервисному уровню**.

7.2.3. ВЗАИМОДЕЙСТВИЕ ETHERNET С ЛОГИКОЙ

Ethernet не оказывает прямого воздействия на:

- состояние замков;
- физическое управление дверями;
- выполнение зависимостей дверей;
- реакции на Alarm.

Все изменения, инициируемые через WEB:

- проходят валидацию;

- применяются только через механизм конфигурации;
- не воздействуют напрямую на текущие состояния дверей.

7.2.4. ETHERNET И РЕЖИМЫ РАБОТЫ СИСТЕМЫ

Ethernet не определяет режим работы системы:

- Normal / Degraded определяется состоянием CAN и логикой;
- отсутствие Ethernet не считается отказом системы;
- восстановление Ethernet не требует ресинхронизации дверей.

Ethernet может быть:

- полностью отключён;
- подключён периодически;
- подключён постоянно — без изменения поведения логики.

7.2.5. ОГРАНИЧЕНИЕ ВЛИЯНИЯ ETHERNET

Для обеспечения устойчивости системы:

- сетевые задачи имеют низкий приоритет;
- HTTP-обработка не блокирует основные циклы;
- ошибки Ethernet не распространяются на другие подсистемы.

7.2.6. КЛЮЧЕВОЙ ПРИНЦИП

Ethernet расширяет удобство эксплуатации, но не участвует в принятии решений управления дверьми.

7.3. ВСТРОЕННЫЙ HTTP-СЕРВЕР НА MASTER

Данный подпункт описывает встроенный HTTP-сервер, работающий на MASTER-контроллере, его назначение и общие принципы функционирования.

7.3.1. НАЗНАЧЕНИЕ HTTP-СЕРВЕРА

HTTP-сервер предназначен для:

- предоставления WEB-интерфейса пользователю;
- приёма и передачи конфигурационных данных;
- отображения текущего состояния системы;
- сервисной диагностики и обслуживания.

HTTP-сервер является **единственной точкой сетевого взаимодействия** с контроллером.

7.3.2. ОБЛАСТЬ РАБОТЫ HTTP-СЕРВЕРА

HTTP-сервер работает **только на MASTER-контроллере**.

SLAVE-контроллеры:

- не поднимают HTTP-сервер;
- не имеют прямого доступа к Ethernet;
- не принимают сетевые соединения.

7.3.3. АРХИТЕКТУРНЫЕ ОГРАНИЧЕНИЯ HTTP-СЕРВЕРА

HTTP-сервер:

- не работает в контуре реального времени;
- не управляет напрямую дверями;
- не обрабатывает события Alarm;
- не взаимодействует напрямую с аппаратными выходами.

Все запросы HTTP-сервера обслуживаются через сервисный слой прошивки.

7.3.4. ВЗАИМОДЕЙСТВИЕ HTTP-СЕРВЕРА С КОНФИГУРАЦИЕЙ

Через HTTP-сервер выполняются операции:

- загрузка новой конфигурации;
- проверка корректности конфигурации;
- активация конфигурации (атомарно);
- чтение текущей конфигурации и её версии.

HTTP-сервер **не изменяет** активную конфигурацию напрямую без процедуры валидации и подтверждения.

7.3.5. ОГРАНИЧЕНИЕ ВЛИЯНИЯ HTTP-НАГРУЗКИ

Для предотвращения влияния сетевой активности:

- HTTP-сервер обслуживается задачами низкого приоритета;
- длительные операции выполняются асинхронно;
- сетевые ошибки не блокируют работу системы.

7.3.6. ОБРАБОТКА ОШИБОК HTTP-СЕРВЕРА

При ошибках HTTP-сервера:

- система продолжает автономную работу;

- логика блокировки дверей не затрагивается;
- ошибки могут быть отображены в WEB-интерфейсе или зафиксированы в диагностике.

7.3.7. Ключевой принцип

HTTP-сервер — это интерфейс управления конфигурацией, а не механизм управления дверьми.

7.4. WEB-СЕАНС УПРАВЛЕНИЯ (УТОЧНЁННАЯ РЕДАКЦИЯ)

Данный подпункт описывает модель работы WEB-сеанса, роли пользователей и правила фиксации действий в системе.

7.4.1. Инициация WEB-СЕАНСА

WEB-сеанс управления:

- инициируется пользователем вручную;
- начинается после успешной аутентификации;
- устанавливается через HTTP-соединение с MASTER-контроллером.

WEB-сеанс всегда создаётся **в контексте роли пользователя**.

7.4.2. Роли пользователей WEB-ИНТЕРФЕЙСА

В системе предусмотрены следующие роли:

Администратор

Администратор имеет право:

- просматривать состояние системы;
- загружать, редактировать и активировать конфигурацию;
- выполнять сервисные операции;
- просматривать журнал событий и изменений.

Пользователь

Пользователь имеет право:

- просматривать текущее состояние системы;
- осуществлять мониторинг дверей (в том числе на карте помещений);
- просматривать диагностическую информацию.

Пользователь **не имеет права**:

- изменять конфигурацию;
- управлять параметрами системы;
- влиять на логику работы дверей.

7.4.3. Главный Администратор

В системе существует роль **Главного Администратора**, который дополнительно имеет право:

- создавать пользователей;
- удалять пользователей;
- изменять роли пользователей (Администратор / Пользователь);
- управлять учетными данными доступа.

Главный Администратор является ответственным лицом за управление доступом к системе.

7.4.4. Поведение системы при активном WEB-СЕАНСЕ

При активном WEB-сеансе:

- разрешён просмотр состояния системы (для всех ролей);
- разрешены операции конфигурирования (только для Администратора);
- разрешены операции управления пользователями (только для Главного Администратора).

При этом:

- логика блокировки дверей продолжает работу без изменений;
- WEB-сеанс не имеет приоритета над Alarm и Safety-механизмами;
- двери не переводятся в специальные режимы из-за WEB-сеанса.

7.4.5. ЗАВЕРШЕНИЕ WEB-СЕАНСА

WEB-сеанс завершается:

- по инициативе пользователя (кнопка «Отключиться»);
- по инициативе системы при явном выходе пользователя.

После завершения WEB-сеанса:

- система возвращается в автономный режим;
- сетевые ресурсы освобождаются;
- состояние дверей не изменяется.

7.4.6. АВАРИЙНОЕ ЗАВЕРШЕНИЕ WEB-СЕАНСА

При внезапном обрыве связи:

- WEB-сеанс считается аварийно завершённым;
- незавершённые операции конфигурации отменяются;
- активная конфигурация остаётся неизменной;
- система продолжает работу автономно.

7.4.7. ЛОГИРОВАНИЕ ДЕЙСТВИЙ ПОЛЬЗОВАТЕЛЕЙ

Все изменения конфигурации и управления пользователями:

- **обязательно логируются;**
- привязываются к:
 - идентификатору пользователя;
 - роли пользователя;
 - времени выполнения операции;
 - типу выполненного действия.

Логирование необходимо для:

- последующего анализа неисправностей;
- аудита изменений;
- определения ответственности при ошибках конфигурации.

7.4.8. КЛЮЧЕВОЙ ПРИНЦИП

Любое изменение системы имеет автора и след в журнале.

Анонимных изменений конфигурации не существует.

7.5. КОНФИГУРИРОВАНИЕ СИСТЕМЫ ЧЕРЕЗ WEB

Данный подпункт описывает процесс конфигурирования системы управления дверьми через WEB-интерфейс, правила применения конфигурации и требования к безопасности и целостности данных.

7.5.1. НАЗНАЧЕНИЕ WEB-КОНФИГУРИРОВАНИЯ

WEB-конфигуратор предназначен для:

- создания и редактирования конфигурации проекта;
- задания параметров дверей;
- задания логики зависимостей между дверями;
- настройки сетевых параметров;
- управления пользователями и их правами;
- просмотра и анализа конфигурации.

WEB-конфигурирование является **основным способом** настройки системы.

7.5.2. СОСТАВ КОНФИГУРАЦИОННЫХ ДАННЫХ

Через WEB задаются:

- список дверей проекта;
- соответствие дверей платам и локальным номерам;
- тип двери (NC / NO / CARD READER);
- параметры тайм-аутов дверей;
- таблица зависимостей дверей;
- параметры CAN, RS-485 (MODBUS), Ethernet;
- параметры пользователей и ролей.

7.5.3. ПРОЦЕСС ИЗМЕНЕНИЯ КОНФИГУРАЦИИ

Процесс конфигурирования включает этапы:

1. редактирование конфигурации в WEB-интерфейсе;
2. предварительная проверка корректности (валидация);
3. передача конфигурации на MASTER;
4. запись конфигурации во временную область памяти;
5. проверка целостности данных;
6. атомарная активация конфигурации.

До завершения всех этапов активная конфигурация остаётся неизменной.

7.5.4. ВАЛИДАЦИЯ КОНФИГУРАЦИИ

Перед активацией конфигурации выполняется проверка:

- корректности идентификаторов дверей;
- отсутствия конфликтов зависимостей;
- допустимости параметров тайм-аутов;
- уникальности сетевых параметров;
- соответствия версии формата конфигурации.

Некорректная конфигурация:

- не активируется;
- сопровождается сообщением об ошибке пользователю;
- не влияет на текущую работу системы.

7.5.5. АТОМАРНАЯ АКТИВАЦИЯ КОНФИГУРАЦИИ

Активация конфигурации выполняется атомарно:

- новая конфигурация становится активной только целиком;
- частично загруженные данные не используются;

- при ошибке происходит откат к предыдущей конфигурации. Это исключает появление промежуточных или неконсистентных состояний.

7.5.6. ПОВЕДЕНИЕ СИСТЕМЫ ПРИ ПРИМЕНЕНИИ КОНФИГУРАЦИИ

При активации новой конфигурации:

- логика зависимостей может быть временно приостановлена;
- физическая безопасность дверей сохраняется;
- резкие переключения состояний исключены;
- после активации выполняется мягкое включение логики.

7.5.7. ПОДТВЕРЖДЕНИЕ УСПЕШНОЙ ЗАГРУЗКИ

После успешной активации:

- MASTER отправляет подтверждение в WEB-интерфейс;
 - фиксируется событие в журнале;
 - обновляется версия активной конфигурации.
- Отсутствие подтверждения означает, что конфигурация не была применена.

7.5.8. ЛОГИРОВАНИЕ ИЗМЕНЕНИЙ КОНФИГУРАЦИИ

Каждое изменение конфигурации:

- фиксируется в журнале событий;
- привязывается к пользователю, выполнившему изменение;
- содержит информацию о времени и характере изменения.

7.5.9. КЛЮЧЕВОЙ ПРИНЦИП

Конфигурация либо применена полностью и корректно, либо система продолжает работу по предыдущей версии.

7.6. ПОВЕДЕНИЕ СИСТЕМЫ ПРИ ОТКАЗЕ ETHERNET

Данный подпункт описывает, как система управления дверьми ведёт себя при отсутствии или потере Ethernet-связи, и какие последствия это имеет для эксплуатации и безопасности.

7.6.1. ОТСУТСТВИЕ ETHERNET КАК ШТАТНЫЙ РЕЖИМ

Отсутствие Ethernet-подключения считается **нормальным и допустимым состоянием** системы.

При отсутствии Ethernet:

- система управления дверьми работает автономно;
- логика блокировки дверей выполняется в полном объёме;
- взаимодействие MASTER–SLAVE по CAN продолжается без изменений;
- локальная безопасность дверей полностью сохраняется.

Ethernet **не является обязательным** для работы системы.

7.6.2. ПОТЕРЯ ETHERNET ПРИ ОТСУТСТВИИ АКТИВНОГО WEB-СЕАНСА

Если Ethernet-связь отсутствует или обрывается, когда WEB-сеанс не активен:

- система продолжает работу без каких-либо изменений;
- не фиксируется аварийное состояние;
- дополнительные действия не требуются.

7.6.3. ПОТЕРЯ ETHERNET ПРИ АКТИВНОМ WEB-СЕАНСЕ

Если Ethernet-связь пропадает во время активного WEB-сеанса:

- WEB-сеанс считается аварийно завершённым;
- пользователь теряет соединение с системой;
- незавершённые операции конфигурирования отменяются;
- активная конфигурация остаётся неизменной и валидной.

Работа логики дверей при этом **не прерывается**.

7.6.4. ПОТЕРЯ ETHERNET ВО ВРЕМЯ ЗАГРУЗКИ КОНФИГУРАЦИИ

Если Ethernet-связь пропадает в процессе загрузки конфигурации:

- новая конфигурация считается **неприменённой**;
- активная конфигурация не изменяется;
- временные данные конфигурации отбрасываются;
- система продолжает работу по предыдущей версии.

Это исключает частичное или неконсистентное применение конфигурации.

7.6.5. ВОССТАНОВЛЕНИЕ ETHERNET-СВЯЗИ

После восстановления Ethernet:

- система готова к новому WEB-сеансу;
- повторная синхронизация логики не требуется;
- двери продолжают работу в текущем состоянии;

- пользователь может повторно подключиться вручную.
- Восстановление Ethernet **не инициирует** автоматических действий в системе.

7.6.6. ДИАГНОСТИКА ОТКАЗА ETHERNET

Отказ Ethernet может:

- фиксироваться как диагностическое событие;
- отображаться в WEB-интерфейсе после восстановления связи;
- использоваться для анализа эксплуатации системы.

Отказ Ethernet **не является критической ошибкой**.

7.6.7. КЛЮЧЕВОЙ ПРИНЦИП

Ethernet — сервисный канал.

Его отказ не влияет на безопасность и работу дверей.

7.7. БЕЗОПАСНОСТЬ СЕТЕВОГО ДОСТУПА

Данный подпункт определяет требования к защите сетевого доступа к системе управления дверьми при использовании Ethernet и WEB-интерфейса.

7.7.1. АУТЕНТИФИКАЦИЯ ПОЛЬЗОВАТЕЛЕЙ

Доступ к WEB-интерфейсу возможен **только после аутентификации**.

Основные требования:

- каждый пользователь имеет уникальную учетную запись;
- аутентификация обязательна для всех ролей;
- учетные данные не передаются в открытом виде;
- неуспешные попытки входа могут фиксироваться в журнале.

7.7.2. РАЗГРАНИЧЕНИЕ ПРАВ ДОСТУПА

Система поддерживает разграничение прав доступа на основе ролей:

- **Пользователь** — только просмотр и мониторинг;
- **Администратор** — конфигурирование и обслуживание;
- **Главный Администратор** — полный доступ и управление пользователями и правами.

Права доступа строго ограничивают доступные операции WEB-интерфейса.

7.7.3. ЗАЩИТА КОНФИГУРАЦИОННЫХ ОПЕРАЦИЙ

Операции изменения конфигурации:

- доступны только пользователям с правами Администратора;
- требуют активного WEB-сеанса;
- проходят обязательную валидацию;
- выполняются атомарно.

Каждая операция изменения конфигурации:

- логируется;
- привязывается к пользователю;
- имеет временную метку.

7.7.4. ЗАЩИТА ОТ НЕСАНКЦИОНИРОВАННОГО ДОСТУПА

Для повышения безопасности:

- отсутствуют “гостевые” или анонимные режимы доступа;
- сетевые сервисы доступны только при необходимости;
- запрещены скрытые или недокументированные интерфейсы управления.

7.7.5. ХРАНЕНИЕ УЧЕТНЫХ ДАННЫХ

Учетные данные пользователей:

- хранятся в защищённом виде;
- не хранятся в открытом текстовом формате;
- не передаются между MASTER и SLAVE.

Изменение учетных данных фиксируется в журнале.

7.7.6. АУДИТ И ТРАССИРУЕМОСТЬ

Система обеспечивает:

- журналирование входов пользователей;
- журналирование изменений конфигурации;
- возможность восстановления цепочки действий при анализе инцидентов.

Это позволяет:

- определить источник ошибки;
- восстановить историю изменений;
- повысить эксплуатационную надёжность.

7.7.7. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- безопасность сетевого доступа не должна влиять на работу логики дверей;
- отказ механизмов аутентификации не должен приводить к блокировке дверей;
- безопасность Ethernet не заменяет физическую безопасность системы.

7.7.8. КЛЮЧЕВОЙ ПРИНЦИП

Доступ к системе возможен только для авторизованных пользователей, а каждое действие имеет автора и след.

7.8. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

Данный подпункт фиксирует ограничения использования Ethernet и WEB-интерфейса, а также допущения, принятые при проектировании системы.

7.8.1. ОГРАНИЧЕНИЯ ETHERNET И WEB-ИНТЕРФЕЙСА

В системе приняты следующие ограничения:

- Ethernet не используется для управления дверьми в реальном времени;
- Ethernet не является резервным каналом управления;
- WEB-интерфейс не имеет прямого доступа к аппаратным выходам;
- WEB-интерфейс не может обходить Safety-инварианты;
- WEB-интерфейс не может принудительно изменить состояние двери вне логики.

Любые операции через WEB выполняются **только через утверждённые механизмы конфигурации**.

7.8.2. ОГРАНИЧЕНИЯ ПО ДОСТУПНОСТИ WEB

WEB-интерфейс:

- может быть недоступен в штатной эксплуатации;
- не требуется для нормальной работы системы;
- не должен рассматриваться как обязательный элемент управления объектом.

Отсутствие WEB-интерфейса не снижает функциональность системы управления дверьми.

7.8.3. ДОПУЩЕНИЯ ПО ЭКСПЛУАТАЦИИ

При проектировании системы предполагается, что:

- конфигурирование выполняется подготовленным персоналом;
- пользователи осознают последствия изменения конфигурации;
- WEB-доступ используется преимущественно на этапе настройки и обслуживания;
- постоянное подключение WEB-клиента не является обязательным.

7.8.4. ДОПУЩЕНИЯ ПО БЕЗОПАСНОСТИ

Приняты следующие допущения:

- физический доступ к контроллеру ограничен;
- безопасность Ethernet не заменяет физическую защиту оборудования;
- при отсутствии питания система переходит в безопасное состояние (все двери разблокированы).

7.8.5. ГРАНИЦЫ ОТВЕТСТВЕННОСТИ СИСТЕМЫ

Система управления дверьми:

- не отвечает за безопасность сети Ethernet;
- не отвечает за безопасность внешних WEB-клиентов;
- не отвечает за корректность действий пользователя;
- гарантирует безопасность дверей в рамках своих аппаратных и программных возможностей.

7.8.6. КЛЮЧЕВОЙ ПРИНЦИП

WEB и Ethernet — инструменты эксплуатации, а безопасность дверей обеспечивается локально и аппаратно.

8. WEB-ИНТЕРФЕЙС (VISUAL STUDIO)

8.1. НАЗНАЧЕНИЕ WEB-ИНТЕРФЕЙСА

WEB-интерфейс является основным инструментом взаимодействия пользователя с системой управления дверьми на этапах **настройки, эксплуатации и мониторинга**.

WEB-интерфейс предназначен для того, чтобы пользователь, **не обладающий знаниями в программировании**, мог:

- быстро сконфигурировать систему управления дверьми;
- задать параметры дверей и их логические зависимости;
- проверить корректность конфигурации;
- загрузить конфигурацию в контроллер;
- контролировать состояние системы и дверей в наглядной форме.

8.1.1. РОЛЬ WEB-ИНТЕРФЕЙСА В СИСТЕМЕ

WEB-интерфейс выполняет следующие функции:

- предоставляет человеко-ориентированное представление конфигурации проекта;
 - скрывает сложность внутренней архитектуры контроллера;
 - служит интерфейсом между пользователем и встроенным HTTP-сервером MASTER-контроллера.
- WEB-интерфейс **не управляет дверьми напрямую** и не взаимодействует с аппаратными выходами.

8.1.2. WEB-ИНТЕРФЕЙС КАК СРЕДСТВО КОНФИГУРИРОВАНИЯ

Основное назначение WEB-интерфейса — **конфигурирование системы**:

- создание и редактирование конфигурации проекта;
- задание параметров дверей;
- задание логики зависимостей между дверями;
- настройка сетевых параметров;
- управление пользователями и их правами.

WEB-интерфейс формирует конфигурацию в структурированном виде, понятном пользователю, и передаёт её в MASTER-контроллер.

8.1.3. WEB-ИНТЕРФЕЙС КАК СРЕДСТВО МОНИТОРИНГА

Дополнительно WEB-интерфейс используется для:

- просмотра текущего состояния дверей;
- мониторинга режимов работы системы;
- отображения аварийных состояний и сигнализации;
- визуализации дверей на карте помещений.

Мониторинг **не является обязательным режимом** и может использоваться по необходимости.

8.1.4. ОГРАНИЧЕНИЯ WEB-ИНТЕРФЕЙСА

WEB-интерфейс:

- не является частью контура реального времени;
- не обеспечивает управление дверьми при отсутствии MASTER;
- не заменяет аппаратные механизмы безопасности;
- не влияет на поведение дверей при отказах сети или питания.

8.1.5. КЛЮЧЕВОЙ ПРИНЦИП

WEB-интерфейс — это средство настройки и наблюдения,
а не средство обеспечения безопасности.

8.2. РОЛИ ПОЛЬЗОВАТЕЛЕЙ И СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

Данный подпункт описывает **категории пользователей WEB-интерфейса**, их права и типовые сценарии работы с системой управления дверьми.

8.2.1. РОЛИ ПОЛЬЗОВАТЕЛЕЙ

В системе WEB-интерфейса определены следующие роли.

Пользователь

Роль предназначена для эксплуатации и наблюдения.

Пользователь имеет право:

- просматривать текущее состояние системы;
- мониторить состояние дверей;
- просматривать карту помещений с дверями;
- просматривать журнал событий (read-only).

Пользователь **не имеет права**:

- изменять конфигурацию;
- загружать конфигурацию в контроллер;
- управлять пользователями;
- влиять на логику работы дверей.

Администратор

Роль предназначена для настройки и обслуживания системы.

Администратор имеет право:

- выполнять все действия Пользователя;
- создавать и редактировать конфигурацию проекта;
- настраивать двери и зависимости;
- загружать и активировать конфигурацию;
- просматривать и анализировать журнал изменений.

Администратор **не имеет права**:

- управлять учетными записями других администраторов;
- изменять роль Главного Администратора.

Главный Администратор

Роль предназначена для управления доступом и ответственностью.

Главный Администратор имеет право:

- выполнять все действия Администратора;

- создавать, удалять и редактировать пользователей;
- назначать роли Пользователь / Администратор;
- управлять учетными данными доступа;
- просматривать полный аудит действий пользователей.

8.2.2. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ WEB-ИНТЕРФЕЙСА

Сценарий 1 — Первичная настройка системы

- Вход под Главным Администратором;
- создание пользователей;
- конфигурирование дверей и логики;
- загрузка конфигурации в MASTER;
- проверка корректности работы.

Сценарий 2 — Эксплуатация и мониторинг

- Вход под Пользователем;
- просмотр текущего состояния дверей;
- мониторинг на карте помещений;
- анализ текущих аварийных состояний.

Сценарий 3 — Изменение конфигурации

- Вход под Администратором;
- корректировка параметров дверей или логики;
- валидация конфигурации;
- атомарная загрузка конфигурации;
- проверка результата.

Сценарий 4 — Администрирование доступа

- Вход под Главным Администратором;
- добавление или удаление пользователей;
- изменение ролей;
- аудит действий пользователей.

8.2.3. ПРИНЦИПЫ РАЗГРАНИЧЕНИЯ ДОСТУПА

В системе применяются следующие принципы:

- права строго привязаны к роли;
- запрещены неявные или скрытые привилегии;
- интерфейс WEB динамически ограничивает доступные действия;
- все действия пользователей с правами изменения фиксируются в журнале.

8.2.4. СВЯЗ РОЛЕЙ С БЕЗОПАСНОСТЬЮ СИСТЕМЫ

Роли пользователей:

- **не влияют** на физическую безопасность дверей;
- **не могут** обходить Safety-инварианты;
- **не могут** напрямую управлять замками.

WEB-интерфейс всегда остаётся сервисным уровнем.

8.2.5. КЛЮЧЕВОЙ ПРИНЦИП

**Пользователь видит,
Администратор настраивает,
Главный Администратор отвечает.**

8.3. УПРАВЛЕНИЕ ПОЛЬЗОВАТЕЛЯМИ

Данный подпункт описывает механизмы управления учетными записями пользователей WEB-интерфейса, их ролями и правилами аудита действий.

8.3.1. НАЗНАЧЕНИЕ УПРАВЛЕНИЯ ПОЛЬЗОВАТЕЛЯМИ

Управление пользователями предназначено для:

- ограничения доступа к функциям WEB-интерфейса;
- разделения ответственности между участниками эксплуатации;
- обеспечения трассируемости изменений конфигурации;
- повышения эксплуатационной и юридической ответственности.

8.3.2. УЧЕТНЫЕ ЗАПИСИ ПОЛЬЗОВАТЕЛЕЙ

Каждый пользователь WEB-интерфейса:

- имеет уникальную учетную запись;
- идентифицируется по имени пользователя;
- аутентифицируется с использованием пароля.

Учетные записи:

- создаются и удаляются только Главным Администратором;
- не могут дублироваться;
- не могут быть анонимными.

8.3.3. НАЗНАЧЕНИЕ И ИЗМЕНЕНИЕ РОЛЕЙ

Каждой учетной записи назначается одна роль:

- Пользователь;
- Администратор.

Назначение и изменение ролей:

- выполняется только Главным Администратором;
- фиксируется в журнале событий;
- вступает в силу немедленно после сохранения.

Главный Администратор не может быть лишён своей роли через WEB-интерфейс.

8.3.4. УПРАВЛЕНИЕ УЧЕТНЫМИ ДАННЫМИ

Управление учетными данными включает:

- смену пароля пользователя;
- сброс пароля (по инициативе Главного Администратора);
- блокировку учетной записи (при необходимости).

Пароли:

- не отображаются в открытом виде;
- хранятся в защищённом формате;
- не передаются по сети в явном виде.

8.3.5. АУДИТ ДЕЙСТВИЙ ПОЛЬЗОВАТЕЛЕЙ

Все действия пользователей с правами изменения:

- конфигурирование системы;
- загрузка конфигурации;
- управление пользователями;

обязательно фиксируются в журнале аудита с указанием:

- идентификатора пользователя;
- роли пользователя;
- времени действия;
- типа операции.

Журнал аудита используется для:

- анализа ошибок;
- расследования инцидентов;
- восстановления истории изменений.

8.3.6. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

- управление пользователями осуществляется только через WEB;
- SLAVE-контроллеры не хранят данные пользователей;
- отказ механизма управления пользователями не влияет на работу дверей.

8.3.7. КЛЮЧЕВОЙ ПРИНЦИП

**Каждый пользователь известен,
каждое действие зафиксировано,
ответственность персонализирована.**

8.4. ОБЩАЯ АРХИТЕКТУРА WEB-ПРИЛОЖЕНИЯ

Данный подпункт описывает общую архитектуру WEB-приложения, принципы его построения и взаимодействие с встроенным HTTP-сервером MASTER-контроллера.

8.4.1. ОБЩИЙ ПОДХОД К АРХИТЕКТУРЕ

WEB-приложение проектируется как **клиентское приложение**, работающее поверх встроенного HTTP-сервера MASTER-контроллера. Основные принципы архитектуры:

- разделение логики отображения и логики управления;
- минимальная нагрузка на контроллер;
- предсказуемое и устойчивое поведение при обрывах связи;
- простота расширения функциональности.

WEB-приложение не содержит логики управления дверьми — вся логика выполняется на стороне контроллера.

8.4.2. АРХИТЕКТУРНЫЕ КОМПОНЕНТЫ WEB-ПРИЛОЖЕНИЯ

WEB-приложение логически состоит из следующих компонентов:

- пользовательский интерфейс (UI);
- слой взаимодействия с HTTP-API контроллера;
- модуль валидации вводимых данных;

- модуль отображения состояния и мониторинга;
 - модуль работы с пользователями и правами доступа.
- Каждый компонент имеет чётко определённую зону ответственности.

8.4.3. ВЗАИМОДЕЙСТВИЕ С HTTP-СЕРВЕРОМ КОНТРОЛЛЕРА

WEB-приложение взаимодействует с MASTER-контроллером через HTTP-API.

Принципы взаимодействия:

- запросы выполняются по инициативе WEB-клиента;
- контроллер не иницирует HTTP-соединения;
- все операции подтверждаются явными ответами сервера;
- ошибки передаются в структурированном виде.

8.4.4. РАБОТА С СОСТОЯНИЕМ СИСТЕМЫ

WEB-приложение:

- запрашивает текущий снимок состояния системы;
- отображает полученные данные пользователю;
- не хранит состояние как источник истины.

Источником истины всегда является MASTER-контроллер.

При расхождении данных:

- приоритет имеет состояние, полученное от контроллера;
- WEB-приложение обязано обновить отображаемые данные.

8.4.5. ОБРАБОТКА ОБРЫВОВ СВЯЗИ

При потере связи с контроллером:

- WEB-приложение отображает состояние «Нет соединения»;
- пользовательские действия блокируются;
- незавершённые операции считаются отменёнными.

После восстановления связи:

- требуется повторное подключение;
- данные состояния запрашиваются заново.

8.4.6. ТРЕБОВАНИЯ К ПРОИЗВОДИТЕЛЬНОСТИ И УСТОЙЧИВОСТИ

WEB-приложение должно:

- корректно работать при медленных соединениях;
- не создавать чрезмерную нагрузку на контроллер;
- корректно обрабатывать ошибки и тайм-ауты;
- оставаться отзывчивым для пользователя.

8.4.7. ОГРАНИЧЕНИЯ АРХИТЕКТУРЫ

WEB-приложение:

- не должно выполнять длительных операций на стороне контроллера;
- не должно полагаться на постоянное соединение;
- не должно влиять на безопасность системы.

8.4.8. КЛЮЧЕВОЙ ПРИНЦИП

WEB-приложение отображает и настраивает, контроллер управляет и отвечает за безопасность.

8.5. ЭКРАН СТРУКТУРЫ ПРОЕКТА (ДВЕРИ И ЗАВИСИМОСТИ)

Данный подпункт описывает основной экран WEB-интерфейса, предназначенный для визуального представления структуры проекта, дверей и их логических зависимостей.

8.5.1. НАЗНАЧЕНИЕ ЭКРАНА СТРУКТУРЫ ПРОЕКТА

Экран структуры проекта используется для:

- отображения всех дверей проекта;
- отображения связей и зависимостей между дверями;
- понимания общей логики работы системы;
- навигации по конфигурации проекта.

Этот экран является **точкой входа** для работы с конфигурацией.

8.5.2. ПРЕДСТАВЛЕНИЕ ДВЕРЕЙ

На экране отображается список дверей с основными атрибутами:

- идентификатор двери (globalDoorId / techId);
- принадлежность к плате (Node ID);
- локальный номер двери;
- тип двери (NC / NO / CARD READER);
- краткое описание (comment).

Отображение должно быть:

- наглядным;
- однозначным;
- не перегруженным техническими деталями.

8.5.3. ОТОБРАЖЕНИЕ ЗАВИСИМОСТЕЙ

Зависимости между дверями отображаются:

- в виде списка связанных дверей;
- в виде графической схемы (опционально);
- с указанием направления зависимости (кто блокирует кого).

Экран позволяет понять:

- какие двери участвуют в логике;
- какие двери блокируются при открытии выбранной двери.

8.5.4. НАВИГАЦИЯ И ФИЛЬТРАЦИЯ

Экран структуры проекта поддерживает:

- фильтрацию дверей по платам;
- поиск двери по идентификатору или комментарию;
- группировку дверей для удобства просмотра.

Навигация не изменяет конфигурацию.

8.5.5. ПЕРЕХОД К КОНФИГУРИРОВАНИЮ

С экрана структуры проекта пользователь может:

- перейти к детальному конфигурированию выбранной двери;
- перейти к редактированию зависимостей;
- просмотреть текущие параметры двери.

Переходы не инициируют изменений без явного подтверждения.

8.5.6. ОГРАНИЧЕНИЯ ЭКРАНА

Экран структуры проекта:

- не выполняет изменений конфигурации напрямую;
- не управляет дверьми;
- не влияет на текущую работу системы.

Все изменения выполняются через специализированные экраны конфигурирования.

8.5.7. КЛЮЧЕВОЙ ПРИНЦИП

Экран структуры проекта показывает, как система устроена, а не как она работает в данный момент.

8.6. ЭКРАН КОНФИГУРИРОВАНИЯ ДВЕРЕЙ

Данный подпункт описывает экран WEB-интерфейса, предназначенный для **настройки параметров конкретной двери** и связанных с ней базовых свойств.

8.6.1. НАЗНАЧЕНИЕ ЭКРАНА КОНФИГУРИРОВАНИЯ ДВЕРИ

Экран конфигурирования двери используется для:

- задания и редактирования параметров двери;
- определения поведения двери в различных режимах;
- подготовки данных для формирования логики зависимостей.

Экран работает **на уровне одной двери** и не изменяет логику других дверей напрямую.

8.6.2. ИДЕНТИФИКАЦИЯ ДВЕРИ

На экране отображаются идентификационные данные двери (read-only):

- globalDoorId;
- Node ID контроллера;
- локальный номер двери на плате;
- технический идентификатор двери (techId);
- обозначение двери по чертежам (drawingId);
- комментарий (comment).

Это позволяет однозначно сопоставить дверь:

- с физическим подключением;
- с проектной документацией.

8.6.3. ТИП ДВЕРИ И БАЗОВОЕ ПОВЕДЕНИЕ

На экране настраиваются:

- тип двери:
 - NC;
 - NO;
 - CARD READER;
- базовое поведение двери в Normal и Degraded режимах.

Изменение типа двери:

- влияет на поведение при деградации;
- учитывается при валидации конфигурации;
- не может нарушать инварианты безопасности.

8.6.4. ТАЙМ-АУТЫ ДВЕРИ

Для двери задаются параметры тайм-аутов:

- тайм-аут допустимого открытого состояния двери;
- тайм-аут после закрытия двери (задержка реакции логики).

Параметры тайм-аутов:

- имеют допустимые диапазоны;
- валидируются перед сохранением;
- применяются после активации конфигурации.

8.6.5. ОГРАНИЧЕНИЯ ЭКРАНА КОНФИГУРИРОВАНИЯ ДВЕРИ

Экран конфигурирования двери:

- не управляет дверью в реальном времени;
- не активирует замок напрямую;
- не изменяет поведение Alarm;
- не изменяет логику сигнализации;
- не может изменить состояние двери вне логики зависимостей.

Все изменения вступают в силу **только после загрузки и активации конфигурации**.

8.6.6. КЛЮЧЕВОЙ ПРИНЦИП

Экран конфигурирования двери задаёт параметры двери, но не влияет на аварийную и сигнальную логику.

8.7. ЭКРАН КОНФИГУРИРОВАНИЯ ЛОГИКИ ЗАВИСИМОСТЕЙ

Данный подпункт описывает экран WEB-интерфейса, предназначенный для **задания и редактирования логики зависимостей между дверями** — ключевого элемента поведения всей системы.

8.7.1. НАЗНАЧЕНИЕ ЭКРАНА ЛОГИКИ ЗАВИСИМОСТЕЙ

Экран конфигурирования логики зависимостей используется для:

- задания правил взаимодействия дверей;
- определения, какие двери должны блокироваться при открытии выбранной двери;
- проверки корректности и полноты логики проекта;
- наглядного представления сложных взаимосвязей между дверями.

Экран работает **на уровне логики**, а не на уровне физических параметров двери.

8.7.2. БАЗОВАЯ МОДЕЛЬ ЛОГИКИ

В WEB-интерфейсе используется **упрощённая и единая модель логики**:

Если дверь физически открыта, все двери, связанные с ней по зависимостям, должны быть заблокированы.

Для каждой двери задаётся:

- список дверей, которые блокируются при её открытии.

Других типов логических условий в системе не предусматривается.

8.7.3. ПРЕДСТАВЛЕНИЕ ЗАВИСИМОСТЕЙ В ИНТЕРФЕЙСЕ

Зависимости отображаются:

- в виде списка для выбранной двери;
- с указанием идентификаторов зависимых дверей;
- с возможностью визуального представления связей (граф / схема — опционально).

Интерфейс должен позволять пользователю:

- легко понимать направление зависимости;
- быстро находить связанные двери.

8.7.4. РЕДАКТИРОВАНИЕ ЗАВИСИМОСТЕЙ

Редактирование зависимостей включает:

- добавление двери в список блокируемых;
- удаление двери из списка блокируемых;
- просмотр всех дверей, на которые влияет выбранная дверь.

При редактировании:

- изменения выполняются только в конфигурации;
- текущая работа системы не затрагивается;
- требуется явное подтверждение изменений.

8.7.5. ВАЛИДАЦИЯ ЛОГИКИ ЗАВИСИМОСТЕЙ

WEB-интерфейс выполняет предварительную валидацию:

- отсутствие ссылок на несуществующие двери;
 - корректность идентификаторов дверей;
 - отсутствие дублирующихся зависимостей;
 - допустимость логики с точки зрения базовых правил системы.
- Глубокая логическая валидация выполняется также на стороне MASTER.

8.7.6. ПЕРЕСЕЧЕНИЕ И СОВМЕСТНОЕ УЧАСТИЕ ДВЕРЕЙ

Экран допускает ситуации, когда:

- одна дверь участвует в нескольких зависимостях;
- двери разных плат участвуют в общей логике;
- логика не совпадает с физическим расположением дверей.

WEB-интерфейс **не ограничивает** такие сценарии, так как они типичны для реальных проектов.

8.7.7. ОГРАНИЧЕНИЯ ЭКРАНА ЛОГИКИ ЗАВИСИМОСТЕЙ

Экран логики зависимостей:

- не управляет дверьми напрямую;
 - не активирует блокировки в реальном времени;
 - не влияет на режим Alarm;
 - не изменяет физическое состояние дверей.
- Все изменения применяются только после загрузки конфигурации.

8.7.8. КЛЮЧЕВОЙ ПРИНЦИП

WEB-интерфейс описывает правила взаимодействия дверей, а их исполнение всегда остаётся за контроллером.

8.8. ЭКРАН МОНИТОРИНГА СОСТОЯНИЯ ДВЕРЕЙ

Данный подпункт описывает экран WEB-интерфейса, предназначенный для **оперативного мониторинга текущего состояния системы и дверей** без возможности влияния на их работу.

8.8.1. НАЗНАЧЕНИЕ ЭКРАНА МОНИТОРИНГА

Экран мониторинга используется для:

- наблюдения за текущим состоянием дверей;
- контроля режимов работы системы (Normal / Degraded);
- оперативного выявления аварийных и нештатных ситуаций;
- эксплуатации системы без вмешательства в конфигурацию.

Экран предназначен для **Пользователей и Администраторов**.

8.8.2. ОТОБРАЖАЕМЫЕ ДАННЫЕ ПО ДВЕРЯМ

Для каждой двери отображаются:

- идентификатор двери (techId / globalDoorId);
- принадлежность к плате (Node ID);
- физическое состояние двери (открыта / закрыта);
- состояние замка (заблокирована / разблокирована);
- логическое состояние (Normal / Degraded / Alarm);
- состояние сигнализации (активна / не активна);
- превышение тайм-аута открытого состояния (если есть).

Данные отображаются **в режиме реального времени мониторинга**, без управления.

8.8.3. ИНДИКАЦИЯ СОСТОЯНИЙ

Состояния дверей визуализируются:

- цветовой индикацией;
- иконками состояния;
- текстовыми статусами.

Аварийные и приоритетные состояния:

- выделяются визуально;
- имеют приоритет над нормальными состояниями.

8.8.4. ОБНОВЛЕНИЕ ДАННЫХ

Обновление данных мониторинга:

- выполняется периодически;
- может быть инициировано вручную;
- не требует постоянного соединения.

Частота обновления:

- не влияет на логику работы системы;
- выбирается с учётом нагрузки на контроллер.

8.8.5. ПОВЕДЕНИЕ ПРИ ПОТЕРЕ СВЯЗИ

При потере связи с контроллером:

- экран отображает состояние «Нет соединения»;
- данные считаются устаревшими;
- пользовательские действия мониторинга блокируются.

После восстановления связи:

- данные обновляются автоматически;
- требуется подтверждение актуальности состояния.

8.8.6. ОГРАНИЧЕНИЯ ЭКРАНА МОНИТОРИНГА

Экран мониторинга:

- не позволяет управлять дверьми;
- не позволяет изменять конфигурацию;
- не отменяет Alarm;
- не влияет на работу логики зависимостей.

8.8.7. КЛЮЧЕВОЙ ПРИНЦИП

Мониторинг показывает текущее состояние, но не вмешивается в работу системы.

8.9. ОТОБРАЖЕНИЕ ДВЕРЕЙ НА КАРТЕ ПОМЕЩЕНИЙ

Данный подпункт описывает функциональность WEB-интерфейса по **визуализации дверей на карте помещений** для удобного мониторинга и ориентации пользователя.

8.9.1. НАЗНАЧЕНИЕ КАРТЫ ПОМЕЩЕНИЙ

Отображение дверей на карте помещений предназначено для:

- наглядного представления расположения дверей;
- быстрого понимания текущего состояния объекта;
- удобного мониторинга больших проектов с множеством дверей;
- сопоставления логических событий с реальной геометрией объекта.

Карта используется **исключительно для мониторинга**.

8.9.2. ИСТОЧНИК И ФОРМАТ КАРТЫ

Карта помещений:

- загружается пользователем в WEB-интерфейс;
- представляет собой статическое изображение или набор изображений;
- не содержит встроенной логики управления.

Поддерживаемые форматы и правила загрузки определяются отдельно.

8.9.3. ПРИВЯЗКА ДВЕРЕЙ К КАРТЕ

Для каждой двери может быть задано:

- положение на карте;
- идентификатор двери, соответствующий проектной документации (drawingId).

Привязка:

- используется только для отображения;
- не влияет на логику зависимостей;
- не влияет на физическое управление дверьми.

8.9.4. ОТОБРАЖЕНИЕ СОСТОЯНИЯ ДВЕРЕЙ НА КАРТЕ

На карте для каждой двери отображается:

- текущее состояние замка (цвет / иконка);
- логическое состояние двери;
- аварийные состояния (Alarm, сигнализация).

Обновление состояния на карте:

- синхронизировано с экраном мониторинга;
- не требует отдельной логики.

8.9.5. ПОВЕДЕНИЕ ПРИ ОТКАЗАХ И ДЕГРАДАЦИИ

При переходе системы или отдельных дверей в Degraded:

- состояние отображается визуально;
- двери помечаются соответствующим статусом;
- карта не изменяет своё поведение.

При потере связи:

- карта отображает последнее известное состояние;
- пользователь уведомляется о неактуальности данных.

8.9.6. ОГРАНИЧЕНИЯ КАРТЫ ПОМЕЩЕНИЙ

Карта помещений:

- не является источником истины;
- не управляет дверьми;
- не позволяет изменять конфигурацию;
- не влияет на безопасность системы.

8.9.7. Ключевой принцип

Карта помещений помогает видеть объект, но не управлять им.

8.10. ЖУРНАЛ СОБЫТИЙ И ИЗМЕНЕНИЙ

Данный подпункт описывает журналирование событий системы и действий пользователей, а также принципы отображения журнала в WEB-интерфейсе.

8.10.1. НАЗНАЧЕНИЕ ЖУРНАЛА

Журнал предназначен для:

- анализа работы системы;
- диагностики неисправностей;
- восстановления последовательности событий;
- аудита действий пользователей;
- определения ответственности при ошибках конфигурации.

Журнал является **информационным и диагностическим инструментом**, а не средством управления.

8.10.2. Типы фиксируемых событий

В журнале фиксируются следующие типы событий.

Системные события

- запуск и перезапуск MASTER;
- переход системы в режим Degraded и выход из него;
- потеря и восстановление CAN-связи;
- ошибки внутренних подсистем.

События дверей

- физическое открытие двери;
- физическое закрытие двери;
- переход двери в состояние Alarm;
- выход двери из состояния Alarm;
- превышение тайм-аута открытой двери.

⚠ Важно:

Журнал фиксирует **факты открытия и закрытия дверей**, а не команды блокировки или разблокировки замков.

8.10.3. СОБЫТИЯ КОНФИГУРАЦИИ

Фиксируются события:

- загрузка новой конфигурации;
- успешная активация конфигурации;
- отказ в активации конфигурации;
- откат к предыдущей конфигурации.

Для каждого события указывается:

- версия конфигурации;
- пользователь, выполнивший действие;
- результат операции.

8.10.4. АУДИТ ДЕЙСТВИЙ ПОЛЬЗОВАТЕЛЕЙ

Журнал аудита включает:

- вход пользователя в WEB-интерфейс;
- выход пользователя;
- изменение конфигурации;
- управление пользователями.

Каждое действие сопровождается:

- идентификатором пользователя;
- ролью пользователя;
- временной меткой;
- типом действия.

8.10.5. ОТОБРАЖЕНИЕ ЖУРНАЛА В WEB-ИНТЕРФЕЙСЕ

WEB-интерфейс предоставляет:

- просмотр журнала в хронологическом порядке;
- фильтрацию по типам событий;
- фильтрацию по дверям;
- фильтрацию по пользователям;

- поиск по журналу.
- Журнал отображается в режиме только чтения.

8.10.6. ХРАНЕНИЕ И ОБЪЁМ ЖУРНАЛА

Журнал хранится:

- в энергонезависимой памяти контроллера (QSPI Flash);
- в кольцевом буфере фиксированного размера.

При переполнении:

- самые старые записи перезаписываются;
- целостность новых записей сохраняется.

8.10.7. ОГРАНИЧЕНИЯ ЖУРНАЛА

Журнал:

- не используется для восстановления логики системы;
- не влияет на управление дверьми;
- не является источником конфигурации.

8.10.8. КЛЮЧЕВОЙ ПРИНЦИП

Журнал отвечает на вопрос “что произошло и кто это сделал”, а не “что должно было произойти”.

8.11. ЗАГРУЗКА, СОХРАНЕНИЕ И ВЕРСИЯ КОНФИГУРАЦИИ

Данный подпункт описывает, как WEB-интерфейс работает с конфигурацией системы: её хранение, загрузку в контроллер, версионирование и отображение информации о конфигурации пользователю.

8.11.1. НАЗНАЧЕНИЕ УПРАВЛЕНИЯ КОНФИГУРАЦИЕЙ

Механизмы загрузки и хранения конфигурации предназначены для:

- безопасного изменения логики работы системы;
- сохранения целостности данных;
- возможности отслеживания изменений;
- исключения неопределённых состояний системы.

Конфигурация является **единственным источником логики работы системы**.

8.11.2. ХРАНЕНИЕ КОНФИГУРАЦИИ

Конфигурация системы:

- хранится на MASTER-контроллере;
- размещается в энергонезависимой памяти (QSPI Flash);
- хранится как структурированный набор данных.

Одновременно поддерживаются:

- **активная конфигурация;**
- **предыдущая конфигурация** (для отката при ошибке).

8.11.3. ВЕРСИЯ КОНФИГУРАЦИИ

Каждая конфигурация имеет версию, включающую:

- номер версии конфигурации;
- дату и время создания;
- идентификатор пользователя, выполнившего изменения;
- версию формата конфигурации.

Версия конфигурации:

- отображается в WEB-интерфейсе;
- фиксируется в журнале;
- используется для диагностики и аудита.

8.11.4. ВЕРСИЯ ФОРМАТА КОНФИГУРАЦИИ

Версия формата конфигурации определяет:

- структуру данных конфигурации;
- набор поддерживаемых параметров;
- правила интерпретации конфигурации прошивкой.

Версия формата:

- задаётся прошивкой;
- проверяется при загрузке конфигурации;
- используется для обеспечения совместимости.

Несовместимая версия формата:

- не принимается системой;
- сопровождается сообщением об ошибке пользователю.

8.11.5. ПРОЦЕСС ЗАГРУЗКИ КОНФИГУРАЦИИ

Загрузка конфигурации через WEB включает этапы:

1. передача конфигурации в MASTER;
2. запись во временную область памяти;
3. проверка целостности данных;
4. проверка версии формата;
5. валидация конфигурации;
6. атомарная активация.

До успешного завершения всех этапов:

- активная конфигурация остаётся без изменений.

8.11.6. ПОДТВЕРЖДЕНИЕ И ОТОБРАЖЕНИЕ РЕЗУЛЬТАТА

После загрузки конфигурации:

- WEB-интерфейс получает явное подтверждение результата;
- отображается статус операции (успех / ошибка);
- обновляется информация о версии активной конфигурации.

Отсутствие подтверждения означает, что конфигурация **не была применена**.

8.11.7. ОГРАНИЧЕНИЯ УПРАВЛЕНИЯ КОНФИГУРАЦИЕЙ

- конфигурация не может изменяться без WEB-сеанса;
- SLAVE-контроллеры не хранят конфигурацию проекта;
- конфигурация не может быть изменена через RS-485 или CAN;
- конфигурация не может быть частично активирована.

8.11.8. КЛЮЧЕВОЙ ПРИНЦИП

Конфигурация либо валидна и активна целиком, либо система продолжает работу по предыдущей версии.

8.12. ОБРАБОТКА ОШИБОК И УВЕДОМЛЕНИЯ ПОЛЬЗОВАТЕЛЯ

Данный подпункт описывает, как WEB-интерфейс информирует пользователя об ошибках, предупреждениях и результатах операций, а также общие принципы обработки нештатных ситуаций.

8.12.1. НАЗНАЧЕНИЕ ОБРАБОТКИ ОШИБОК

Механизмы обработки ошибок предназначены для:

- информирования пользователя о проблемах и ограничениях;
- предотвращения некорректных действий;
- повышения предсказуемости работы системы;
- упрощения диагностики и эксплуатации.

WEB-интерфейс не скрывает ошибки и не пытается автоматически «исправлять» их без явного участия пользователя.

8.12.2. КЛАССИФИКАЦИЯ ОШИБОК

Ошибки и уведомления разделяются на следующие категории:

- **Информационные сообщения**
(подтверждение операций, состояние системы, результаты действий);
- **Предупреждения**
(потенциально опасные или нежелательные конфигурации, нетипичные параметры);
- **Ошибки**
(невозможность выполнения операции, некорректные данные, отказ загрузки конфигурации).

8.12.3. ИСТОЧНИКИ ОШИБОК

Ошибки могут возникать на следующих уровнях:

- WEB-приложение (ошибки ввода, недопустимые значения);
- HTTP-сервер MASTER-контроллера;
- логика валидации конфигурации;
- сетевые ошибки (потеря связи, тайм-ауты);
- внутренние ошибки контроллера.

WEB-интерфейс должен корректно отображать источник ошибки.

8.12.4. ОТОБРАЖЕНИЕ ОШИБОК ПОЛЬЗОВАТЕЛЮ

WEB-интерфейс обязан:

- отображать сообщения в понятной форме;
- указывать причину ошибки;
- при возможности — указывать способ её устранения;
- не допускать двусмысленных или скрытых сообщений.

Критические ошибки:

- отображаются явно;
- требуют подтверждения пользователя.

8.12.5. ПОВЕДЕНИЕ ПРИ ОШИБКАХ КОНФИГУРИРОВАНИЯ

При ошибках конфигурирования:

- операция прерывается;
 - конфигурация не активируется;
 - активная конфигурация остаётся неизменной;
 - пользователь уведомляется о причине отказа.
- Ошибки конфигурирования не влияют на текущую работу дверей.

8.12.6. ПОВЕДЕНИЕ ПРИ СЕТЕВЫХ ОШИБКАХ

При сетевых ошибках:

- пользователь уведомляется о потере соединения;
- WEB-сеанс считается завершённым;
- пользовательские действия блокируются;
- требуется повторное подключение.

После восстановления связи WEB-интерфейс не предполагает, что операция была завершена успешно.

8.12.7. ЖУРНАЛИРОВАНИЕ ОШИБОК

Все существенные ошибки:

- фиксируются в журнале событий;
- сопровождаются временной меткой;
- при необходимости связываются с пользователем.

Это позволяет:

- анализировать причины неисправностей;
- восстанавливать цепочку событий;
- улучшать эксплуатационную надёжность.

8.12.8. КЛЮЧЕВОЙ ПРИНЦИП

Пользователь всегда должен понимать, что произошло и почему операция не была выполнена.

8.13. ОГРАНИЧЕНИЯ И ДОПУЩЕНИЯ

Данный подпункт фиксирует ограничения WEB-интерфейса и допущения, принятые при его проектировании и эксплуатации.

8.13.1. ОГРАНИЧЕНИЯ WEB-ИНТЕРФЕЙСА

WEB-интерфейс имеет следующие ограничения:

- не участвует в контуре реального времени;
- не управляет дверьми напрямую;
- не активирует и не деактивирует замки;
- не отменяет и не инициирует Alarm;
- не может нарушить инварианты безопасности системы;
- не функционирует без MASTER-контроллера.

WEB-интерфейс является **вспомогательным сервисным инструментом**, а не частью управляющего ядра системы.

8.13.2. ОГРАНИЧЕНИЯ ПРИ ОТСУТСТВИИ СВЯЗИ

При отсутствии связи с контроллером:

- WEB-интерфейс не может гарантировать актуальность данных;
- любые операции конфигурирования запрещены;
- мониторинг невозможен или помечается как неактуальный.

WEB-интерфейс не предпринимает попыток восстановить управление дверьми.

8.13.3. ДОПУЩЕНИЯ ПО ЭКСПЛУАТАЦИИ

При проектировании WEB-интерфейса предполагается, что:

- конфигурирование выполняется квалифицированным персоналом;
- пользователи понимают последствия изменения конфигурации;
- WEB-интерфейс используется преимущественно для настройки и мониторинга;
- постоянное подключение WEB-клиента не является обязательным.

8.13.4. ДОПУЩЕНИЯ ПО БЕЗОПАСНОСТИ

Приняты следующие допущения:

- физический доступ к оборудованию ограничен;
- безопасность Ethernet не заменяет физическую защиту;
- при отказе WEB-интерфейса система остаётся безопасной;
- при отсутствии питания все двери переходят в разблокированное (Safety) состояние.

8.13.5. ГРАНИЦЫ ОТВЕТСТВЕННОСТИ WEB-ИНТЕРФЕЙСА

WEB-интерфейс:

- не отвечает за физическую безопасность дверей;
- не отвечает за корректность аппаратных подключений;

- не отвечает за внешнюю IT-инфраструктуру;
- отвечает только за корректность отображения и передачи данных.

8.13.6. Ключевой принцип

Отказ WEB-интерфейса не должен ухудшать безопасность и работоспособность системы.

9. БЕЗОПАСНОСТЬ И ПРОМЫШЛЕННАЯ НАДЁЖНОСТЬ

Данный раздел описывает принципы обеспечения безопасности, устойчивости и предсказуемого поведения системы управления дверьми в условиях отказов, внешних воздействий и ошибок эксплуатации.

Система проектируется как **промышленная**, с приоритетом:

- безопасности людей;
- отказоустойчивости;
- детерминированного поведения.

9.1. МОДЕЛЬ УГРОЗ (LAN / CAN / RS-485 / ФИЗИЧЕСКИЙ ДОСТУП)

В системе рассматриваются следующие классы угроз.

9.1.1. УГРОЗЫ СО СТОРОНЫ LAN (ETHERNET)

Потенциальные угрозы:

- несанкционированный доступ к WEB-интерфейсу;
- попытка изменения конфигурации;
- попытка получения информации о системе.

Меры защиты:

- обязательная аутентификация пользователей;
- разграничение прав доступа;
- невозможность управления дверьми через Ethernet;
- логирование всех действий пользователей.

Отказ или компрометация LAN:

- **не влияет** на физическую безопасность дверей;
- **не влияет** на выполнение логики блокировки.

9.1.2. УГРОЗЫ СО СТОРОНЫ CANBUS

Потенциальные угрозы:

- потеря связи между платами;
- задержки или искажения сообщений;
- отказ одного из узлов сети.

Меры защиты:

- режим Degraded при потере связи;
- автономная безопасная работа дверей;
- отсутствие прямого управления замками по CAN без локальных проверок.

CANBUS рассматривается как **внутренняя доверенная сеть**, но с учётом возможных отказов.

9.1.3. УГРОЗЫ СО СТОРОНЫ RS-485

Потенциальные угрозы:

- попытка управления системой через RS-485;
- подмена или искажение данных.

Меры защиты:

- RS-485 работает только в режиме read-only;
- используется только MASTER-контроллер;
- любые команды записи игнорируются.

RS-485 **не может** изменить состояние дверей.

9.1.4. УГРОЗЫ ФИЗИЧЕСКОГО ДОСТУПА

Потенциальные угрозы:

- доступ к плате контроллера;
- вмешательство в датчики или исполнительные механизмы;
- обход аппаратных цепей.

Допущения:

- физическая защита объекта обеспечивается на уровне объекта;
- система не гарантирует безопасность при физическом обходе датчиков.

9.2. АУТЕНТИФИКАЦИЯ И ПРАВА ДОСТУПА

9.2.1. Принципы аутентификации

- доступ к WEB возможен только после аутентификации;

- каждый пользователь имеет уникальную учётную запись;
- отсутствуют анонимные режимы доступа.

9.2.2. РАЗГРАНИЧЕНИЕ ПРАВ

Используются роли:

- Пользователь;
- Администратор;
- Главный Администратор.

Каждая роль строго ограничивает доступные действия.

9.2.3. ПРИВЯЗКА ДЕЙСТВИЙ К ПОЛЬЗОВАТЕЛЮ

Все действия, влияющие на систему:

- изменения конфигурации;
- управление пользователями;

жёстко привязываются к пользователю и фиксируются в журнале.

9.3. ЗАЩИТА КОНФИГУРАЦИИ (ЦЕЛОСТНОСТЬ / ПОДМЕНА)

9.3.1. ЦЕЛОСТНОСТЬ КОНФИГУРАЦИИ

Конфигурация:

- хранится в QSPI Flash;
- имеет версию и контроль целостности (CRC);
- проверяется при загрузке и активации.

9.3.2. ЗАЩИТА ОТ ПОДМЕНЫ

Меры защиты:

- конфигурация принимается только через WEB;
- проверяется формат и версия;
- частичная активация невозможна.

9.3.3. АТОМАРНОСТЬ И ОТКАТ

- новая конфигурация активируется атомарно;
- при ошибке выполняется откат к предыдущей версии;
- система никогда не работает с «полуконфигурацией».

9.4. FAIL-SAFE СТРАТЕГИИ

9.4.1. ОТКАЗ ПИТАНИЯ

При отсутствии питания:

- все замки-соленоиды переходят в разблокированное состояние;
- все двери становятся доступными для выхода;
- система переходит в состояние **Safety**.

Это соответствует требованиям эвакуации и пожарной безопасности.

9.4.2. ОТКАЗ КОНТРОЛЛЕРА

При отказе MASTER или SLAVE:

- замки не удерживаются в заблокированном состоянии;
- Alarm не блокируется;
- двери остаются в безопасном состоянии.

9.4.3. ОТКАЗ CAN-СЕТИ

При потере CAN:

- система переходит в режим Degraded;
- логика зависимостей не применяется;
- двери работают автономно по типу NC / NO / CARD READER;
- Alarm всегда работает локально.

9.4.4. ОТКАЗ ETHERNET

Отказ Ethernet:

- не считается аварией;
- не влияет на безопасность;
- не влияет на логику дверей.

9.4.5. ОТКАЗ ДАТЧИКОВ И ИСПОЛНИТЕЛЕЙ

При неисправности:

- система не активирует замок при открытой двери;

- при неопределённости выбирается безопасное состояние;
- событие фиксируется в журнале.

9.5. EMC / ESD / ПОМЕХОУСТОЙЧИВОСТЬ НА УРОВНЕ ПО

9.5.1. ПРОГРАММНАЯ УСТОЙЧИВОСТЬ К ПОМЕХАМ

ПО должно учитывать:

- дребезг контактов датчиков и кнопок;
- кратковременные помехи линий;
- ложные переходы состояний.

Используются:

- программные фильтры;
- временные окна подтверждения состояний.

9.5.2. ЗАЩИТА ОТ ЛОЖНЫХ СОБЫТИЙ

- события считаются валидными только после подтверждения;
- одиночные импульсы не приводят к изменению логики;
- ошибки фиксируются, но не эскалируются без подтверждения.

9.5.3. ПОВЕДЕНИЕ ПРИ НЕОПРЕДЕЛЁННЫХ СОСТОЯНИЯХ

В любой неопределённой ситуации система:

- выбирает безопасное состояние;
- не удерживает дверь заблокированной;
- не блокирует эвакуацию.

9.6. КЛЮЧЕВЫЕ ПРИНЦИПЫ РАЗДЕЛА 9

- безопасность людей важнее логики;
- любой отказ должен приводить к безопасному состоянию;
- удалённые интерфейсы не управляют дверьми;
- все действия трассируемы;
- система предсказуема и детерминирована.

10. ТЕСТИРОВАНИЕ И ВАЛИДАЦИЯ

Данный раздел описывает подходы, методы и объём тестирования системы управления дверьми с целью подтверждения:

- корректности реализации логики;
- соответствия требованиям безопасности;
- устойчивости к отказам;
- готовности системы к промышленной эксплуатации.

Тестирование рассматривается как **обязательная часть жизненного цикла системы**, а не как вспомогательный этап.

10.1. МОДУЛЬНЫЕ ТЕСТЫ ЛОГИКИ

Модульные тесты направлены на проверку **отдельных программных компонентов** прошивки контроллера.

10.1.1. ОБЛАСТЬ МОДУЛЬНОГО ТЕСТИРОВАНИЯ

Модульно тестируются:

- логика обработки состояния двери;
- логика зависимостей дверей;
- обработка событий (открытие / закрытие / Alarm / тайм-аут);
- переходы между состояниями (Normal / Degraded / Alarm);
- обработка ошибок и граничных условий.

Каждый модуль тестируется **изолированно**, без зависимости от реального оборудования.

10.1.2. ПРОВЕРЯЕМЫЕ СЦЕНАРИИ

В рамках модульных тестов проверяется:

- корректность реакций на одиночные события;
- корректность последовательностей событий;
- соблюдение инвариантов безопасности;
- невозможность блокировки двери при физическом открытии;
- приоритет Alarm над логикой.

10.1.3. ЦЕЛЬ МОДУЛЬНЫХ ТЕСТОВ

Целью модульных тестов является:

- раннее выявление логических ошибок;

- снижение сложности интеграционного тестирования;
- обеспечение предсказуемости поведения системы.

10.2. ИНТЕГРАЦИОННЫЕ ТЕСТЫ (CAN / RS-485 / ETHERNET)

Интеграционные тесты проверяют взаимодействие подсистем между собой.

10.2.1. ИНТЕГРАЦИЯ MASTER–SLAVE по CAN

Проверяется:

- корректность обмена состояниями дверей;
- корректность передачи логических команд;
- устойчивость к потере и восстановлению CAN-связи;
- корректный переход в режим Degraded.

10.2.2. ИНТЕГРАЦИЯ RS-485 (MODBUS RTU)

Проверяется:

- корректность формирования кадров;
- соответствие формата данным проекта;
- передача данных только с MASTER;
- невозможность изменения состояния дверей по RS-485.

10.2.3. ИНТЕГРАЦИЯ ETHERNET И WEB

Проверяется:

- корректность WEB-сеансов;
- загрузка и активация конфигурации;
- поведение при обрыве Ethernet;
- невозможность влияния WEB на безопасность дверей.

10.3. СТЕНДОВЫЕ ИСПЫТАНИЯ ДВЕРЕЙ

Стендовые испытания проводятся с реальным оборудованием.

10.3.1. АППАРАТНАЯ ЧАСТЬ ИСПЫТАНИЙ

Используются:

- реальные замки-соленоиды;
- реальные датчики двери;
- кнопки Alarm;
- светофоры и зуммеры.

Испытания проводятся как в режиме Standalone, так и MASTER–SLAVE.

10.3.2. ПРОВЕРЯЕМЫЕ СЦЕНАРИИ

Проверяется:

- корректность блокировки и разблокировки;
- работа сигнализации;
- соблюдение тайм-аутов;
- поведение при быстрых и повторяющихся событиях;
- отсутствие ложных срабатываний.

10.3.3. ЦЕЛЬ СТЕНДОВЫХ ИСПЫТАНИЙ

Целью является подтверждение того, что:

- ПО корректно работает с реальной аппаратурой;
- нет расхождений между логикой и физикой двери.

10.4. ТЕСТЫ ОТКАЗОВ (ДАТЧИКИ, КНОПКИ, ПИТАНИЕ)

Тесты отказов проверяют поведение системы в нештатных ситуациях.

10.4.1. ОТКАЗЫ ДАТЧИКОВ

Проверяется:

- обрыв или залипание датчика двери;
- ложные переходы состояний;
- реакция системы на неопределённые состояния.

Система должна:

- фиксировать событие;
- выбирать безопасное поведение.

10.4.2. ОТКАЗЫ КНОПКИ ALARM

Проверяется:

- залипание кнопки;

- ложные срабатывания;
 - многократные нажатия.
- Alarm всегда должен сохранять приоритет.

10.4.3. ОТКАЗ ПИТАНИЯ

Проверяется:

- полное отключение питания;
- восстановление питания;
- отсутствие удержания замков без питания.

Все двери должны переходить в **разблокированное (Safety) состояние**.

10.5. НАГРУЗОЧНЫЕ ТЕСТЫ И УСТОЙЧИВОСТЬ RTOS

Нагрузочные тесты направлены на проверку устойчивости системы при длительной и интенсивной работе.

10.5.1. НАГРУЗОЧНЫЕ СЦЕНАРИИ

Проверяется:

- одновременная работа большого количества дверей;
- частые события открытия/закрытия;
- активный обмен по CAN;
- одновременный мониторинг через WEB.

10.5.2. УСТОЙЧИВОСТЬ ПЛАНИРОВЩИКА И ЗАДАЧ

Проверяется:

- отсутствие зависаний;
- отсутствие утечек памяти;
- корректная работа таймеров;
- устойчивость к перегрузке событий.

10.5.3. ДЛИТЕЛЬНАЯ РАБОТА

Проводятся тесты:

- непрерывной работы в течение длительного времени;
- многократных перезапусков;
- повторных переходов Normal ↔ Degraded.

10.6. ИТОГОВАЯ ВАЛИДАЦИЯ

По результатам тестирования подтверждается:

- соответствие системы требованиям проекта;
- соблюдение принципов безопасности;
- готовность системы к промышленной эксплуатации.

Результаты тестирования:

- документируются;
- используются для приёмки системы;
- могут быть основой для дальнейшей сертификации.

10.7. КЛЮЧЕВОЙ ПРИНЦИП РАЗДЕЛА 10

Система считается готовой только тогда, когда она корректно работает не только в норме, но и при отказах.

11. ПРОИЗВОДСТВЕННАЯ ПОДГОТОВКА И СОПРОВОЖДЕНИЕ

Данный раздел описывает процессы, связанные с **подготовкой системы к серийному производству, её вводу в эксплуатацию**, а также **дальнейшему сопровождению и обслуживанию** на протяжении жизненного цикла.

Раздел ориентирован не на разработчика, а на:

- производство;
- монтаж;
- сервис;
- эксплуатацию на объекте.

11.1. ПЕРВИЧНАЯ ПРОШИВКА И СЕРИЙНАЯ НАСТРОЙКА

11.1.1. ПЕРВИЧНАЯ ПРОШИВКА КОНТРОЛЛЕРОВ

Первичная прошивка выполняется на этапе производства или предпродажной подготовки.

В процессе первичной прошивки:

- в контроллер загружается базовая версия прошивки;
- проверяется корректность запуска системы;

- проверяется работа основных интерфейсов (CAN, RS-485, Ethernet);
 - выполняется базовая самодиагностика.
- Прошивка выполняется **без привязки к конкретному проекту**.

11.1.2. НАЗНАЧЕНИЕ NODE ID И РОЛИ КОНТРОЛЛЕРА

На этапе серийной настройки задаются:

- Node ID контроллера (1...10);
- роль контроллера:
 - MASTER;
 - SLAVE.

Назначенные параметры:

- сохраняются в энергонезависимой памяти;
- используются системой при запуске;
- могут быть изменены только в сервисном режиме.

11.1.3. ПРОВЕРКА АППАРАТНЫХ КАНАЛОВ

Перед отправкой в проект проверяются:

- входы датчиков дверей;
- выходы управления замками;
- кнопка Alarm;
- световая и звуковая индикация.

Проверка выполняется с использованием тестовой конфигурации.

11.1.4. СБРОС К ЗАВОДСКОМУ СОСТОЯНИЮ

Контроллер поддерживает режим сброса:

- конфигурации проекта;
- пользователей WEB;
- журналов событий (опционально).

Сброс возвращает контроллер в состояние готовности к новой установке.

11.2. СЕРВИСНЫЙ РЕЖИМ И ДИАГНОСТИКА

11.2.1. НАЗНАЧЕНИЕ СЕРВИСНОГО РЕЖИМА

Сервисный режим предназначен для:

- диагностики оборудования;
- проверки подключений;
- обслуживания и ремонта;
- локального тестирования дверей.

Сервисный режим **не предназначен для эксплуатации**.

11.2.2. ВОЗМОЖНОСТИ СЕРВИСНОГО РЕЖИМА

В сервисном режиме доступны:

- просмотр состояния входов и выходов;
- тестирование индикации и зуммера;
- проверка реакции датчиков;
- диагностика CAN и RS-485.

При этом:

- логика зависимостей может быть отключена;
- замки не удерживаются в опасных состояниях.

11.2.3. ОГРАНИЧЕНИЯ СЕРВИСНОГО РЕЖИМА

Сервисный режим:

- требует физического доступа к контроллеру;
- недоступен через WEB для обычных пользователей;
- не используется в нормальной эксплуатации.

11.3. ДОКУМЕНТАЦИЯ ДЛЯ МОНТАЖНИКА / ИНТЕГРАТОРА

11.3.1. 1 ДОКУМЕНТАЦИЯ ДЛЯ МОНТАЖНИКА

Документация для монтажника включает:

- схемы подключения дверей;
- требования к кабелям и разъёмам;
- правила подключения замков и датчиков;
- требования к заземлению и питанию.

Документация ориентирована на **корректную установку оборудования**.

11.3.2. ДОКУМЕНТАЦИЯ ДЛЯ ИНТЕГРАТОРА

Документация для интегратора включает:

- описание логики системы;
- правила конфигурирования дверей;
- описание режимов работы;
- описание интерфейсов CAN и RS-485.

Документация ориентирована на **правильную настройку системы под объект**.

11.3.3. ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ

Эксплуатационная документация включает:

- инструкции для операторов;
- описание сигнализации и индикации;
- порядок действий при авариях;
- рекомендации по обслуживанию.

11.4. ВЕРСИОНИРОВАНИЕ И РЕЛИЗНЫЙ ЦИКЛ (FW + WEB)

11.4.1. ВЕРСИОНИРОВАНИЕ ПРОШИВКИ (FW)

Прошивка контроллера имеет версию, включающую:

- номер версии;
- дату выпуска;
- список изменений (changelog).

Версия прошивки:

- отображается в WEB-интерфейсе;
- фиксируется в журнале;
- используется при диагностике.

11.4.2. ВЕРСИОНИРОВАНИЕ WEB-ПРИЛОЖЕНИЯ

WEB-приложение также имеет собственную версию.

Версии FW и WEB:

- могут обновляться независимо;
- должны быть совместимы по версии формата конфигурации.

11.4.3. ПРОЦЕСС ОБНОВЛЕНИЯ

Обновление прошивки и WEB:

- выполняется в сервисном режиме или под контролем администратора; не должно приводить к потере конфигурации;
- не должно нарушать безопасность дверей.

При ошибке обновления:

- система должна остаться в безопасном состоянии.

11.4.4. ПОДДЕРЖКА И СОПРОВОЖДЕНИЕ

В рамках сопровождения:

- ведётся учёт версий;
- анализируются журналы событий;
- выпускаются обновления с исправлениями;
- поддерживается обратная совместимость, где это возможно.

11.5. КЛЮЧЕВОЙ ПРИНЦИП РАЗДЕЛА 11

Система должна быть удобна не только в разработке, но и в производстве, монтаже, эксплуатации и сервисе.